

考虑变物性与径向收缩的药材 热湿传递及烘干时长预测

摘 要

针对圆柱形药材烘干过程中温湿分布不均、材料物性变化和失水收缩问题，建立由固定域向收缩域递进的径向热湿传递模型，计算规定位置的温度与干基含水率，并以全域最大含水率确定烘干时长。

针对问题一，从圆柱环层的热量与水分收支出发，建立常热物性导热方程和含水率相关的非线性扩散方程，中心采用对称边界，表面采用对流交换边界。对烘房实测数据作保形三次插值，使用表面加密有限体积法和隐式 BDF 求解，并以 Bessel–Duhamel 展开独立核验温度。1800 s 时，中心与表面温度分别为 33.5758、36.7879 °C，含水率分别为 2.5500、1.5102 kg/kg，表明预热期升温已传入内部，而失水主要集中在外层。

针对问题二，从初始状态统一采用附录 3 的经验物性，通过含水率对热容量、导热系数的影响及温度对扩散系数的影响构成双向耦合。3 h 时中心温度为 49.8494 °C、含水率为 1.7662 kg/kg。温度趋于均衡并不意味着水分已达到干燥要求，需继续计算非线性水分迁移过程。

针对问题三，将全部内部节点的最大含水率与 0.15 kg/kg 之差定义为事件函数，在前 4 h 观测输入及其后 50 °C、0.05 kg/kg 恒定环境假设下进行连续事件定位。区分临界根与严格低于阈值的报告时刻，结合时空加密结果取烘干时长为 57.4731 h；此时最大含水率为 0.1499993913 kg/kg，位于中心。

针对问题四，利用给定半径轨迹和比例径向收缩假设，由干物质与水质量守恒推导材料坐标方程。坐标变换保留内部扩散的 R^{-2} 因子与表面交换的 R^{-1} 因子，避免对干基含水率错误添加体积压缩项。采用附录 4 物性得到烘干时长 51.0877 h，终时半径为 1.2000 cm，最大含水率为 0.1499989333 kg/kg，计算未超出半径观测范围。

通过解析对照、退化情形、收支与网格加密检验数值实现。第三、四问的细网格临界时间差分别为 0.5851 s、0.1135 s。结果表明，保留径向梯度、非线性物性和移动表面有助于一致地求解四问；所报时长为等效模型下的预测，端面效应、潜热及界面平衡关系仍构成适用限制。

关键词：热湿传递；干基含水率；有限体积法；材料坐标；全域阈值

一 问题重述与分析

1.1 问题重述

热风烘干通过烘房与药材表面的热量、水分交换改变药材内部状态。题给药材近似为长 25 cm、初始半径 2 cm 的圆柱，初始温度 28 °C、干基含水率 2.55 kg/kg。附件 1 记录初期烘房温湿环境，附件 2 记录烘干中的半径变化。本文围绕以下四个任务建立模型。

问题一采用附录 2 参数，计算前 1800 s 的径向温度与含水率分布，在指定七个时刻、五个半径位置列出结果，并按 1 s、0.1 cm 间隔输出完整数据。问题二从初始状态统一采用附录 3 经验物性，计算前 3 h 的温湿分布。问题三在问题二模型基础上确定药材各处含水率低于 0.15 kg/kg 的累计时间，并输出每 6 h 的径向含水率。问题四利用附件 2 的半径轨迹与附录 4 物性，重新计算收缩条件下的烘干时间和含水率分布。

问题一、二的细时间结果分别保存至 `result1.xlsx`、`result2.xlsx`；问题三、四按 60 s 输出至 `result3.xlsx`、`result4.xlsx`。所有结果表保留四位小数，数值推进和达标判定使用未舍入值。

1.2 模型的递进关系

四问的共同核心是内部扩散与表面交换的共同作用。集中参数模型只能描述平均状态，难以直接满足题目对不同径向位置的要求。Hussain 和 Dincer 研究了圆柱湿物体干燥中的二维热湿传递问题 [1]；Kaya 等讨论了圆柱湿物体强制对流干燥的数值建模 [2]。本文保留径向空间分布，以题给物性和边界数据建立可计算的有效模型，不移植其他材料的经验参数。

问题一的热物性为常数，且扩散系数仅依赖含水率，故在不显式考虑潜热的条件下，温度场和水分场可以分别求解。问题二、三的热物性依赖含水率，水分扩散系数同时依赖局部温度，必须联合推进两个场。问题三的新增任务是对全域最大值进行事件定位，而不是利用平均含水率或早期失水斜率外推时长。问题四进一步改变空间域：既要描述材料运动，又要把固定材料坐标的解映射回当前物理半径。

上述任务均为初边值预测问题。烘房条件与半径轨迹在模型中属于输入，不引入题目没有要求的工艺优化目标。四问均从题给初始状态计算，第二问不拼接第一问末态，第四问也不读取第三问的最终状态。

1.3 输入整理与时间延续

附件 1 含 241 组观测，时间为 0 至 14400 s，间隔 60 s。前 1800 s 和前 10800 s 分别包含 31 组、181 组观测。对各问所需观测区间采用保形分段三次 Hermite 插值 (PCHIP) [3]。记相邻观测点为 (t_j, f_j) ，区间长度为 h_j ， $z = (t - t_j)/h_j$ ，则插值表达式为

$$\begin{aligned} f(t) = & (2z^3 - 3z^2 + 1)f_j + (-2z^3 + 3z^2)f_{j+1} \\ & + (z^3 - 2z^2 + z)h_jm_j + (z^3 - z^2)h_jm_{j+1}, \end{aligned} \quad (1)$$

其中 m_j 由保形规则确定， f 取烘房温度 T_a 或水分量 C_a 。式(1)精确通过观测点，不删除小幅波动，也不作全局平滑。问题一的环境输入见图 1。

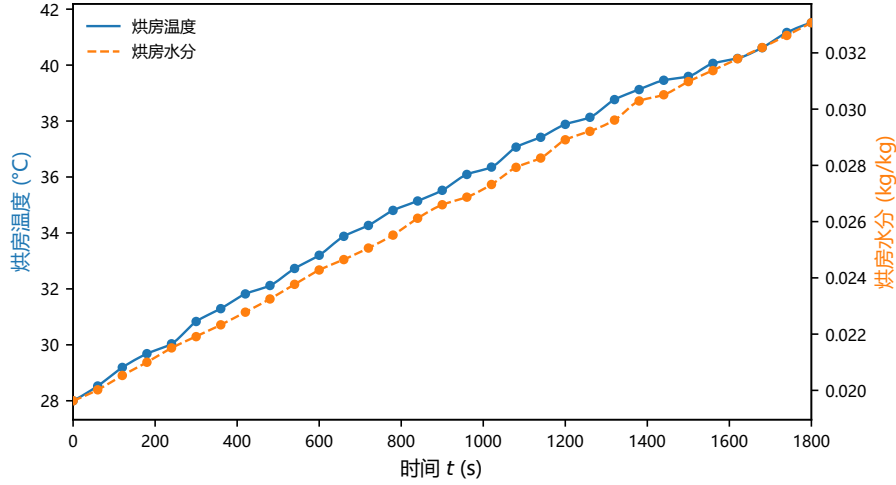


图1 前 30 min 烘房温度和水分量随时间变化。曲线由附件 1 观测插值得到，输入升温不等于药材内部同步升温。

附件 1 没有覆盖完整干燥时段。问题三、四在 $t \leq 14400$ s 时保留全部观测输入，随后采用恒温阶段延续假设

$$T_a(t) = 50\text{ }^{\circ}\text{C}, \quad C_a(t) = 0.05\text{ kg/kg}, \quad t > 14400\text{ s}. \quad (2)$$

式(2)是基于恒温干燥情景的模型输入，不是后期实测数据。切换处保留原观测终值，允许边界输入发生小幅变化，内部温湿状态保持连续。

附件 2 含 145 组半径观测，覆盖 0 至 72 h，间隔 0.5 h，半径从 2 cm 降至 1.198 cm。将半径换算为 m 后使用同类插值构造 $R(t)$ 。仅在数据覆盖区间内计算，72 h 不作为预设的烘干答案。

二 模型假设与符号说明

2.1 模型假设及其用途

- (1) 药材横截面内满足轴对称，材料在周向上均匀，表面交换条件沿周向一致。忽略轴向梯度和端面交换，建立长度中部截面的一维径向模型。该假设用于将传递方程降为径向形式；长时间计算的端面影响作为局限保留。
- (2) 初始温度与干基含水率均匀；问题一至三半径与长度固定。问题四长度仍取 0.25 m，骨架按给定 $R(t)$ 作比例径向收缩，干物质不流失。比例收缩用于由外半径轨迹闭合内部速度场。
- (3) 题给 ρc_p 作为有效体积热容量，采用 Fourier 导热和有效 Fick 扩散描述内部传递。热方程不显式包含汽化潜热、水分携焓及机械功。此处理用于在题给数据下封闭模型，不代表这些机制已被实验排除。
- (4) 沿用 $h = 25\text{ W}/(\text{m}^2 \cdot \text{K})$ 和 $h_m = 8 \times 10^{-7}\text{ m/s}$ 作为表面交换系数。将 $h_m(C_s - C_a)$ 理解为题给信息下的等效水分交换关系。气相与固相的 kg/kg 具有不同干基载体，未将 C_a 直接认定为实测药材平衡含水率。

- (5) 固定域采用均匀、恒定的参考干物质密度；收缩域按干物质守恒更新该密度。它与用于热方程的经验 $\rho(C)$ 分别记账，不强制二者逐点满足湿密度换算关系。

径向近似在问题一中可作尺度说明。初始长度与直径之比为 6.25，两端总面积与侧面积之比为 0.08。附录 2 给出热扩散率 $\alpha = 0.36/(820 \times 2600) = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ ，1800 s 内的传播尺度 $\sqrt{\alpha t} = 0.0174 \text{ m}$ ，小于中截面到端面的距离 0.125 m。这支持短时中截面的径向近似，但面积比和传播尺度都不是后续数日预测的误差上界。

2.2 主要符号

符号	含义	单位
t, r	入炉后的时间、当前径向距离	s, m
$R, R(t), L$	固定半径、随时间变化的半径、长度	m
T, T_a	药材局部温度、烘房温度	°C
T_K	药材局部绝对温度, $T_K = T + 273.15$	K
C, C_a	药材干基含水率、烘房题给水分量	kg/kg
ρ, c_p, k	有效密度、比热容、导热系数	kg/m ³ , J/(kg·K), W/(m·K)
$B = \rho c_p, D$	有效体积热容量、水分扩散系数	J/(m ³ ·K), m ² /s
h, h_m	对流换热系数、有效传质系数	W/(m ² ·K), m/s
s, v	当前体积干物质密度、骨架径向速度	kg/m ³ , m/s
ξ, U, Θ	材料坐标、该坐标下的含水率和温度	1, kg/kg, °C
W_i, w_i	物理域、材料域控制体的径向权重	m ² , 1
M, t_*, t_{rep}	全域最大含水率、临界时刻、报告时刻	kg/kg, s, s

下标 s 表示当前表面， i 表示内部计算节点。正文中的时间公式以 s 为单位，时长结果及表 3 至表 6 换算为 h。

三 问题一：预热阶段的径向热湿传递

3.1 由环层收支建立控制方程

取半径 r 至 $r + dr$ 、长度 L 的圆柱环层，以向外为正方向，Fourier 定律给出热流密度 $q = -kT_r$ 。根据假设 (1) 至 (3)，环层蓄热等于净流入热量，即

$$\rho c_p (2\pi r L dr) T_t = -\partial_r (2\pi r L q) dr.$$

约去公共几何因子，得到

$$\rho c_p T_t = \frac{1}{r} \partial_r (rkT_r), \quad 0 < r < R. \quad (3)$$

问题一取 $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg·K)}$ 、 $k = 0.36 \text{ W/(m·K)}$ ，所以式(3)可写为 $T_t = \alpha(T_{rr} + T_r/r)$ 。

含水率 C 是单位干物质质量所含的水质量。按假设 (5)，令参考干物质密度为常数 s_0 ，水质量浓度为 $s_0 C$ ，相对扩散通量为 $j = -s_0 D(C) C_r$ 。环层水分收支为

$$s_0(2\pi r L \, dr) C_t = -\partial_r(2\pi r L j) \, dr.$$

消去 s_0 与几何因子后得到

$$C_t = \frac{1}{r} \partial_r [r D(C) C_r], \quad D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right). \quad (4)$$

式(4)中 D 的单位为 m^2/s 。其展开形式为

$$C_t = D(C) \left(C_{rr} + \frac{C_r}{r} \right) + D'(C) (C_r)^2, \quad D'(C) = \frac{0.89 D(C)}{C^2}.$$

可见把 D 移到散度算子外会漏掉梯度平方项。实际计算保留式(4)的通量形式。此时热方程不依赖 C 、水分方程不依赖 T ，两场在所选假设下相互独立。

3.2 初始条件与交换边界

题给均匀初值与中心对称条件为

$$T(r, 0) = 28, \text{quad} C(r, 0) = 2.55; \quad T_r(0, t) = C_r(0, t) = 0. \quad (5)$$

由假设 (4)，外表面取 Robin 边界

$$\begin{aligned} -k T_r(R, t) &= h [T_s - T_a(t)], \\ -D(C_s) C_r(R, t) &= h_m [C_s - C_a(t)]. \end{aligned} \quad (6)$$

式(6)保留了药材表面与烘房之间的差异： $T_a > T_s$ 时向外热流为负，表示热量进入药材； $C_s > C_a$ 时水分向外迁移。表面值由方程求解，不能直接设为环境值。式(5)的中心对称条件也不等同于把中心固定在初始状态。

均匀初始水分使表面初始梯度为零，而交换边界在接触烘房后立即产生失水通量，二者在 $t = 0$ 角点不相容。本文保留题给初值，在 $t > 0$ 施加表面交换，求得初始表面层的瞬态发展，并把前 1 s 的表面结果纳入网格检验。

3.3 表面加密网格与有限体积离散

为同时保留题目输出位置和表面薄层，将 $[0, 0.02] \text{ m}$ 分为 20 个宽 0.001 m 的基本区间。第 j 个区间 ($j = 0, \dots, 19$) 再等分为 mb_j 份，其中

$$b_j = \begin{cases} 8, & 0 \leq j \leq 14, \\ 16, & 15 \leq j \leq 17, \\ 32, & j = 18, \\ 64, & j = 19. \end{cases} \quad r_{j,\ell} = 0.001 \left(j + \frac{\ell}{mb_j} \right). \quad (7)$$

去除相邻区间重复端点后, 网格 G_m 共有 $264m + 1$ 个节点。因此 G_2, G_4, G_8, G_{16} 分别有 529、1057、2113、4225 个节点。式(7)使 0、0.1、...、2.0 cm 均为实际节点; 问题一采用 G_8 , 最小表面间距为 1.953125×10^{-6} m。

设全部节点依次为 r_i ($i = 0, \dots, N$), 内部面为 $r_{i+1/2} = (r_i + r_{i+1})/2$ 。控制体左右端点为 a_i, b_i^* , 中心端取 0、表面端取 R , 定义

$$W_i = \int_{a_i}^{b_i^*} r \, dr = \frac{(b_i^*)^2 - a_i^2}{2}, \quad \sum_{i=0}^N W_i = \frac{R^2}{2}. \quad (8)$$

真实控制体体积为 $2\pi L W_i$ 。在内部面上采用相邻节点的调和平均系数, 得到已乘半径的向外通量

$$\begin{aligned} F_{i+1/2}^T &= -r_{i+1/2} k \frac{T_{i+1} - T_i}{r_{i+1} - r_i}, \\ F_{i+1/2}^C &= -r_{i+1/2} \frac{2D_i D_{i+1}}{D_i + D_{i+1}} \frac{C_{i+1} - C_i}{r_{i+1} - r_i}. \end{aligned} \quad (9)$$

中心外侧虚拟面通量取零, 表面通量直接取 $Rh(T_N - T_a)$ 、 $Rh_m(C_N - C_a)$ 。对式(3)和式(4)作控制体积分, 并用节点状态近似体平均, 得到

$$\rho c_p W_i \dot{T}_i = F_{i-1/2}^T - F_{i+1/2}^T, \quad W_i \dot{C}_i = F_{i-1/2}^C - F_{i+1/2}^C. \quad (10)$$

式(8)至式(10)使同一内部面通量在相邻控制体中以相反符号出现, 求和后仅保留外边界通量。

中心不直接计算 $1/r$ 。若第一个节点间距为 $\delta = r_1 - r_0$, 则 $W_0 = \delta^2/8$, 由式(9)可得

$$\dot{T}_0 = \frac{4\alpha}{\delta^2} (T_1 - T_0), \quad \dot{C}_0 = \frac{4D_{1/2}}{\delta^2} (C_1 - C_0).$$

圆柱中心系数为 4, 区别于平板中心离散。表面控制体则满足

$$\begin{aligned} \rho c_p W_N \dot{T}_N &= \frac{r_{N-1/2} k}{r_N - r_{N-1}} (T_{N-1} - T_N) + Rh(T_a - T_N), \\ W_N \dot{C}_N &= \frac{r_{N-1/2} D_{N-1/2}}{r_N - r_{N-1}} (C_{N-1} - C_N) - Rh_m(C_N - C_a). \end{aligned}$$

表面节点保留蓄积项, 避免用靠近表面的内部节点替代真实表面。

3.4 隐式推进与 Bessel–Duhamel 核验

空间离散后形成常微分方程 $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$ 。表面间距较小使系统具有刚性, 因此采用隐式变阶 BDF 及稀疏雅可比结构求解 [4]。以等步长二阶公式说明其隐式形式,

$$\frac{3\mathbf{y}^{n+1} - 4\mathbf{y}^n + \mathbf{y}^{n-1}}{2\Delta t} = \mathbf{f}(t_{n+1}, \mathbf{y}^{n+1}).$$

实际实现采用变步长、变阶积分, 而非固定使用上式。非线性含水率场在迭代中更新 D 。问题一相对容差为 10^{-10} , 温度与含水率绝对容差分别为 10^{-10} 、 10^{-12} , 最大步长 0.5 s; 整秒仅为输出时刻。

常热物性允许独立构造温度参考解。先考虑环境恒为 T_f 的情况，令 $u = T - T_f = X(r)G(t)$ ，分离变量后径向方程为

$$r^2 X'' + rX' + \lambda^2 \frac{r^2}{R^2} X = 0.$$

中心有界性排除第二类 Bessel 函数，故 $X(r) = J_0(\lambda r/R)$ 。表面 Robin 条件给出特征方程

$$\lambda_n J_1(\lambda_n) = \text{Bi } J_0(\lambda_n), \quad \text{Bi} = \frac{hR}{k}. \quad (11)$$

以 r 为权展开均匀初值，模态系数为

$$A_n = \frac{\int_0^1 x J_0(\lambda_n x) dx}{\int_0^1 x J_0^2(\lambda_n x) dx} = \frac{2J_1(\lambda_n)}{\lambda_n [J_0^2(\lambda_n) + J_1^2(\lambda_n)]}.$$

由式(11)得恒定环境解

$$T(r, t) = T_f + (28 - T_f) \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) e^{-\beta_n t}, \quad \beta_n = \alpha \lambda_n^2 / R^2.$$

对 PCHIP 环境应用 Duhamel 叠加，由 $T_a(0) = 28$ 得到

$$T(r, t) = T_a(t) - \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) I_n(t), \quad (12)$$

$$I_n(t) = \int_0^t e^{-\beta_n(t-\tau)} T_a'(\tau) d\tau, \quad \dot{I}_n = -\beta_n I_n + T_a'(t), \quad I_n(0) = 0.$$

式(12)通过独立模态 ODE 计算。级数从 480 项增至 960 项的最大变化为 $2.7632 \times 10^{-9} \text{ } ^\circ\text{C}$ ，960 项参考解与有限体积输出的最大温度差为 $2.1547 \times 10^{-7} \text{ } ^\circ\text{C}$ 。水分仅在冻结 D 的测试问题中使用同类解析展开，实际结果仍采用式(4)。

3.5 计算结果与分析

表 1、表 2 给出题目要求的温度和含水率，图 2 展示径向分布。至 1800 s，表面温度 $36.7879 \text{ } ^\circ\text{C}$ 仍低于环境 $41.5130 \text{ } ^\circ\text{C}$ ，中心升至 $33.5758 \text{ } ^\circ\text{C}$ ，说明有限表面换热与内部导热共同形成温度滞后。

末时刻中心含水率未舍入值为 2.5499924 kg/kg ，四位显示为 2.5500 ；表面已降至 1.5102 kg/kg ，形成明显水分梯度。这不是把中心设为恒定水源的结果，而是初期水分尚未充分向外迁移。实际 D 由初始 4.9377×10^{-9} 降至局部最小 $3.8830 \times 10^{-9} \text{ m}^2/\text{s}$ ；与冻结初始 D 的解相比，含水率最大差达到 0.0419 kg/kg ，故冻结扩散系数不能替代本问正式结果。

表 1 30 分钟内药材的温度 (°C)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	28.0000	28.0003	28.0038	28.0319	28.1795
300	28.0406	28.0634	28.1513	28.3673	28.8486
600	28.4533	28.5361	28.8039	29.3155	30.1641
900	29.3243	29.4583	29.8757	30.6164	31.7295
1200	30.5429	30.7099	31.2225	32.1130	33.4295
1500	31.9960	32.1871	32.7667	33.7476	35.1211
1800	33.5758	33.7724	34.3642	35.3626	36.7879

表 2 30 分钟内药材的水分浓度 (kg/kg)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

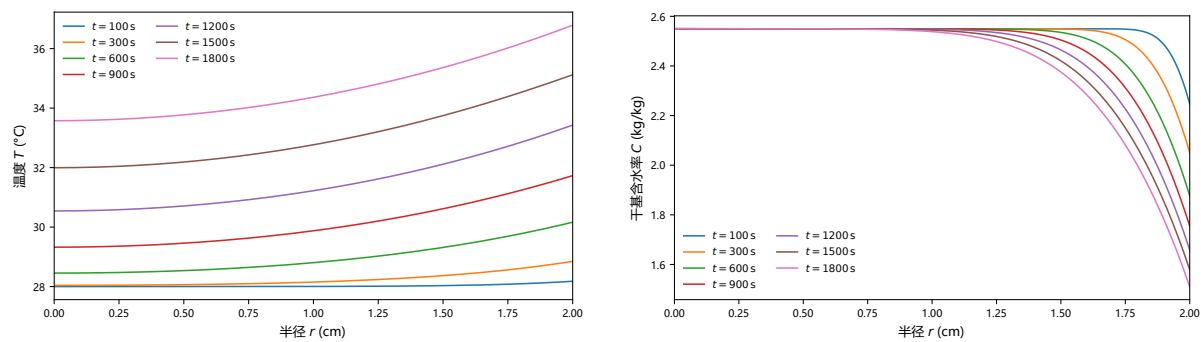


图 2 预热阶段温度与含水率径向剖面。外层先升温、先失水，中心含水率在 30 min 内变化很小；不同曲线对应题定输出时刻。

四 问题二：变物性条件下的热湿耦合

4.1 统一物性与耦合方程

问题二要求全程统一使用附录 3 经验式。因此保留问题一的固定几何、原始初值与边界结构，从 $t = 0$ 重新积分。局部物性为

$$\begin{aligned}\rho(C) &= 650 + 128C, & c_p(C) &= 1450 + 2736 \frac{C}{1+C}, \\ k(C) &= 0.21 + 0.38 \frac{C}{1+C}, \\ D(C, T) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T + 273.15}\right].\end{aligned}\tag{13}$$

其中温度场仍按 $^{\circ}\text{C}$ 表示，扩散系数指数仅使用药材局部绝对温度 $T + 273.15$ 。将式(13)代入环层模型，得到

$$\begin{aligned}B(C)T_t &= \frac{1}{r} \partial_r [rk(C)T_r], \\ C_t &= \frac{1}{r} \partial_r [rD(C, T)C_r], \quad B(C) = \rho(C)c_p(C).\end{aligned}\tag{14}$$

初始条件沿用式(5)，表面边界沿用式(6)并将 k, D 替换为当前表面物性。题目附录 3 未另给交换系数，按假设 (4) 继续使用前述 h, h_m 。

式(14)中的耦合有两个方向： C 改变 B 和 k ，从而改变温度传播； T 改变 D ，从而改变水分迁移。进一步展开水分散度项可得

$$C_t = D \left(C_{rr} + \frac{C_r}{r} \right) + D_C (C_r)^2 + D_T T_r C_r.$$

这里的 $D_T T_r C_r$ 由空间变化的扩散系数产生，不是另加的热扩散经验项。采用原散度形式离散，可同时保留上述非线性项。

4.2 有效热容量与干物质记账

问题二沿用固定参考干物质骨架，水分方程中的参考干密度保持常数。经验 $\rho(C)$ 仅用于构造 $B(C)$ ，不能在水分守恒中又取 $s = \rho(C)/(1 + C)$ 并随时间更新，否则会与固定体积中干物质质量不变的假设冲突。

同样， $B(C)T_t$ 不能直接替换为 $\partial_t [B(C)T]$ 。取参考温度 $T_0 = 28^{\circ}\text{C}$ ，有

$$\partial_t [B(C)(T - T_0)] = B(C)T_t + B'(C)(T - T_0)C_t.\tag{15}$$

式(15)中的第二项反映所定义状态量随组成变化而改变。本文采用温度形式的有效热方程，在收支检验中保留该修正项；不据此宣称已经描述完整多相系统的能量交换。

4.3 联合求解与结果

沿用问题一的环形控制体，热导率与扩散系数均取相邻节点的调和平均。将 $\mathbf{y} = (T_0, \dots, T_N, C_0, \dots, C_N)^T$ 作为联合状态，在每次迭代中按局部温湿状态更新物性。每个

节点只与相邻节点的两个场相联系，利用这一稀疏结构进行 BDF 积分。生产网格采用 G_4 ，相对容差 10^{-9} ，温度、含水率绝对容差分别为 10^{-8} 、 10^{-10} ，最大步长 1 s。

表 3 和表 4 给出每 0.5 h 的结果，图 3 展示相应径向剖面。3 h 时中心温度为 49.8494 °C，已接近烘房温度，但中心含水率仍为 1.7662 kg/kg，远高于 0.15 kg/kg 的干燥要求。由此可见，温度接近稳定不能作为停止烘干的条件。

表 3 3 小时内药材的温度 (°C)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	32.1896	32.3824	32.9660	33.9608	35.4132
1	40.3822	40.5542	41.0609	41.8783	43.0000
1.5	45.8471	45.9351	46.1932	46.6055	47.1397
2	48.4503	48.4881	48.5985	48.7736	49.0033
2.5	49.4670	49.4793	49.5136	49.5645	49.6617
3	49.8494	49.8553	49.8746	49.9096	49.9645

表 4 3 小时内药材的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3	1.7662	1.7165	1.5702	1.3333	1.0081

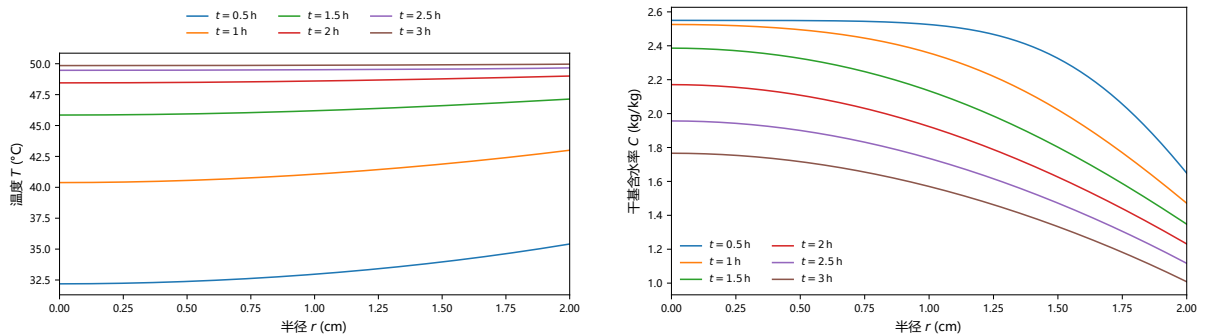


图 3 前 3 h 变物性模型的温度与含水率剖面。温度逐渐接近烘房水平，水分仍保持中心高、表面低的径向差异。

两问在相同时间的结果差异源于采用了不同物性。问题二不能在 1800 s 处读取问题一末态后才切换参数，否则前 0.5 h 的过程不再满足题目要求的统一经验式。全部前 3 h 结果按整秒、0.1 cm 输出，共包含 453600 个温度与含水率数值。

4.4 升温与失水对扩散系数的竞争

由式(13)直接求导可得

$$\frac{\partial \ln D}{\partial C} = \frac{0.45}{C^2} > 0, \quad \frac{\partial \ln D}{\partial T} = \frac{3850}{(T + 273.15)^2} > 0. \quad (16)$$

升温倾向于增大扩散系数，失水则使其减小，两种作用沿实际轨迹竞争。固定 $C = 2.55$ 时，由 28 升至 50 °C 使 D 增至约 2.388 倍；固定 $T = 50$ °C 时， C 从 2.55 降至 0.15 使 D 降至原值约 5.94%。这些是对经验式的条件性代入，不是独立温度控制实验。

数值结果中前 3 h 局部 D 约在 5.5476×10^{-9} 至 $1.2522 \times 10^{-8} \text{ m}^2/\text{s}$ 之间变化。因各位置升温 and 失水程度不同，不能用单一常数代表整个时空场。与冻结初始 D 的对照相比，含水率最大差为 0.3981 kg/kg，也说明后续时长应由联合非线性模型求得。

五 问题三：全域达标条件与烘干时间

5.1 从空间约束到事件函数

沿用问题二的固定域耦合方程，前 4 h 使用观测环境，后期使用式(2)。为表达题目“各处低于 0.15 kg/kg”的要求，定义

$$M(t) = \max_{0 \leq r \leq R} C(r, t), \quad g(t) = M(t) - 0.15, \quad t_* = \inf\{t \geq 0 : M(t) < 0.15\}. \quad (17)$$

在连续向下穿越的情况下，式(17)定义的临界时刻满足 $M(t_*) = 0.15$ 。严格低于阈值的时间集合通常没有最小元素，因此另给稍晚的报告时刻 t_{rep} ，并直接复核未舍入的 $M(t_{\text{rep}}) < 0.15$ 。

计算中使用全部内部节点的最大值 $M_h(t) = \max_i C_i(t)$ ，记录最大值位置并检查径向分布。只有数值剖面支持中心最湿时，才用中心曲线解释控制位置；不事先用中心值替代全域搜索。题目要求的 5 个表格位置和 21 个 Excel 位置均是输出采样，不承担全域最大值检查。

5.2 连续事件定位与严格裕量

采用 G_8 网格联合积分；相对容差 10^{-9} ，温度、含水率绝对容差分别为 10^{-8} 、 10^{-10} 。前 4 h 最大步长 2 s，之后 60 s，在输入切换处分段接续。搜索 g 由正到负的括号后，在事件附近以最大 0.5 s 步长局部精算，再用带括号求根定位临界时刻。

为避免小幅离散误差改变严格不等式，在同一物理时刻 $t_{*,h}$ 比较生产解、细网格解和时间收紧解。令

$$\varepsilon_C = 2 \max \{|M_h(t_{*,h}) - M_{h/2}(t_{*,h})|, |M_h(t_{*,h}) - M_{h,\text{tight}}(t_{*,h})|\} + 10^{-8}. \quad (18)$$

本问式(18)给出 $\varepsilon_C = 3.5088 \times 10^{-7} \text{ kg/kg}$ 。先寻找 $M_h < 0.15 - \varepsilon_C$ 的时刻，再向上选择整秒并向上保留四位小时，最终在三组解上复核。该裕量反映已完成的加密比较，是经验数值裕量，不是连续 PDE 解的严格误差界。

生产网格临界时刻为 57.4725195022 h，最终报告

$$t_{\text{rep},3} = 57.4731 \text{ h} = 206903.16 \text{ s.} \quad (19)$$

式(19)对应的生产解最大含水率为 0.1499993913 kg/kg；细网格和时间收紧解在同一时刻的最大值分别为 0.1499992209、0.1499993913 kg/kg，均严格低于 0.15。最大值位于中心，保存剖面从中心向外不增。

5.3 径向分布与后期干燥特征

表 5 给出每 6 h 及最终报告时刻的水分浓度，图 4 展示最大含水率随时间的变化，图 5 展示径向剖面。

表 5 固定半径模型药材烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4731	0.1500	0.1477	0.1402	0.1249	0.0527

注：终时中心未舍入值为 0.1499993913 kg/kg，严格低于 0.15；0.1500 为四位显示。

例如，18 h 时表面已降至 0.0845 kg/kg，而中心仍为 0.2988 kg/kg；48 h 时中心仍为 0.1618 kg/kg，尚未达标。这表明单凭表面测量或平均含水率判定结束会遗漏内部湿区。后期温度接近恒定后，失水使局部扩散系数继续下降，因而不能把早期失水速率线性外推到阈值。

表 5 末行中心显示 0.1500，是 0.1499993913 保留四位小数的结果；达标判断使用未舍入值，不以显示值是否为 0.1499 为准。完整结果文件从 60 s 起每 60 s 保存一次，另追加 206903.16 s 的准确终时，不将最近整分钟冒充最终报告时刻。

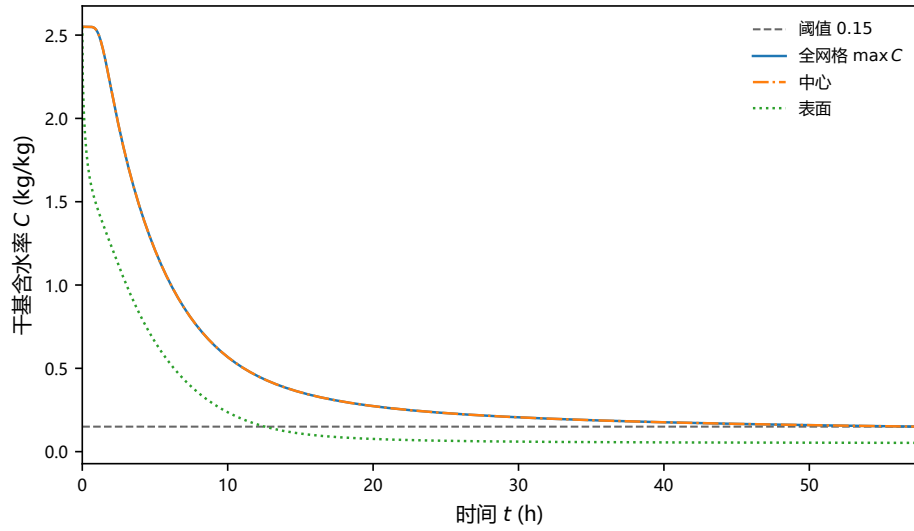


图4 全域最大含水率逐步接近干燥阈值。后期下降速度减慢，表面先达标不能代表内部已完成干燥。

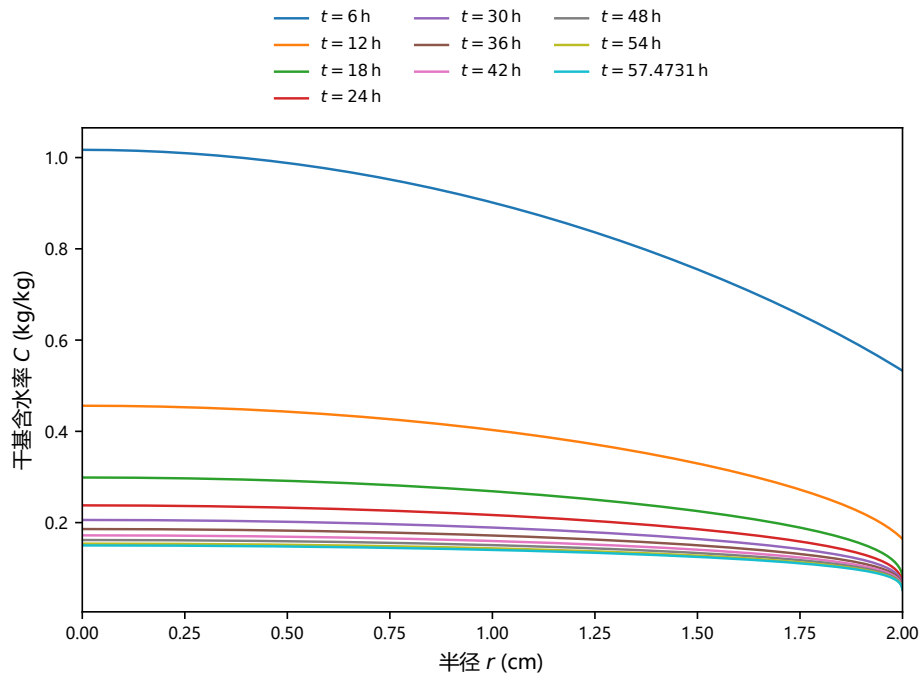


图5 固定半径模型每6 h及终时的含水率剖面。中心保持较高含水率，最终由中心附近控制全域达标。

六 问题四：给定径向收缩的移动边界模型

6.1 由干物质守恒推导材料运动

问题四同时给出了半径变化和一套新的经验物性，不能仅把问题三固定半径替换为末时半径。Brasiello 等研究了食品干燥的非等温移动边界模型 [5]，将收缩与传热传质联系起来。本文直接利用附件 2 构造已知 $R(t)$ ，按假设 (2) 取比例径向收缩速度

$$v(r, t) = \frac{\dot{R}(t)}{R(t)} r. \quad (20)$$

外半径数据本身不能唯一确定内部运动，式(20)是用于闭合速度场的假设。

设 $s(r, t)$ 为单位当前体积的干物质质量， j 为相对固体骨架的向外水质量通量。干物质不流失时，干物质与水质量分别满足

$$\begin{aligned} s_t + \frac{1}{r} \partial_r (r s v) &= 0, \\ \partial_t (s C) + \frac{1}{r} \partial_r [r (s C v + j)] &= 0, \quad j = -s D(C, T) C_r. \end{aligned} \quad (21)$$

将第二式展开，减去第一式乘以 C ，得到

$$C_t + v C_r = \frac{1}{s r} \partial_r (r s D C_r). \quad (22)$$

初始 s 均匀时，将式(20)代入干物质方程，可得

$$\dot{s} + 2 \frac{\dot{R}}{R} s = 0, \quad s(t) = s_0 \left[\frac{R_0}{R(t)} \right]^2. \quad (23)$$

因此 s 保持空间均匀，可从式(22)的空间散度中约去。式(21)至式(23)说明，纯收缩不会改变同一材料单元的水质量与干物质质量之比；对于基含水率直接添加 $-C \nabla \cdot v$ 项会重复计算几何变化。

6.2 材料坐标变换与控制方程

定义固定区间上的材料坐标和状态

$$\xi = \frac{r}{R(t)}, \quad U(\xi, t) = C(R(t)\xi, t), \quad \Theta(\xi, t) = T(R(t)\xi, t), \quad 0 \leq \xi \leq 1. \quad (24)$$

由链式法则， $U_t = C_t + \xi \dot{R} C_r = C_t + v C_r$ ，且 $C_r = U_\xi / R$ 。式(22)于是变为

$$\begin{aligned} U_t &= \frac{1}{R(t)^2 \xi} \partial_\xi [\xi D(U, \Theta) U_\xi], \\ B(U) \Theta_t &= \frac{1}{R(t)^2 \xi} \partial_\xi [\xi k(U) \Theta_\xi]. \end{aligned} \quad (25)$$

材料运动与坐标网格同步，使显式对流项抵消。式(25)的热方程是前述有效传热近似在材料坐标下的表达，不由水质量守恒自动推出，也未包含收缩机械功或完整水分携焓。

本问全部物性改为附录 4 经验式：

$$\begin{aligned}\rho(U) &= 760 + 90U, & c_p(U) &= 1850 + 2150 \frac{U}{1+U}, \\ k(U) &= 0.12 + 0.20 \frac{U}{1+U}, \\ D(U, \Theta) &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{U}\right) \exp\left[-\frac{3850}{\Theta + 273.15}\right].\end{aligned}\quad (26)$$

将式(26)与 $R(0) = 0.02 \text{ m}$ 、 $U(\xi, 0) = 2.55$ 、 $\Theta(\xi, 0) = 28$ 共同使用。中心仍取 $U_\xi(0, t) = \Theta_\xi(0, t) = 0$ ；外表面为

$$-\frac{D_s}{R}U_\xi(1, t) = h_m(U_s - C_a), \quad -\frac{k_s}{R}\Theta_\xi(1, t) = h(\Theta_s - T_a). \quad (27)$$

式(24)至式(27)共同保留了内部扩散的 R^{-2} 与边界梯度的 R^{-1} ，不能只改控制方程而遗漏边界尺度。

6.3 固定材料网格上的求解与物理输出

将问题一的 G_8 节点除以 R_0 得到固定材料节点 ξ_i ，令内部面 $\xi_{i+1/2} = (\xi_i + \xi_{i+1})/2$ ，径向权重 $w_i = (\xi_{i+1/2}^2 - \xi_{i-1/2}^2)/2$ ，端点按 0 和 1 截取。记

$$\hat{F}_{i+1/2}^C = -\xi_{i+1/2} D_{i+1/2} \frac{U_{i+1} - U_i}{\xi_{i+1} - \xi_i}, \quad \hat{F}_{i+1/2}^T = -\xi_{i+1/2} k_{i+1/2} \frac{\Theta_{i+1} - \Theta_i}{\xi_{i+1} - \xi_i}.$$

表面通量分别取 $Rh_m(U_N - C_a)$ 与 $Rh(\Theta_N - T_a)$ 。半离散方程为

$$R^2 w_i \dot{U}_i = \hat{F}_{i-1/2}^C - \hat{F}_{i+1/2}^C, \quad R^2 w_i B_i \dot{\Theta}_i = \hat{F}_{i-1/2}^T - \hat{F}_{i+1/2}^T. \quad (28)$$

式(28)使用 2113 个材料节点，BDF 相对容差为 10^{-8} ，温度与含水率绝对容差分别为 10^{-8} 、 10^{-10} ；前 4 h 最大步长 10 s，其后 60 s。环境切换处分段，过程状态连续。以全部材料节点的最大含水率定位事件，并在细网格和时间收紧解上核验同一报告时刻。

题目要求的是当前物理距离。对每个 $r_j \leq R(t)$ ，先求 $\xi_j = r_j/R(t)$ ，再对材料网格上的 U 作保形插值；固定 $r_j > R(t)$ 的位置已在药材外，单元格留空。表面始终使用 $\xi = 1$ 的实际节点并单列，不能用固定的 2 cm 列表示移动表面。

6.4 烘干时长与径向含水率

生产解临界时刻为 51.0871229810 h。程序在生产、空间加密和时间收紧解的最晚临界时刻之后增加 2 s，再向上选择四位小时报告值，并复核三组未舍入最大值。最终得到

$$t_{\text{rep},4} = 51.0877 \text{ h} = 183915.72 \text{ s}. \quad (29)$$

式(29)对应最大含水率为 0.1499989333 kg/kg，最大值位于中心，当前半径为 1.2000 cm，位于附件 2 的 72 h 覆盖范围内，无需半径外推。表 6 为题目要求的结果，图 6 同时展示含水率历程与当前物理半径剖面。

表 6 收缩模型药材烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm					药材表面	$R(t)/\text{cm}$
	0	0.5	1.0	1.5	2.0		
6	1.7188	1.5370	1.0221			0.4205	1.3740
12	0.7374	0.6544	0.4082			0.1669	1.2480
18	0.4086	0.3686	0.2397			0.0892	1.2140
24	0.2851	0.2610	0.1790			0.0673	1.2040
30	0.2263	0.2094	0.1493			0.0595	1.2010
36	0.1928	0.1796	0.1316			0.0560	1.2000
42	0.1712	0.1604	0.1200			0.0541	1.2000
48	0.1561	0.1468	0.1116			0.0530	1.2000
51.0877	0.1500	0.1413	0.1081			0.0526	1.2000

注：域外位置留空，药材表面对应当前 $R(t)$ 。终时中心未舍入值为 0.1499989333 kg/kg。

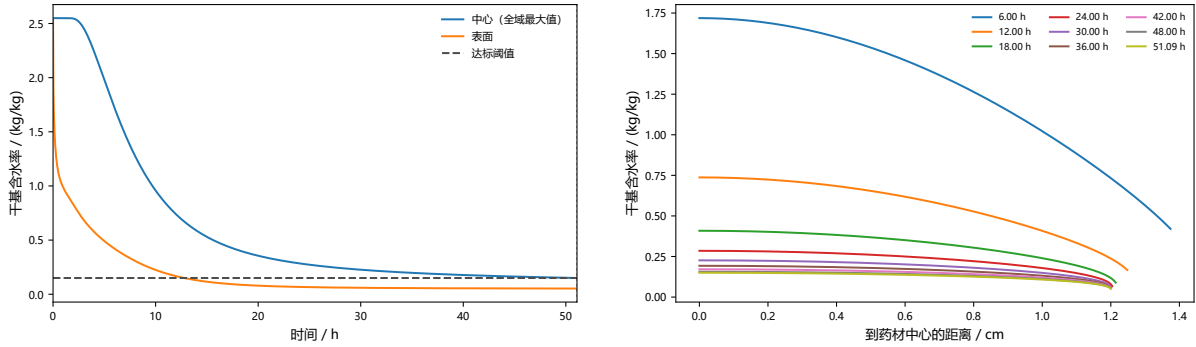


图 6 收缩模型的含水率变化。左图为中心和表面历程；右图以当前物理半径绘制剖面，曲线右端随收缩移动，始终对应真实表面。

表 6 各列含水率单位均为 kg/kg，空白表示该固定距离已超出当时半径；附加的 $R(t)$ 列用于明确表面位置。终时中心四位显示为 0.1500，与问题三相同，应结合未舍入最大值理解严格达标条件。

第四问比第三问的报告时长减少 6.3854 h，约为第三问的 11.11%。这个差异同时包含附录 3 变为附录 4 以及几何收缩两项改变，不能据此认定收缩单独使时长减少 11.11%。若要分离二者作用，需要另作相同物性下固定域与收缩域的对照，本文不将尚未完成的对照写成结论。

6.5 收缩过程的干基收支

按式(23)，干物质总量 $M_d = s_0 \pi R_0^2 L$ 保持不变，水质量满足

$$\frac{M_w}{M_d} = 2 \int_0^1 U \xi \, d\xi, \quad \frac{d}{dt} \left(\frac{M_w}{M_d} \right) = -\frac{2h_m}{R(t)} [U_s - C_a(t)]. \quad (30)$$

式(30)由材料域方程积分并使用表面条件得到，可不指定未知 s_0 而检验归一化水分收支。关闭边界交换时，均匀初始含水率在纯收缩下保持不变；令 R 为常数时，模型退化为采用同一附录 4 物性的固定域方程。两种退化测试均与现有程序结果一致。

七 模型检验与误差分析

7.1 水分收支与有效热方程检验

固定域对半离散方程求和，内部通量相消，得到

$$\sum_i W_i [C_i(t) - C_i(0)] = -Rh_m \int_0^t [C_s(\tau) - C_a(\tau)] d\tau. \quad (31)$$

式(31)以统一参考干密度归一化，未乘干密度和长度时不能直接标为失水质量 kg 。收缩域则使用式(30)，累计表面通量在 BDF 接受步区间上积分。

问题一热容量恒定，总有效蓄热增量应与累计对流输入一致。问题二、三则令

$$E(t) = 2\pi L \sum_i W_i B(C_i)(T_i - T_0),$$

由式(15)推导

$$\begin{aligned} E(t) - E(0) - 2\pi L \int_0^t \sum_i W_i B'(C_i)(T_i - T_0) \dot{C}_i d\tau \\ = 2\pi L R h \int_0^t (T_a - T_s) d\tau. \end{aligned} \quad (32)$$

式(32)中的组成修正项不能省略。相对残差以收支两侧之差的绝对值除以相应累计交换量的尺度计算；接近零时设正的小量下限。问题三在连续解各接受步上采用 4 点、8 点 Gauss 积分对照，避免用 60 s 输出样本的粗梯形积分代替高精度轨迹积分。

现有计算的收支残差见表 7。第四问的热量检查限定为加权半离散热方程残差及温度收敛，不套用固定几何下的累计储量公式。残差较小支持离散通量与所选方程的一致

表 7 各问收支检验：相对残差仅针对所选有效模型

问题	水分相对残差	有效热收支相对残差	检验范围
一	2.4254×10^{-11}	2.0484×10^{-9}	0 至 1800 s
二	4.312×10^{-7}	3.565×10^{-8}	0 至 10800 s
三	4.0221×10^{-11}	3.6640×10^{-8}	至报告时刻
四	4.0558×10^{-9}	不作同类累计比较	生产计算至 72 h

性，不验证潜热、相平衡或端面简化的真实物理误差。

7.2 空间与时间加密

对固定域共享节点、收缩域共享材料节点，在相同物理时刻比较温度和含水率。每个比较量定义为 $\max_{i,j} |u_i^{(a)}(t_j) - u_i^{(b)}(t_j)|$ 。表 8 列出各问主要空间、时间比较值，温度和

表 8 主要网格与时间加密差异

问题	空间比较	ΔT_{space} °C	ΔC_{space} kg/kg	ΔT_{time} °C	ΔC_{time} kg/kg
一	G_4/G_8	5.895×10^{-7}	4.564×10^{-6}	2.874×10^{-8}	1.179×10^{-9}
二	G_2/G_4	2.452×10^{-6}	1.493×10^{-5}	4.108×10^{-7}	2.381×10^{-8}
三	G_8/G_{16}	9.545×10^{-7}	1.537×10^{-5}	1.002×10^{-6}	7.716×10^{-9}
四	G_8/G_{16}	1.012×10^{-5}	8.681×10^{-7}	1.188×10^{-5}	6.383×10^{-8}

含水率分别按自身单位评价，不将二者合成无量纲误差。第一、二问比较全部规定输出点；第三、四问还比较内部共享节点。第四问共享节点比较覆盖用于验证的 72 h 轨迹。上述数值是相邻计算配置之间的差，不是已知真解误差。第二问另有 G_4/G_8 独立复核，温度、含水率最大差分别为 6.244×10^{-7} °C、 3.732×10^{-6} kg/kg。

第三问的 G_4/G_8 内部水分最大差为 6.2693×10^{-5} kg/kg，超过所取 2×10^{-5} 的数值比较目标，因此采用 G_8 生产。第四问 G_4/G_8 在 36 h、1.0 cm 处跨越四位舍入边界，采用 G_8 并以 G_{16} 复核后，题定表格有效单元格四位稳定。可见时长稳定与表格显示稳定应分别检查。

临界时间对照见表 9。第三问空间加密临界时间差为 0.585132 s，时间收紧差为 1.7506×10^{-5} s；第四问对应为 0.113486 s 和 0.002883 s。六张题定结果表在相应空间及时间检验下四位一致，但部分 Excel 单元格可能恰处于舍入边界，不能据此声称所有输出的第四位已完全冻结。

表 9 不同计算配置下的临界烘干时间 (h)

配置	问题三	问题四
G_4	57.4731739851	51.0872487284
G_8	57.4725195022	51.0871229810
G_{16}	57.4723569654	51.0870914570
G_8 时间收紧	57.4725194974	51.0871221801

7.3 独立对照与输出一致性

问题一以独立 Bessel–Duhamel 展开核验温度，并将冻结扩散系数的水分数值解与同类解析解比较，最大差为 1.5090×10^{-6} kg/kg。问题二在短时制造解测试中保留变系数导数项，温度和含水率最大误差分别为 6.0949×10^{-8} °C 和 8.9808×10^{-9} kg/kg。制造解用于检验实现，不是药材实验数据。

第三问与第二问在 1800、3600、7200、9000 s 的重叠输出比较中，温度、水分最大差分别为 4.66×10^{-7} °C 和 9.12×10^{-7} kg/kg，支持独立起算的两个过程在共享区间内一致。第四问则采用同一附录 4 物性的固定半径退化检查及关闭边界交换的纯收缩检查，不把附录 3 结果直接作为附录 4 的退化参照。

输出环节保持源数组、论文表格和 Excel 来自同一状态。数值非有限、含水率越界、

必需收支或收敛检查失败时，程序阻断正式文件写入。第三问另存准确终时，第四问逐行核对当前半径、域外空白与移动表面。这些检查服务于结果可追溯性，不替代模型的物理验证。

7.4 局部敏感性与适用边界

式(16)给出扩散系数对状态变量的局部敏感性。在 $T = 50\text{ }^{\circ}\text{C}$ 、 $C = 0.15$ 时，问题三有

$$\frac{\partial \ln D}{\partial T} \approx 0.0369\text{ K}^{-1}, \quad \frac{\partial \ln D}{\partial C} = 20\text{ (kg/kg)}^{-1}.$$

因而在固定其他状态的一阶近似下，升温 $1\text{ }^{\circ}\text{C}$ 对应局部 D 约 3.69% 的相对变化；该结论不能直接等同于烘干时长变化 3.69% 。问题四对应 $\partial \ln D / \partial U = 0.30/U^2$ ，在阈值处为 $13.3333\text{ (kg/kg)}^{-1}$ 。材料域扩散系数前的几何因子满足 $\partial \ln(R^{-2}) / \partial \ln R = -2$ ，可说明局部方程中的几何敏感方向，不能代替全过程几何对照。

若事件附近最大值位置不变且 $M'(t_*) \neq 0$ ，一阶扰动给出

$$\delta t_* \approx -\frac{\delta M(t_*)}{M'(t_*)}. \quad (33)$$

式(33)说明后期曲线越平缓，相同含水率误差可能造成越大的时间偏差，因而必须直接检验事件时间收敛。本文完成的是解析局部敏感性和数值配置加密；未实施后期环境或经验参数的独立扰动实验，不对其全局稳健性作量化保证。

八 模型评价与结论

8.1 模型特点

本文围绕同一环层通量结构依次处理常热物性、双向物性耦合、全域阈值和径向收缩，四问的变量含义、初值与输出口径保持一致。有限体积形式便于保留变系数散度、处理圆柱中心并检查全局收支；表面加密网格兼顾薄层分辨与题定输出位置。温度解析展开、制造解和退化测试从不同角度检查实现，降低单一验证方式的局限。

达标计算区分临界根、严格不等式与四位显示，避免将输出整分钟或表格中的 0.1500 误作未经核验的终止条件。收缩问题则从两类质量守恒出发引入材料坐标，明确干基含水率与体积水浓度的区别，且按当前物理距离输出结果。

8.2 局限与可推广方向

一维模型忽略端面交换，短时中截面的尺度分析不能证明数日预测的端部误差很小。若补充轴向输出或端面条件，可进一步建立轴对称二维模型，并对温度、最大含水率位置和达标时长重新核验。

等效水分边界缺少独立吸附等温关系，热方程也未显式描述汽化潜热和水分携焓。更完整的界面模型需要药材平衡含水率、相变和气固传质数据。仅凭当前方程的收支残差，不能证明这些机制对真实预测可忽略。

经验密度与几何干物质密度分开使用，是本模型的一项有效近似。以问题四为例，若同时把 $\rho(U)$ 视为真实湿密度，则初始干密度为 $(760 + 90 \times 2.55)/(1 + 2.55) \approx 278.73$

kg/m^3 。72 h 半径对应的体积比为 $(1.198/2)^2 = 0.358801$ ，固定长度且干物质守恒要求平均干密度约 776.84 kg/m^3 ；但对任意 $U \geq 0$ ，

$$\frac{760 + 90U}{1 + U} = 90 + \frac{670}{1 + U} \leq 760 \text{ kg/m}^3.$$

这说明上述解释不能同时精确成立，因此本文只将 ρc_p 用于有效热容量。该检查针对 72 h 数据和指定解释，不是 51.0877 h 报告时刻的误差估计，也不说明题目无法建模。

此外，4 h 之后的环境采用恒定延续，未获得内部温湿实测数据进行参数辨识或预测验证。未来可在补充测量范围后开展环境敏感性、相同物性下收缩与固定域对照；现有资料不支持给出这些扩展的实验结论。

8.3 结论

在题给初值、经验物性和本文明确的等效假设下，问题一 1800 s 时中心温度 33.5758°C 、表面 36.7879°C ，水分损失主要集中在外层；问题二 3 h 时中心温度 49.8494°C 、含水率 1.7662 kg/kg ，温度接近平衡早于内部达到干燥标准。全域阈值计算给出固定半径模型的报告时长 57.4731 h、给定收缩模型的报告时长 51.0877 h。二者均经未舍入末态和时空加密核验，且移动边界计算未超出半径观测范围。

AI 工具使用声明

本文工作使用了 AI 工具，主要用于既有建模材料与程序的辅助核对、论文结构整理、中文正文起草、公式与结果的一致性检查以及 LaTeX 排版。具体使用范围、交互方式和核验记录见支撑材料中的《AI 工具使用详情》。

参考文献

- [1] Hussain M M, Dincer I. Two-dimensional heat and moisture transfer analysis of a cylindrical moist object subjected to drying: A finite-difference approach[J]. International Journal of Heat and Mass Transfer, 2003, 46(21): 4033–4039. DOI: 10.1016/S0017-9310(03)00229-1.
- [2] Kaya A, Aydin O, Dincer I. Numerical modeling of forced-convection drying of cylindrical moist objects[J]. Numerical Heat Transfer, Part A: Applications, 2007, 51(9): 843–854. DOI: 10.1080/10407780601112753.
- [3] The SciPy community. PchipInterpolator: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.PchipInterpolator.html>.
- [4] The SciPy community. solve_ivp: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html.
- [5] Brasiello A, Venditti C, Adrover A. Non-isothermal Moving-boundary Model for Food Drying[J]. Chemical Engineering Transactions, 2021, 87: 193–198. DOI: 10.3303/CET2187033.

附录 A 支撑材料清单与复现说明

支撑材料包含 `code/` 中的 18 个 Python 源文件、`tmp/` 中的 4 个 JavaScript 导出与检查脚本、四个正式 Excel 结果文件、题定结果 CSV、四问验证 JSON、主要验证报告、依赖清单、源文件哈希映射和 AI 工具使用详情。赛题原始数据按原附件目录放置，较大的内部网格数组可通过程序复算生成。

Python 依赖为 NumPy、SciPy、openpyxl 和 Matplotlib；原生产运行记录为 Python 3.14.6、NumPy 2.5.1、SciPy 1.18.0。代码采用相对项目根目录，需提供 `data/` 附件/附件 1.xlsx、附件 2.xlsx 及附件 3/ 中的题给模板。所有初值、物性和时间单位与正文一致。

按下列次序可复算四问数值与所用加密对照。第三问使用 `--no-xlsx` 可独立复核数值而不依赖 Node；随附正式 Excel 不受该参数更新。

```
python code/problem1.py
python code/problem2.py
python code/problem3.py --no-xlsx
python code/problem4.py --level 2
python code/problem4.py --level 4
python code/problem4.py --level 4 --tight
python code/problem4.py --level 8
python code/problem4.py --level 16
python code/problem4.py --level 8 --tight
python code/verify_q4_results.py
python code/plot_q3_results.py
```

原第 3、4 问 Excel 导出脚本依赖 Node.js 与 `@oai/artifact-tool`。配置依赖后，第 3 问不加 `--no-xlsx` 运行，第 4 问执行 `node tmp/q4_export/build.mjs`。支撑材料附完整导出脚本。为使代码可迁移，仅将原计算机的 Node 路径改为从 PATH 查找，将第 3 问布局模板路径改为随附结果文件；数值方程与参数保持原实现。

以下列出本论文模型与验证所使用的完整源程序。部分文件保留历史诊断函数；正式入口及当前输出以以上复现顺序为准。各程序运行会在本地生成结果与诊断文件，宜在支撑材料副本中执行。

附录 B 完整源程序

B.1 `code/utils.py`

```
1 """Shared paths and attachment readers for Problem A."""
2 from __future__ import annotations
3
4 from pathlib import Path
5
6 import numpy as np
7 from openpyxl import load_workbook
8 from scipy.interpolate import PchipInterpolator
9
```

```

10 ROOT = Path(__file__).resolve().parents[1]
11 DATA_DIR = ROOT / "data"
12 ATTACH_DIR = DATA_DIR / " 附件"
13 RESULTS_DIR = ROOT / "results"
14 FIGURES_DIR = ROOT / "figures"
15 REPORTS_DIR = ROOT / "reports"
16
17 # Appendix 2 (Problem 1)
18 R = 0.02
19 L = 0.25
20 RHO = 820.0
21 CP = 2600.0
22 K = 0.36
23 H = 25.0
24 HM = 8.0e-7
25 T0 = 28.0
26 C0 = 2.55
27 D_PREFACTOR = 7.0e-9
28 D_EXP_COEF = 0.89
29 T_END = 1800.0
30 ALPHA = K / (RHO * CP)
31
32
33 def moisture_diffusivity(C: np.ndarray | float, *, protect: bool = True) -> np.ndarray | float:
34     """D(C) = 7e-9 exp(-0.89/C). Nonpositive C uses D=0 only as a solver-domain guard."""
35     C_arr = np.asarray(C, dtype=float)
36     out = np.zeros_like(C_arr, dtype=float)
37     positive = C_arr > 0.0
38     if np.any(positive):
39         out[positive] = D_PREFACTOR * np.exp(-D_EXP_COEF / C_arr[positive])
40     if not protect and np.any(~positive):
41         raise ValueError("moisture diffusivity evaluated at nonpositive C without protection")
42     if np.isscalar(C):
43         return float(out)
44     return out
45
46
47 def load_oven_table() -> tuple[np.ndarray, np.ndarray, np.ndarray]:
48     path = ATTACH_DIR / " 附件 1.xlsx"
49     wb = load_workbook(path, read_only=True, data_only=True)
50     rows = list(wb.active.values)
51     wb.close()
52     data = np.asarray(rows[1:], dtype=float)
53     t, Ta, Ca = data[:, 0], data[:, 1], data[:, 2]
54     if t[0] != 0.0:
55         raise ValueError(f"oven table must start at t=0, got {t[0]}")
56     return t, Ta, Ca
57
58
59 class PiecewiseLinear:
60     """Piecewise linear interpolant; kept for comparison, not used in Q1 production."""
61
62     def __init__(self, t: np.ndarray, y: np.ndarray):
63         self.t = np.asarray(t, dtype=float)
64         self.y = np.asarray(y, dtype=float)
65         self.kind = "linear"
66         if self.t.ndim != 1 or self.y.shape != self.t.shape:
67             raise ValueError("interpolant requires matching 1-D arrays")
68         if np.any(np.diff(self.t) <= 0):
69             raise ValueError("breakpoints must be strictly increasing")
70
71     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
72         return np.interp(t, self.t, self.y)
73
74     def slope(self, t: float) -> float:

```

```

75         t = float(t)
76         if t <= self.t[0] or t >= self.t[-1]:
77             if t >= self.t[-1]:
78                 dt = self.t[-1] - self.t[-2]
79                 return (self.y[-1] - self.y[-2]) / dt
80             return (self.y[1] - self.y[0]) / (self.t[1] - self.t[0])
81         i = int(np.searchsorted(self.t, t, side="right") - 1)
82         i = min(i, len(self.t) - 2)
83         return (self.y[i + 1] - self.y[i]) / (self.t[i + 1] - self.t[i])
84
85
86 class PchipOven:
87     """Monotone piecewise cubic Hermite interpolant (PCHIP) through attachment knots."""
88
89     def __init__(self, t: np.ndarray, y: np.ndarray):
90         self.t = np.asarray(t, dtype=float)
91         self.y = np.asarray(y, dtype=float)
92         self.kind = "pchip"
93         if self.t.ndim != 1 or self.y.shape != self.t.shape:
94             raise ValueError("interpolant requires matching 1-D arrays")
95         if np.any(np.diff(self.t) <= 0):
96             raise ValueError("breakpoints must be strictly increasing")
97         self._pchip = PchipInterpolator(self.t, self.y, extrapolate=False)
98         self._deriv = self._pchip.derivative()
99
100     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
101         tt = np.clip(np.asarray(t, dtype=float), self.t[0], self.t[-1])
102         val = self._pchip(tt)
103         if np.ndim(t) == 0:
104             return float(val)
105         return np.asarray(val, dtype=float)
106
107     def slope(self, t: float) -> float:
108         tt = float(np.clip(t, self.t[0], self.t[-1]))
109         return float(self._deriv(tt))

```

B.2 code/appendix3.py

```

1  """Appendix 3 effective properties for Problems 2 and 3."""
2  from __future__ import annotations
3
4  import numpy as np
5
6  RHO_A = 650.0
7  RHO_B = 128.0
8  CP_A = 1450.0
9  CP_B = 2736.0
10 K_A = 0.21
11 K_B = 0.38
12 D_PREFACTOR = 2.4e-3
13 D_C_COEF = 0.45
14 D_T_COEF = 3850.0
15
16
17 def rho(C: np.ndarray | float) -> np.ndarray | float:
18     return RHO_A + RHO_B * np.asarray(C, dtype=float)
19
20
21 def cp(C: np.ndarray | float) -> np.ndarray | float:
22     C_arr = np.asarray(C, dtype=float)
23     return CP_A + CP_B * C_arr / (1.0 + C_arr)
24
25
26 def conductivity(C: np.ndarray | float) -> np.ndarray | float:
27     C_arr = np.asarray(C, dtype=float)

```

```

28     return K_A + K_B * C_arr / (1.0 + C_arr)
29
30
31 def heat_capacity(C: np.ndarray | float) -> np.ndarray | float:
32     return rho(C) * cp(C)
33
34
35 def conductivity_deriv(C: np.ndarray | float) -> np.ndarray | float:
36     C_arr = np.asarray(C, dtype=float)
37     return K_B / (1.0 + C_arr) ** 2
38
39
40 def heat_capacity_deriv(C: np.ndarray | float) -> np.ndarray | float:
41     C_arr = np.asarray(C, dtype=float)
42     return RHO_B * cp(C_arr) + rho(C_arr) * CP_B / (1.0 + C_arr) ** 2
43
44
45 def moisture_diffusivity_q2(
46     C: np.ndarray | float,
47     T_celsius: np.ndarray | float,
48     *,
49     protect: bool = True,
50 ) -> np.ndarray | float:
51     """D(C,T) with T in Celsius; Kelvin is used only inside the Arrhenius factor."""
52     C_arr = np.asarray(C, dtype=float)
53     T_arr = np.asarray(T_celsius, dtype=float)
54     out = np.zeros(np.broadcast(C_arr, T_arr).shape, dtype=float)
55     C_b, T_b = np.broadcast_arrays(C_arr, T_arr)
56     positive = C_b > 0.0
57     if np.any(positive):
58         Tk = T_b[positive] + 273.15
59         out[positive] = D_PREFACTOR * np.exp(-D_C_COEF / C_b[positive]) * np.exp(-D_T_COEF / Tk)
60     if not protect and np.any(~positive):
61         raise ValueError("q2 diffusivity evaluated at nonpositive C without protection")
62     if np.ndim(C) == 0 and np.ndim(T_celsius) == 0:
63         return float(np.asarray(out))
64     return out
65
66
67 def diffusivity_partials(C: np.ndarray, T_celsius: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪ np.ndarray]:
68     D = np.asarray(moisture_diffusivity_q2(C, T_celsius, protect=True), dtype=float)
69     C_arr = np.asarray(C, dtype=float)
70     T_arr = np.asarray(T_celsius, dtype=float)
71     dD_dC = np.zeros_like(D)
72     dD_dT = np.zeros_like(D)
73     ok = C_arr > 0.0
74     dD_dC[ok] = D[ok] * (D_C_COEF / C_arr[ok] ** 2)
75     Tk = T_arr + 273.15
76     dD_dT[ok] = D[ok] * (D_T_COEF / Tk[ok] ** 2)
77     return D, dD_dC, dD_dT

```

B.3 code/radial_fvm.py

```

1  """Node-centered radial finite-volume operators for Problem 1."""
2  from __future__ import annotations
3
4  import numpy as np
5  from scipy.sparse import diags
6
7  from utils import CP, H, HM, K, R, RHO, moisture_diffusivity
8
9  OUTPUT_RADII_M = 0.001 * np.arange(21)
10
11

```

```

12 def graded_radial_nodes(level: int) -> np.ndarray:
13     """Nodes denser near the surface; every 0.1 cm output radius remains a node.
14
15     Interval subdivisions (before multiplying by ``level``):
16     0-1.5 cm: 8; 1.5-1.8 cm: 16; 1.8-1.9 cm: 32; 1.9-2.0 cm: 64.
17     Surface spacing at level L is 1e-3 / (64 L) metres.
18     """
19     level = int(level)
20     if level < 1:
21         raise ValueError("graded mesh level must be a positive integer")
22     n_sub = np.full(20, 8 * level, dtype=int)
23     n_sub[15:] = 16 * level
24     n_sub[18] = 32 * level
25     n_sub[19] = 64 * level
26     pts = [0.0]
27     for j in range(20):
28         r0 = 0.001 * j
29         r1 = 0.001 * (j + 1)
30         m = int(n_sub[j])
31         for k in range(1, m + 1):
32             pts.append(r0 + (r1 - r0) * k / m)
33     r = np.asarray(pts, dtype=float)
34     r[0] = 0.0
35     r[-1] = R
36     return r
37
38
39 class RadialFVM:
40     def __init__(self, N: int | None = None, r: np.ndarray | None = None):
41         if r is not None:
42             self.r = np.asarray(r, dtype=float)
43             if self.r.ndim != 1 or self.r.size < 3:
44                 raise ValueError("r must be a 1-D array with at least 3 nodes")
45             if abs(self.r[0]) > 1e-15 or abs(self.r[-1] - R) > 1e-14:
46                 raise ValueError("radial nodes must start at 0 and end at R")
47             if np.any(np.diff(self.r) <= 0.0):
48                 raise ValueError("radial nodes must be strictly increasing")
49         else:
50             if N is None or int(N) < 2:
51                 raise ValueError("N must be at least 2")
52             self.r = np.linspace(0.0, R, int(N) + 1)
53             self.n_nodes = int(self.r.size)
54             self.N = self.n_nodes - 1
55             self.dr_face = np.diff(self.r)
56             self.r_half = 0.5 * (self.r[:-1] + self.r[1:])
57             a = np.empty(self.n_nodes)
58             b = np.empty(self.n_nodes)
59             a[0] = 0.0
60             b[0] = self.r_half[0]
61             a[-1] = self.r_half[-1]
62             b[-1] = R
63             if self.n_nodes > 2:
64                 a[1:-1] = self.r_half[:-1]
65                 b[1:-1] = self.r_half[1:]
66             self.W = 0.5 * (b**2 - a**2)
67             self.dr = float(self.dr_face.min())
68             if abs(self.W.sum() - 0.5 * R**2) > 1e-14:
69                 raise RuntimeError("radial weights do not sum to R^2/2")
70
71             self.M_heat = self._heat_matrix()
72             self.M_heat_sparse = diags(
73                 [np.diag(self.M_heat, -1), np.diag(self.M_heat, 0), np.diag(self.M_heat, 1)],
74                 offsets=[-1, 0, 1],
75                 shape=self.M_heat.shape,
76                 format="csc",

```

```

77         )
78         self._heat_force_coef = (R * H) / (RHO * CP * self.W[-1])
79         self._heat_force_index = self.n_nodes - 1
80
81     def output_indices(self) -> np.ndarray:
82         """Indices of 0, 0.1, ..., 2.0 cm nodes."""
83         idx = np.empty(OUTPUT_RADII_M.size, dtype=int)
84         for j, rc in enumerate(OUTPUT_RADII_M):
85             k = int(np.argmin(np.abs(self.r - rc)))
86             if abs(self.r[k] - rc) > 1e-12:
87                 raise ValueError(f"grid misses output radius {rc}, nearest {self.r[k]}")
88             idx[j] = k
89         return idx
90
91     def _interior_face_coeff(self) -> np.ndarray:
92         return self.r_half / self.dr_face
93
94     def _heat_matrix(self) -> np.ndarray:
95         n = self.n_nodes
96         M = np.zeros((n, n))
97         face = K * self._interior_face_coeff()
98         den = RHO * CP * self.W
99         M[0, 0] = -face[0] / den[0]
100        M[0, 1] = face[0] / den[0]
101        for i in range(1, n - 1):
102            M[i, i - 1] = face[i - 1] / den[i]
103            M[i, i + 1] = face[i] / den[i]
104            M[i, i] = -(face[i - 1] + face[i]) / den[i]
105        i = n - 1
106        M[i, i - 1] = face[i - 1] / den[i]
107        M[i, i] = (-face[i - 1] - R * H) / den[i]
108        return M
109
110     def heat_rhs(self, t: float, T: np.ndarray, Ta: float) -> np.ndarray:
111         out = self.M_heat @ T
112         out[self._heat_force_index] += self._heat_force_coef * Ta
113         return out
114
115     def heat_jac(self, t: float, T: np.ndarray):
116         return self.M_heat_sparse
117
118     def heat_jac_sparsity(self):
119         n = self.n_nodes
120         ones_m = np.ones(n - 1, dtype=float)
121         ones_d = np.ones(n, dtype=float)
122         return diags([ones_m, ones_d, ones_m], offsets=[-1, 0, 1], shape=(n, n), format="csc")
123
124     def moisture_rhs(self, t: float, C: np.ndarray, Ca: float, D_const: float | None = None) -> np.ndarray:
125         if D_const is None:
126             D_node = moisture_diffusivity(C, protect=True)
127         else:
128             D_node = np.full_like(C, float(D_const))
129         D_face = harmonic_mean(D_node[:-1], D_node[1:])
130         F = -self.r_half * D_face * np.diff(C) / self.dr_face
131         Cdot = np.empty_like(C)
132         Cdot[0] = -F[0] / self.W[0]
133         Cdot[1:-1] = (F[:-1] - F[1:]) / self.W[1:-1]
134         F_surf = R * HM * (C[-1] - Ca)
135         Cdot[-1] = (F[-1] - F_surf) / self.W[-1]
136         return Cdot
137
138     def moisture_jac_sparsity(self):
139         return self.heat_jac_sparsity()
140
141     def surface_heat_influx_density(self, T_s: float, Ta: float) -> float:

```

```

142     """Heat into the solid per unit surface area, W/m^2."""
143     return H * (Ta - T_s)
144
145     def surface_moisture_outflux(self, C_s: float, Ca: float) -> float:
146         """Outward moisture flux in the normalized (no rho_d) units, m/s * kg/kg."""
147         return HM * (C_s - Ca)
148
149
150 def harmonic_mean(a: np.ndarray, b: np.ndarray) -> np.ndarray:
151     out = np.zeros_like(a, dtype=float)
152     both = (a > 0.0) & (b > 0.0)
153     out[both] = 2.0 * a[both] * b[both] / (a[both] + b[both])
154     return out

```

B.4 code/radial_coupled.py

```

1  """Coupled variable-property radial FVM for Problem 2."""
2  from __future__ import annotations
3
4  import numpy as np
5  from scipy.sparse import lil_matrix
6
7  from appendix3 import conductivity, heat_capacity, moisture_diffusivity_q2
8  from radial_fvm import RadialFVM, harmonic_mean
9  from utils import H, HM, R
10
11
12 class CoupledRadialFVM:
13     def __init__(self, r: np.ndarray):
14         geom = RadialFVM(r=r)
15         self.r = geom.r
16         self.W = geom.W
17         self.r_half = geom.r_half
18         self.dr_face = geom.dr_face
19         self.n_nodes = geom.n_nodes
20         self.N = geom.N
21         self._geom = geom
22
23     def output_indices(self) -> np.ndarray:
24         return self._geom.output_indices()
25
26     def pack(self, T: np.ndarray, C: np.ndarray) -> np.ndarray:
27         return np.concatenate([T, C])
28
29     def unpack(self, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
30         n = self.n_nodes
31         return y[:n], y[n:]
32
33     def rhs(
34         self,
35         t: float,
36         y: np.ndarray,
37         Ta: float,
38         Ca: float,
39         *,
40         D_const: float | None = None,
41         ST: np.ndarray | None = None,
42         SC: np.ndarray | None = None,
43     ) -> np.ndarray:
44         T, C = self.unpack(y)
45         kn = np.asarray(conductivity(C), dtype=float)
46         Bn = np.asarray(heat_capacity(C), dtype=float)
47         if D_const is None:
48             Dn = np.asarray(moisture_diffusivity_q2(C, T, protect=True), dtype=float)
49         else:

```

```

50         Dn = np.full_like(C, float(D_const))
51         k_face = harmonic_mean(kn[:-1], kn[1:])
52         D_face = harmonic_mean(Dn[:-1], Dn[1:])
53         FT = -self.r_half * k_face * np.diff(T) / self.dr_face
54         FC = -self.r_half * D_face * np.diff(C) / self.dr_face
55         Tdot = np.empty_like(T)
56         Cdot = np.empty_like(C)
57         Tdot[0] = -FT[0] / (Bn[0] * self.W[0])
58         Cdot[0] = -FC[0] / self.W[0]
59         Tdot[1:-1] = (FT[:-1] - FT[1:]) / (Bn[1:-1] * self.W[1:-1])
60         Cdot[1:-1] = (FC[:-1] - FC[1:]) / self.W[1:-1]
61         FTsurf = R * H * (T[-1] - Ta)
62         FCsurf = R * HM * (C[-1] - Ca)
63         Tdot[-1] = (FT[-1] - FTsurf) / (Bn[-1] * self.W[-1])
64         Cdot[-1] = (FC[-1] - FCsurf) / self.W[-1]
65         if ST is not None:
66             Tdot = Tdot + ST / Bn
67         if SC is not None:
68             Cdot = Cdot + SC
69         return self.pack(Tdot, Cdot)
70
71     def jac_sparsity(self):
72         n = self.n_nodes
73         J = lil_matrix((2 * n, 2 * n), dtype=float)
74         for i in range(n):
75             js = [i]
76             if i > 0:
77                 js.append(i - 1)
78             if i + 1 < n:
79                 js.append(i + 1)
80             for j in js:
81                 J[i, j] = 1.0
82                 J[i, n + j] = 1.0
83                 J[n + i, j] = 1.0
84                 J[n + i, n + j] = 1.0
85         return J.tocsc()

```

B.5 code/q1_analytic.py

```

1  """Independent Bessel-Duhamel reference for the linear radial heat (or frozen-D moisture) problem."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  import numpy as np
7  from scipy.integrate import solve_ivp
8  from scipy.optimize import brentq
9  from scipy.special import j0, j1
10 from scipy.sparse import diags
11
12
13 def robin_eigenvalues(Bi: float, n_roots: int) -> np.ndarray:
14     """Positive roots of  $\lambda * J_1(\lambda) = Bi * J_0(\lambda)$ ."""
15     if Bi <= 0:
16         raise ValueError("Biot number must be positive")
17     n_roots = int(n_roots)
18
19     def f(lam: float) -> float:
20         return lam * j1(lam) - Bi * j0(lam)
21
22     lam_max = (n_roots + 4) * np.pi
23     grid = np.linspace(1e-10, lam_max, max(8000, 200 * n_roots))
24     vals = grid * j1(grid) - Bi * j0(grid)
25     roots: list[float] = []
26     for i in range(len(grid) - 1):

```

```

27     v0, v1 = vals[i], vals[i + 1]
28     if not np.isfinite(v0) or not np.isfinite(v1):
29         continue
30     a, b = float(grid[i]), float(grid[i + 1])
31     if v0 == 0.0:
32         cand = a
33     elif v0 * v1 < 0.0:
34         cand = float(brentq(f, a, b, xtol=1e-14, maxiter=200))
35     else:
36         continue
37     if not roots or abs(cand - roots[-1]) > 1e-10:
38         roots.append(cand)
39     if len(roots) >= n_roots:
40         break
41     if len(roots) < n_roots:
42         raise RuntimeError(f"found only {len(roots)} Robin Bessel roots, need {n_roots}")
43     return np.asarray(roots, dtype=float)
44
45
46 def expansion_coefficients(lam: np.ndarray) -> np.ndarray:
47     j0l = j0(lam)
48     j1l = j1(lam)
49     return 2.0 * j1l / (lam * (j0l**2 + j1l**2))
50
51
52 class BesselDuhamel:
53     def __init__(self, Bi: float, diffusivity: float, R: float, n_terms: int = 120):
54         self.Bi = float(Bi)
55         self.alpha = float(diffusivity)
56         self.R = float(R)
57         self.lam = robin_eigenvalues(self.Bi, n_terms)
58         self.A = expansion_coefficients(self.lam)
59         self.beta = self.alpha * self.lam**2 / self.R**2
60
61     def radial_modes(self, r: np.ndarray) -> np.ndarray:
62         """Return array shape (n_terms, n_r) of A_n J0(lambda_n r/R)."""
63         r = np.asarray(r, dtype=float)
64         return self.A[:, None] * j0(self.lam[:, None] * (r[None, :] / self.R))
65
66     def evaluate(self, r: np.ndarray, t: float, env: Any, u0: float) -> np.ndarray:
67         return self.evaluate_many(r, np.array([float(t)]), env, u0)[0]
68
69     def evaluate_many(self, r: np.ndarray, times: np.ndarray, env: Any, u0: float) -> np.ndarray:
70         """T = env(t) - sum A_n J0 I_n, with I' = -beta I + env'(t), I(0)=env(0)-u0.
71
72         Valid for any C1 interpolant that exposes env.y, env.slope, and env(t).
73         """
74         r = np.asarray(r, dtype=float)
75         times = np.asarray(times, dtype=float)
76         if times.size == 0:
77             return np.empty((0, r.size), dtype=float)
78         modes = self.radial_modes(r)
79         beta = self.beta
80         I0 = np.full(beta.size, float(env.y[0] - u0))
81         t_end = float(np.max(times))
82         out = np.empty((times.size, r.size), dtype=float)
83         zero = times == 0.0
84         if np.all(zero):
85             out[:] = u0
86             return out
87         t_eval = times[~zero]
88         sol = solve_ivp(
89             lambda t, I: -beta * I + env.slope(t),
90             (0.0, max(t_end, 1e-30)),
91             I0,

```

```

92         method="BDF",
93         jac=diags(-beta, format="csc"),
94         rtol=1e-11,
95         atol=1e-13,
96         t_eval=t_eval,
97         max_step=1.0,
98         vectorized=False,
99     )
100     if not sol.success:
101         raise RuntimeError(f"Bessel Duhamel ODE failed: {sol.message}")
102     Ihist = sol.y.T
103     env_t = np.array([float(env(t)) for t in t_eval], dtype=float)
104     nonzero_vals = env_t[:, None] - Ihist @ modes
105     out[zero] = u0
106     out[~zero] = nonzero_vals
107     return out
108
109
110 def converged_reference(Bi, diffusivity, R, r, times, env, u0, start_terms=320, tolerance=1e-8):
111     """Double the modal truncation and explicitly check every requested point."""
112     previous = BesselDuhamel(Bi, diffusivity, R, start_terms).evaluate_many(r, times, env, u0)
113     history = []
114     for terms in (start_terms * 2, start_terms * 4, start_terms * 8):
115         current = BesselDuhamel(Bi, diffusivity, R, terms).evaluate_many(r, times, env, u0)
116         error = float(np.max(np.abs(current - previous)))
117         history.append({"terms": terms, "previous_terms": terms // 2, "max_abs_difference": error})
118         if np.isfinite(error) and error <= tolerance:
119             return current, {"passed": True, "tolerance": tolerance, "history": history, "terms": terms}
120         previous = current
121     return current, {"passed": False, "tolerance": tolerance, "history": history, "terms": terms}

```

B.6 code/delivery.py

```

1  """Fail-closed numerical delivery checks shared by Q1 and Q2."""
2  import json
3  from pathlib import Path
4
5
6  def make_gate(checks):
7      checks = {name: bool(value) for name, value in checks.items()}
8      return {"checks": checks, "failed_checks": [k for k, v in checks.items() if not v],
9              "meets_delivery_gate": bool(checks) and all(checks.values())}
10
11
12 def require_delivery(validation, diagnostic_path):
13     """Persist diagnostics first; never reach any production writer on failure."""
14     path = Path(diagnostic_path)
15     path.parent.mkdir(parents=True, exist_ok=True)
16     path.write_text(json.dumps(validation, ensure_ascii=False, indent=2, default=float), encoding="utf-8")
17     gate = validation["delivery_gate"]
18     checks = gate.get("checks", {})
19     if not checks or not all(value is True for value in checks.values()) or gate.get("meets_delivery_gate")
20     ↪ is not True:
21         raise RuntimeError("Delivery blocked; see " + str(path) + ": " + str(gate.get("failed_checks",
22         ↪ [])))

```

B.7 code/problem1.py

```

1  """Problem 1: radial FVM + BDF for preheating temperature and moisture."""
2  from __future__ import annotations
3
4  import json
5  import sys
6  import time

```

```

7 from pathlib import Path
8
9 import numpy as np
10 from openpyxl import Workbook
11 from scipy.integrate import solve_ivp, quad
12
13 CODE_DIR = Path(__file__).resolve().parent
14 if str(CODE_DIR) not in sys.path:
15     sys.path.insert(0, str(CODE_DIR))
16
17 from q1_analytic import BesselDuhamel, converged_reference
18 from delivery import make_gate, require_delivery
19 from radial_fvm import RadialFVM
20 from radial_fvm import graded_radial_nodes
21 from utils import (
22     ALPHA,
23     C0,
24     CP,
25     FIGURES_DIR,
26     H,
27     HM,
28     K,
29     L,
30     R,
31     RESULTS_DIR,
32     RHO,
33     T0,
34     T_END,
35     PchipOven,
36     load_oven_table,
37     moisture_diffusivity,
38 )
39
40 OUTPUT_TIMES = np.arange(1.0, T_END + 0.5, 1.0)
41 TABLE_TIMES = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
42 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
43 OUTPUT_RADII_M = 0.001 * np.arange(21) # 0, 0.1, ..., 2.0 cm
44 PAPER_TIMES = TABLE_TIMES.copy()
45
46
47 def load_q1_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
48     t, Ta, Ca = load_oven_table()
49     mask = t <= T_END + 1e-12
50     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
51     if t.size != 31 or abs(t[-1] - T_END) > 1e-12:
52         raise ValueError(f"expected 31 oven records on [0, 1800], got {t.size}, t_end={t[-1]}")
53     if abs(Ta[0] - T0) > 1e-12:
54         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
55     Ta_fun = PchipOven(t, Ta)
56     Ca_fun = PchipOven(t, Ca)
57     for tj, yj in zip(t, Ta):
58         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
59             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
60     for tj, yj in zip(t, Ca):
61         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
62             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
63     return Ta_fun, Ca_fun, t
64
65
66 def integrate_mol(
67     rhs,
68     y0: np.ndarray,
69     t_breaks: np.ndarray,
70     *,
71     rtol: float,

```

```

72     atol: float,
73     max_step: float,
74     jac=None,
75     jac_sparsity=None,
76     field_name: str = "field",
77 ):
78     y = np.asarray(y0, dtype=float).copy()
79     times = [0.0]
80     states = [y.copy()]
81     pieces: list[tuple[float, float, object]] = []
82     t_end = float(t_breaks[-1])
83     for i in range(len(t_breaks) - 1):
84         t0 = float(t_breaks[i])
85         t1 = float(min(t_breaks[i + 1], t_end))
86         if t1 <= t0:
87             continue
88         t_eval = np.arange(np.floor(t0) + 1.0, np.floor(t1) + 1.0 + 1e-12, 1.0)
89         t_eval = t_eval[(t_eval > t0 + 1e-14) & (t_eval <= t1 + 1e-14)]
90         kwargs = dict(
91             fun=rhs,
92             t_span=(t0, t1),
93             y0=y,
94             method="BDF",
95             rtol=rtol,
96             atol=atol,
97             max_step=max_step,
98             dense_output=True,
99             vectorized=False,
100         )
101         if t_eval.size:
102             kwargs["t_eval"] = t_eval
103         if jac is not None:
104             kwargs["jac"] = jac
105         if jac_sparsity is not None:
106             kwargs["jac_sparsity"] = jac_sparsity
107         sol = solve_ivp(**kwargs)
108         if not sol.success:
109             raise RuntimeError(f"{field_name} BDF failed on [{t0}, {t1}]: {sol.message}")
110         pieces.append((t0, t1, sol))
111         if sol.t.size:
112             for k, tk in enumerate(sol.t):
113                 if times and abs(tk - times[-1]) < 1e-12:
114                     states[-1] = sol.y[:, k].copy()
115                     continue
116                 times.append(float(tk))
117                 states.append(sol.y[:, k].copy())
118             y = sol.y[:, -1].copy() if sol.y.size else y
119         t_arr = np.asarray(times, dtype=float)
120         Y = np.stack(states, axis=0)
121         return t_arr, Y, pieces
122
123
124 def dense_state(pieces, t: float) -> np.ndarray:
125     t = float(t)
126     for t0, t1, sol in pieces:
127         if t0 - 1e-12 <= t <= t1 + 1e-12:
128             return np.asarray(sol.sol(np.clip(t, t0, t1)), dtype=float)
129     raise ValueError(f"time {t} not covered by dense output")
130
131
132 def extract_output_grid(fvm: RadialFVM, Y: np.ndarray) -> np.ndarray:
133     idx = fvm.output_indices()
134     return Y[:, idx]
135
136

```

```

137 def conservation_heat(fvm: RadialFVM, pieces, Ta_fun: PchipOven, T_init: np.ndarray, t: float) -> dict:
138     T_t = dense_state(pieces, t)
139     E = 2.0 * np.pi * L * RHO * CP * np.dot(fvm.W, T_t - T_init)
140     flux = lambda x: float(Ta_fun(x)) - float(dense_state(pieces, x)[-1])
141     Qraw, qerr = quad(flux, 0, t, points=Ta_fun.t[(Ta_fun.t > 0) & (Ta_fun.t < t)], epsabs=1e-9,
142     ↪ epsrel=1e-10, limit=500)
143     Q = 2.0 * np.pi * R * L * H * Qraw
144     denom = max(abs(Q), abs(E), 1e-12)
145     return {
146         "E_relative_J": E,
147         "Q_surface_J": Q,
148         "relative_residual": abs(E - Q) / denom,
149         "quadrature_abs_error": qerr, "quadrature": "adaptive QUADPACK on PCHIP intervals",
150     }
151
152 def conservation_moisture(fvm: RadialFVM, pieces, Ca_fun: PchipOven, C_init: np.ndarray, t: float) -> dict:
153     C_t = dense_state(pieces, t)
154     MC = float(np.dot(fvm.W, C_t))
155     MC0 = float(np.dot(fvm.W, C_init))
156     flux = lambda x: HM * (float(dense_state(pieces, x)[-1]) - float(Ca_fun(x)))
157     I, qerr = quad(flux, 0, t, points=Ca_fun.t[(Ca_fun.t > 0) & (Ca_fun.t < t)], epsabs=1e-14,
158     ↪ epsrel=1e-10, limit=500)
159     rhs = MC0 - R * I
160     denom = max(abs(MC0 - MC), abs(R * I), 1e-18)
161     return {
162         "MC": MC,
163         "MC0": MC0,
164         "minus_R_int_hm": rhs,
165         "relative_residual": abs(MC - rhs) / denom,
166         "quadrature_abs_error": qerr, "quadrature": "adaptive QUADPACK on PCHIP intervals",
167     }
168
169 def compare_on_output(A: np.ndarray, B: np.ndarray, times: np.ndarray) -> dict:
170     diff = np.abs(A - B)
171     k = int(np.argmax(diff))
172     n_t, n_r = diff.shape
173     i, j = divmod(k, n_r)
174     return {
175         "max_abs": float(diff.max()),
176         "max_time_s": float(times[i]),
177         "max_radius_cm": 0.1 * j,
178         "mean_abs": float(diff.mean()),
179         "paper_table_max_abs": float(np.abs(A[np.isin(times, TABLE_TIMES)][:, :5] - B[np.isin(times,
180     ↪ TABLE_TIMES)][:, :5]).max())
181         if A.shape[1] == 21
182         else None,
183     }
184
185 def four_decimal_stats(A: np.ndarray, B: np.ndarray, times: np.ndarray) -> dict:
186     Ar = np.round(A, 4)
187     Br = np.round(B, 4)
188     changed = Ar != Br
189     paper_t = np.isin(times, TABLE_TIMES)
190     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
191     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
192     early = times <= 100.0
193     return {
194         "changed_cells": int(np.sum(changed)),
195         "total_cells": int(A.size),
196         "paper_changed_cells": int(np.sum(paper_changed)),
197         "paper_total_cells": int(paper_changed.size),
198         "early100_changed_cells": int(np.sum(changed[early])),

```

```

199         "max_abs": float(np.max(np.abs(A - B))),
200         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
201     }
202
203
204 def solve_temperature(fvm: RadialFVM, Ta_fun: PchipOven, t_breaks, rtol, atol, max_step):
205     T_init = np.full(fvm.n_nodes, T0)
206     def rhs(t, T):
207         return fvm.heat_rhs(t, T, float(Ta_fun(t)))
208     def jac(t, T):
209         return fvm.heat_jac(t, T)
210     return integrate_mol(
211         rhs, T_init, t_breaks,
212         rtol=rtol, atol=atol, max_step=max_step,
213         jac=jac, jac_sparsity=fvm.heat_jac_sparsity(),
214         field_name="temperature",
215     )
216
217
218 def solve_moisture(fvm: RadialFVM, Ca_fun: PchipOven, t_breaks, rtol, atol, max_step, D_const=None):
219     C_init = np.full(fvm.n_nodes, C0)
220     def rhs(t, C):
221         return fvm.moisture_rhs(t, C, float(Ca_fun(t)), D_const=D_const)
222     return integrate_mol(
223         rhs, C_init, t_breaks,
224         rtol=rtol, atol=atol, max_step=max_step,
225         jac_sparsity=fvm.moisture_jac_sparsity(),
226         field_name="moisture",
227     )
228
229
230 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
231     Ta_max = float(np.max(Ta_fun.y))
232     Ta_min = float(np.min(Ta_fun.y))
233     Ca_min = float(np.min(Ca_fun.y))
234     info = {
235         "T_min": float(T_out.min()),
236         "T_max": float(T_out.max()),
237         "C_min": float(C_out.min()),
238         "C_max": float(C_out.max()),
239         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
240         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),
241         "C_above_C0": float(max(0.0, C_out.max() - C0)),
242         "C_below_Ca_min": float(max(0.0, Ca_min - C_out.min())),
243         "Ta_range": [Ta_min, Ta_max],
244         "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
245     }
246     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-6
247     info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_below_Ca_min"] <= 1e-8
248     info["C_nonnegative"] = bool(C_out.min() >= -1e-8)
249     return info
250
251
252 def write_result_workbook(path: Path, T_out: np.ndarray, C_out: np.ndarray, times: np.ndarray):
253     radii_cm = 0.1 * np.arange(21)
254     wb = Workbook()
255     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
256         ws = wb.create_sheet(title)
257         ws["A1"] = " 时间\\到药材中心的距离"
258         for j, rc in enumerate(radii_cm, start=2):
259             ws.cell(1, j, float(rc))
260         for i, t in enumerate(times):
261             ws.cell(i + 2, 1, int(round(float(t))))
262             for j in range(21):
263                 cell = ws.cell(i + 2, j + 2, float(arr[i, j]))

```

```

264         cell.number_format = "0.0000"
265     if "Sheet" in wb.sheetnames:
266         del wb["Sheet"]
267     path.parent.mkdir(parents=True, exist_ok=True)
268     wb.save(path)
269
270
271 def write_paper_csv(path: Path, arr_out: np.ndarray, times: np.ndarray, radii_cm=TABLE_RADII_CM):
272     t_idx = {float(t): i for i, t in enumerate(times)}
273     r_idx = {round(0.1 * j, 10): j for j in range(21)}
274     lines = ["t_s," + ",".join(f"r_{rc:g}cm" for rc in radii_cm)]
275     for t in TABLE_TIMES:
276         row = [f"{t:.0f}"]
277         i = t_idx[float(t)]
278         for rc in radii_cm:
279             j = r_idx[float(rc)]
280             row.append(f"{arr_out[i, j]:.8f}")
281         lines.append(",".join(row))
282     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
283
284
285 def setup_mpl():
286     import matplotlib
287     matplotlib.use("Agg")
288     import matplotlib.pyplot as plt
289     plt.rcParams.update({
290         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],
291         "axes.unicode_minus": False,
292         "font.size": 9,
293         "axes.labelsize": 10,
294         "legend.fontsize": 8,
295         "xtick.labelsize": 8,
296         "ytick.labelsize": 8,
297         "axes.linewidth": 0.8,
298         "pdf.fonttype": 42,
299         "ps.fonttype": 42,
300     })
301     return plt
302
303
304 def plot_figures(plt, fvm, times, T_full, C_full, T_out, C_out, Ta_fun, Ca_fun):
305     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
306     r_cm = fvm.r * 100.0
307     r_out_cm = OUTPUT_RADII_M * 100.0
308     t_marks = PAPER_TIMES
309     idx = {float(t): i for i, t in enumerate(times)}
310
311     fig, ax = plt.subplots(figsize=(6.4, 3.8))
312     for t in t_marks:
313         ax.plot(r_cm, T_full[idx[float(t)]], lw=1.2, label=fr"$t={int(t)}\, \mathrm{{{s}}}$")
314     ax.set_xlabel("半径 $r$ (cm)")
315     ax.set_ylabel("温度 $T$ (°C)")
316     ax.set_xlim(0, 2)
317     ax.legend(frameon=False, ncol=2)
318     fig.tight_layout()
319     fig.savefig(FIGURES_DIR / "q1_T_profiles.pdf")
320     plt.close(fig)
321
322     fig, ax = plt.subplots(figsize=(6.4, 3.8))
323     for t in t_marks:
324         ax.plot(r_cm, C_full[idx[float(t)]], lw=1.2, label=fr"$t={int(t)}\, \mathrm{{{s}}}$")
325     ax.set_xlabel("半径 $r$ (cm)")
326     ax.set_ylabel("干基含水率 $C$ (kg/kg)")
327     ax.set_xlim(0, 2)
328     ax.legend(frameon=False, ncol=2)

```

```

329     fig.tight_layout()
330     fig.savefig(FIGURES_DIR / "q1_C_profiles.pdf")
331     plt.close(fig)
332
333     t_plot = np.concatenate([[0.0], times])
334     T_c = np.concatenate([[T0], T_out[:, 0]])
335     T_s = np.concatenate([[T0], T_out[:, -1]])
336     Ta = np.array([Ta_fun(t) for t in t_plot])
337     fig, ax = plt.subplots(figsize=(6.4, 3.8))
338     ax.plot(t_plot, Ta, lw=1.0, ls="--", label="烘房 $T_a$")
339     ax.plot(t_plot, T_s, lw=1.2, label="表面")
340     ax.plot(t_plot, T_c, lw=1.2, label="中心")
341     ax.set_xlabel("时间 $t$ (s)")
342     ax.set_ylabel("温度 (°C)")
343     ax.set_xlim(0, 1800)
344     ax.legend(frameon=False)
345     fig.tight_layout()
346     fig.savefig(FIGURES_DIR / "q1_T_center_surface.pdf")
347     plt.close(fig)
348
349     C_c = np.concatenate([[C0], C_out[:, 0]])
350     C_s = np.concatenate([[C0], C_out[:, -1]])
351     fig, ax = plt.subplots(figsize=(6.4, 3.8))
352     ax.plot(t_plot, C_s, lw=1.2, label="表面")
353     ax.plot(t_plot, C_c, lw=1.2, label="中心")
354     ax.set_xlabel("时间 $t$ (s)")
355     ax.set_ylabel(r"干基含水率 $C$ (kg/kg)")
356     ax.set_xlim(0, 1800)
357     ax.legend(frameon=False)
358     fig.tight_layout()
359     fig.savefig(FIGURES_DIR / "q1_C_center_surface.pdf")
360     plt.close(fig)
361
362     fig, ax1 = plt.subplots(figsize=(6.4, 3.6))
363     ax2 = ax1.twinx()
364     t_dense = np.linspace(0.0, T_END, 1801)
365     ax1.plot(t_dense, np.array([Ta_fun(t) for t in t_dense]), color="C0", lw=1.2, label="烘房温度")
366     ax2.plot(t_dense, np.array([Ca_fun(t) for t in t_dense]), color="C1", lw=1.2, ls="--",
367         ↪ label="烘房水分")
367     ax1.scatter(Ta_fun.t, Ta_fun.y, s=14, color="C0", zorder=3)
368     ax2.scatter(Ca_fun.t, Ca_fun.y, s=14, color="C1", zorder=3)
369     ax1.set_xlabel("时间 $t$ (s)")
370     ax1.set_ylabel("烘房温度 (°C)", color="C0")
371     ax2.set_ylabel("烘房水分 (kg/kg)", color="C1")
372     ax1.set_xlim(0, 1800)
373     h1, l1 = ax1.get_legend_handles_labels()
374     h2, l2 = ax2.get_legend_handles_labels()
375     ax1.legend(h1 + h2, l1 + l2, frameon=False)
376     fig.tight_layout()
377     fig.savefig(FIGURES_DIR / "q1_oven_input.pdf")
378     plt.close(fig)
379     _ = r_out_cm
380
381
382 def main():
383     t_wall0 = time.perf_counter()
384     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
385     Ta_fun, Ca_fun, knots = load_q1_inputs()
386     # PCHIP is C1; integrate as one span rather than 60 s linear kinks.
387     t_breaks = np.array([0.0, T_END], dtype=float)
388     print(
389         "oven points", knots.size,
390         "kind", Ta_fun.kind,
391         "Ta(1800)=", Ta_fun(T_END),
392         "Ca(1800)=", Ca_fun(T_END),

```

```

393 )
394 print("alpha", ALPHA, "D(C0)", moisture_diffusivity(C0), "Bi_T", H * R / K)
395
396 mesh_levels = [2, 4, 8]
397 rtol, atol_T, atol_C, max_step = 1e-10, 1e-10, 1e-12, 0.5
398 solutions = {}
399 for level in mesh_levels:
400     r = graded_radial_nodes(level)
401     fvm = RadialFVM(r=r)
402     tag = f"G{level}"
403     print(
404         f"\n=== {tag} nodes={fvm.n_nodes} surface_dr={fvm.dr_face[-1]:.4e} m ==="
405     )
406     print(f"=== {tag} temperature ===")
407     tT, YT, piecesT = solve_temperature(fvm, Ta_fun, t_breaks, rtol, atol_T, max_step)
408     print(f"=== {tag} moisture ===")
409     tC, YC, piecesC = solve_moisture(fvm, Ca_fun, t_breaks, rtol, atol_C, max_step)
410     if not np.allclose(tT, tC):
411         raise RuntimeError("temperature and moisture output times differ")
412     times = tT[tT > 0]
413     YT_pos = YT[tT > 0]
414     YC_pos = YC[tC > 0]
415     T_out = extract_output_grid(fvm, YT_pos)
416     C_out = extract_output_grid(fvm, YC_pos)
417     solutions[level] = dict(
418         tag=tag,
419         level=level,
420         fvm=fvm,
421         times=times,
422         YT=YT_pos,
423         YC=YC_pos,
424         T_out=T_out,
425         C_out=C_out,
426         piecesT=piecesT,
427         piecesC=piecesC,
428         T_init=np.full(fvm.n_nodes, T0),
429         C_init=np.full(fvm.n_nodes, C0),
430         n_nodes=fvm.n_nodes,
431         surface_dr=float(fvm.dr_face[-1]),
432     )
433     print(
434         "done", tag,
435         "T range", T_out.min(), T_out.max(),
436         "C range", C_out.min(), C_out.max(),
437         "C(t=1,s)", C_out[0, -1],
438         "C(t=100,s)", C_out[99, -1],
439     )
440
441 conv = {}
442 fourdec = {}
443 for coarse, fine in ((2, 4), (4, 8)):
444     t = solutions[fine]["times"]
445     conv[f"T_G{coarse}_vs_G{fine}"] = compare_on_output(
446         solutions[coarse]["T_out"], solutions[fine]["T_out"], t
447     )
448     conv[f"C_G{coarse}_vs_G{fine}"] = compare_on_output(
449         solutions[coarse]["C_out"], solutions[fine]["C_out"], t
450     )
451     fourdec[f"T_G{coarse}_vs_G{fine}"] = four_decimal_stats(
452         solutions[coarse]["T_out"], solutions[fine]["T_out"], t
453     )
454     fourdec[f"C_G{coarse}_vs_G{fine}"] = four_decimal_stats(
455         solutions[coarse]["C_out"], solutions[fine]["C_out"], t
456     )
457 print("conv T", coarse, fine, conv[f"T_G{coarse}_vs_G{fine}"])

```

```

458     print("conv C", coarse, fine, conv[f"C_G{coarse}_vs_G{fine}"])
459     print("4dp T", coarse, fine, fourdec[f"T_G{coarse}_vs_G{fine}"])
460     print("4dp C", coarse, fine, fourdec[f"C_G{coarse}_vs_G{fine}"])
461
462     fd_T = fourdec["T_G4_vs_G8"]
463     fd_C = fourdec["C_G4_vs_G8"]
464     paper_4dp_stable = (
465         fd_T["paper_changed_cells"] == 0 and fd_C["paper_changed_cells"] == 0
466     )
467     full_4dp_stable = fd_T["changed_cells"] == 0 and fd_C["changed_cells"] == 0
468     abs_2e5 = (
469         conv["T_G4_vs_G8"]["max_abs"] <= 2e-5
470         and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
471     )
472     pub_level = 4 if paper_4dp_stable and full_4dp_stable else 8
473     pub = solutions[pub_level]
474     fvm = pub["fvm"]
475     times = pub["times"]
476     print(
477         "publishing", pub["tag"],
478         "n_nodes", pub["n_nodes"],
479         "paper_4dp_stable", paper_4dp_stable,
480         "full_4dp_stable", full_4dp_stable,
481         "abs_2e5", abs_2e5,
482     )
483
484     print("time refinement on published mesh")
485     tT2, YT2, _ = solve_temperature(fvm, Ta_fun, t_breaks, rtol / 5, atol_T / 5, 0.25)
486     tC2, YC2, _ = solve_moisture(fvm, Ca_fun, t_breaks, rtol / 5, atol_C / 5, 0.25)
487     T2 = extract_output_grid(fvm, YT2[tT2 > 0])
488     C2 = extract_output_grid(fvm, YC2[tC2 > 0])
489     time_sens = {
490         "T": compare_on_output(pub["T_out"], T2, times),
491         "C": compare_on_output(pub["C_out"], C2, times),
492     }
493     time_4dp = {
494         "T": four_decimal_stats(pub["T_out"], T2, times),
495         "C": four_decimal_stats(pub["C_out"], C2, times),
496     }
497     print("time sens", time_sens)
498     print("time 4dp", time_4dp)
499
500     print("Bessel temperature reference")
501     Bi = H * R / K
502     T_ref, heat_truncation = converged_reference(Bi, ALPHA, R, OUTPUT_RADII_M, times, Ta_fun, T0,
503         ↪ start_terms=120)
504     bessel_T = compare_on_output(pub["T_out"], T_ref, times)
505     print("Bessel T", bessel_T)
506
507     print("frozen-D moisture probe")
508     D0 = float(moisture_diffusivity(C0))
509     Bi_m = HM * R / D0
510     C_ref, moisture_truncation = converged_reference(Bi_m, D0, R, OUTPUT_RADII_M, times, Ca_fun, C0)
511     tCf, YCf, _ = solve_moisture(fvm, Ca_fun, t_breaks, rtol, atol_C, max_step, D_const=D0)
512     C_frozen = extract_output_grid(fvm, YCf[tCf > 0])
513     frozen = compare_on_output(C_frozen, C_ref, times)
514     nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
515     print("nonlinear vs frozen D", nl_vs_frozen)
516     D_field = moisture_diffusivity(pub["YC"])
517     D_stats = {
518         "D0": D0,
519         "D_min": float(np.min(D_field)),
520         "D_max": float(np.max(D_field)),
521         "D_min_over_D0": float(np.min(D_field) / D0),

```

```

522     "D_max_over_D0": float(np.max(D_field) / D0),
523     "C_min_field": float(np.min(pub["YC"])),
524     "C_max_field": float(np.max(pub["YC"])),
525     "surface_C_min": float(np.min(pub["C_out"][:, -1])),
526     "center_C_end": float(pub["C_out"][-1, 0]),
527     "surface_C_end": float(pub["C_out"][-1, -1]),
528     "surface_C_t1": float(pub["C_out"][0, -1]),
529     "surface_C_t100": float(pub["C_out"][99, -1]),
530 }
531 print("D stats", D_stats)
532
533 bounds = assert_physical(pub["YT"], pub["YC"], Ta_fun, Ca_fun)
534 print("bounds", bounds)
535
536 cons_T = conservation_heat(fvm, pub["piecesT"], Ta_fun, pub["T_init"], T_END)
537 cons_C = conservation_moisture(fvm, pub["piecesC"], Ca_fun, pub["C_init"], T_END)
538 print("conservation T", cons_T)
539 print("conservation C", cons_C)
540
541 delivery = make_gate({
542     "space_abs": abs_2e5,
543     "space_paper": paper_4dp_stable,
544     "all_finite": np.isfinite(pub["YT"]).all() and np.isfinite(pub["YC"]).all(),
545     "heat_reference_truncation": heat_truncation["passed"],
546     "heat_reference_error": bessel_T["max_abs"] <= 2e-5,
547     "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
548     "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
    ↪ 0,
549     "temperature_bounds": bounds["T_bounds_ok"],
550     "moisture_bounds": bounds["C_bounds_ok"],
551     "moisture_reference_truncation": moisture_truncation["passed"],
552     "frozen_reference_error": frozen["max_abs"] <= 2e-5,
553     "heat_balance": cons_T["relative_residual"] <= 1e-6,
554     "moisture_balance": cons_C["relative_residual"] <= 1e-6,
555 })
556 print("delivery", delivery)
557
558 paper_T = {
559     f"t{int(t)}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
560     for t in TABLE_TIMES for rc in TABLE_RADII_CM
561 }
562 paper_C = {
563     f"t{int(t)}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
564     for t in TABLE_TIMES for rc in TABLE_RADII_CM
565 }
566
567 validation = {
568     "result_version": "q1-closeout-v1",
569     "interpolation": Ta_fun.kind,
570     "published_mesh": pub["tag"],
571     "published_n_nodes": pub["n_nodes"],
572     "published_surface_dr_m": pub["surface_dr"],
573     "mesh_levels": mesh_levels,
574     "mesh_n_nodes": {f"G{lv}": solutions[lv]["n_nodes"] for lv in mesh_levels},
575     "solver": "FVM+BDF",
576     "rtol": rtol,
577     "atol_T": atol_T,
578     "atol_C": atol_C,
579     "max_step": max_step,
580     "t_breaks": t_breaks.tolist(),
581     "grid_convergence": conv,
582     "four_decimal_grid": fourdec,
583     "time_sensitivity": time_sens,
584     "four_decimal_time": time_4dp,
585     "delivery_gate": delivery,

```

```

586     "reference_truncation_T": heat_truncation,
587     "bessel_temperature": bessel_T,
588     "reference_truncation_C": moisture_truncation,
589     "frozen_D_moisture_vs_analytic": frozen,
590     "nonlinear_C_vs_frozen_D": nl_vs_frozen,
591     "D_variation": D_stats,
592     "conservation_T_1800s": cons_T,
593     "conservation_C_1800s": cons_C,
594     "physical_bounds": bounds,
595     "paper_table_T": paper_T,
596     "paper_table_C": paper_C,
597     "elapsed_s": time.perf_counter() - t_wall0,
598     "python": sys.version,
599 }
600 require_delivery(validation, RESULTS_DIR / "diagnostics/q1_delivery/latest.json")
601 np.savez(
602     RESULTS_DIR / "q1_solution.npz",
603     N=pub["n_nodes"] - 1,
604     mesh_level=pub_level,
605     n_nodes=pub["n_nodes"],
606     times=times,
607     r=fvm.r,
608     checkpoint_times=np.r_[0., TABLE_TIMES],
609     T_checkpoints=np.vstack([pub["T_init"], pub["YT"][TABLE_TIMES.astype(int)-1]]),
610     C_checkpoints=np.vstack([pub["C_init"], pub["YC"][TABLE_TIMES.astype(int)-1]]),
611     T_full=pub["YT"],
612     C_full=pub["YC"],
613     T_out=pub["T_out"],
614     C_out=pub["C_out"],
615     output_radii_m=OUTPUT_RADII_M,
616     interpolation=np.array(Ta_fun.kind),
617     rtol=rtol,
618     atol_T=atol_T,
619     atol_C=atol_C,
620     max_step=max_step,
621     surface_dr=pub["surface_dr"],
622 )
623 write_result_workbook(RESULTS_DIR / "result1.xlsx", pub["T_out"], pub["C_out"], times)
624 write_paper_csv(RESULTS_DIR / "q1_table_temperature.csv", pub["T_out"], times)
625 write_paper_csv(RESULTS_DIR / "q1_table_moisture.csv", pub["C_out"], times)
626
627 plt = setup_mpl()
628 plot_figures(plt, fvm, times, pub["YT"], pub["YC"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
629
630 (RESULTS_DIR / "q1_validation.json").write_text(
631     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
632     encoding="utf-8",
633 )
634 print("elapsed_s", validation["elapsed_s"])
635 print("wrote results to", RESULTS_DIR)
636
637
638
639 if __name__ == "__main__":
640     main()

```

B.8 code/problem2.py

```

1 """Problem 2: coupled variable-property radial drying for 0-3 h."""
2 from __future__ import annotations
3
4 import json
5 import sys
6 import time
7 from pathlib import Path

```

```

8
9 import numpy as np
10 from openpyxl import Workbook
11 from scipy.integrate import solve_ivp
12
13 CODE_DIR = Path(__file__).resolve().parent
14 if str(CODE_DIR) not in sys.path:
15     sys.path.insert(0, str(CODE_DIR))
16
17 from appendix3 import (
18     conductivity,
19     conductivity_deriv,
20     diffusivity_partials,
21     heat_capacity,
22     heat_capacity_deriv,
23     moisture_diffusivity_q2,
24 )
25 from q1_analytic import BesselDuhamel, converged_reference
26 from delivery import make_gate, require_delivery
27 from radial_coupled import CoupledRadialFVM
28 from radial_fvm import graded_radial_nodes
29 from utils import (
30     C0,
31     FIGURES_DIR,
32     H,
33     HM,
34     L,
35     R,
36     RESULTS_DIR,
37     T0,
38     PchipOven,
39     load_oven_table,
40 )
41
42 T_END = 10800.0
43 OUTPUT_RADII_M = 0.001 * np.arange(21)
44 TABLE_HOURS = np.array([0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
45 TABLE_TIMES = 3600.0 * TABLE_HOURS
46 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
47 TREF = 28.0
48
49
50 def load_q2_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
51     t, Ta, Ca = load_oven_table()
52     mask = t <= T_END + 1e-12
53     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
54     if t.size != 181 or abs(t[-1] - T_END) > 1e-12:
55         raise ValueError(f"expected 181 oven records on [0, 10800], got {t.size}, t_end={t[-1]}")
56     if abs(Ta[0] - T0) > 1e-12:
57         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
58     Ta_fun = PchipOven(t, Ta)
59     Ca_fun = PchipOven(t, Ca)
60     for tj, yj in zip(t, Ta):
61         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
62             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
63     for tj, yj in zip(t, Ca):
64         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
65             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
66     return Ta_fun, Ca_fun, t
67
68
69 def integrate_coupled(fvm: CoupledRadialFVM, y0, t_end, Ta_fun, Ca_fun, rtol, atol, max_step,
70     D_const=None, source=None, t_eval=None):
71     y0 = np.asarray(y0, dtype=float)
72     n = fvm.n_nodes

```

```

73     atol_arr = np.asarray(atol, dtype=float)
74     if atol_arr.ndim == 0:
75         atol_arr = np.concatenate([np.full(n, float(atol_arr)), np.full(n, float(atol_arr) * 0.01)])
76
77     def rhs(t, y):
78         ST = SC = None
79         if source is not None:
80             ST, SC = source(t, fvm.r)
81         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)), D_const=D_const, ST=ST, SC=SC)
82
83     if t_eval is None:
84         t_eval = np.arange(1.0, t_end + 0.5, 1.0)
85     sol = solve_ivp(
86         rhs,
87         (0.0, float(t_end)),
88         y0,
89         method="BDF",
90         rtol=rtol,
91         atol=atol_arr,
92         max_step=max_step,
93         t_eval=t_eval,
94         dense_output=False,
95         vectorized=False,
96         jac_sparsity=fvm.jac_sparsity(),
97     )
98     if not sol.success:
99         raise RuntimeError(f"coupled BDF failed: {sol.message}")
100     times = np.concatenate([[0.0], sol.t])
101     Y = np.vstack([y0, sol.y.T])
102     return times, Y
103
104
105 def extract_output(fvm: CoupledRadialFVM, Y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
106     n = fvm.n_nodes
107     idx = fvm.output_indices()
108     T = Y[:, :n][:, idx]
109     C = Y[:, n:][:, idx]
110     return T, C
111
112
113 def compare_on_output(A, B, times) -> dict:
114     diff = np.abs(A - B)
115     k = int(np.argmax(diff))
116     n_t, n_r = diff.shape
117     i, j = divmod(k, n_r)
118     paper_t = np.isin(times, TABLE_TIMES)
119     paper_r = slice(None, None, 5) if n_r == 21 else slice(None)
120     return {
121         "max_abs": float(diff.max()),
122         "max_time_s": float(times[i]),
123         "max_radius_cm": 0.1 * j,
124         "mean_abs": float(diff.mean()),
125         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
126     }
127
128
129 def four_decimal_stats(A, B, times) -> dict:
130     Ar, Br = np.round(A, 4), np.round(B, 4)
131     changed = Ar != Br
132     paper_t = np.isin(times, TABLE_TIMES)
133     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
134     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
135     return {
136         "changed_cells": int(np.sum(changed)),
137         "total_cells": int(A.size),

```

```

138     "paper_changed_cells": int(np.sum(paper_changed)),
139     "paper_total_cells": int(paper_changed.size),
140     "max_abs": float(np.max(np.abs(A - B))),
141     "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
142 }
143
144
145 def manufactured_fields(r: np.ndarray, t: float) -> tuple[np.ndarray, np.ndarray]:
146     x2 = (r / R) ** 2
147     T = 28.0 + 2.0 * np.exp(-t / 600.0) * (1.0 + x2)
148     C = 1.5 + 0.2 * np.exp(-t / 500.0) * (1.0 + x2)
149     return T, C
150
151
152 def manufactured_env(r_surface: float, t: float) -> tuple[float, float]:
153     T, C = manufactured_fields(np.array([r_surface]), t)
154     Ts, Cs = float(T[0]), float(C[0])
155     u = np.exp(-t / 600.0)
156     v = np.exp(-t / 500.0)
157     Tr = 4.0 / R * u
158     Cr = 4.0 / R * v * 0.1 # C amplitude 0.2 vs T amplitude 2: Cr = 0.2*v*2r/R^2 at r=R -> 0.4 v / R
159     Cr = 0.4 / R * v
160     kn = float(conductivity(Cs))
161     Dn = float(moisture_diffusivity_q2(Cs, Ts))
162     Ta = Ts + kn * Tr / H
163     Ca = Cs + Dn * Cr / HM
164     return Ta, Ca
165
166
167 class ManufacturedOven:
168     def __init__(self, kind: str):
169         self.kind = kind
170
171     def __call__(self, t):
172         Ta, Ca = manufactured_env(R, float(t))
173         return Ta if self.kind == "T" else Ca
174
175
176 def manufactured_sources(t: float, r: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
177     T, C = manufactured_fields(r, t)
178     x2 = (r / R) ** 2
179     u = np.exp(-t / 600.0)
180     v = np.exp(-t / 500.0)
181     Tt = 2.0 * (-1.0 / 600.0) * u * (1.0 + x2)
182     Ct = 0.2 * (-1.0 / 500.0) * v * (1.0 + x2)
183     Tr = 4.0 * r / (R ** 2) * u
184     Trr = np.full_like(r, 4.0 / (R ** 2) * u)
185     Cr = 0.4 * r / (R ** 2) * v
186     Crr = np.full_like(r, 0.4 / (R ** 2) * v)
187     lapT = np.empty_like(r)
188     lapC = np.empty_like(r)
189     center = r < 1e-16
190     lapT[center] = 2.0 * Trr[center]
191     lapC[center] = 2.0 * Crr[center]
192     away = ~center
193     lapT[away] = Trr[away] + Tr[away] / r[away]
194     lapC[away] = Crr[away] + Cr[away] / r[away]
195     kn = np.asarray(conductivity(C), dtype=float)
196     kp = np.asarray(conductivity_deriv(C), dtype=float)
197     Bn = np.asarray(heat_capacity(C), dtype=float)
198     D, dD_dC, dD_dT = diffusivity_partials(C, T)
199     ST = Bn * Tt - (kn * lapT + kp * Cr * Tr)
200     SC = Ct - (D * lapC + dD_dC * Cr ** 2 + dD_dT * Tr * Cr)
201     return ST, SC
202

```

```

203
204 def conservation_moisture(fvm, times, Y, Ca_fun) -> dict:
205     n = fvm.n_nodes
206     C = Y[:, n:]
207     M = C @ fvm.W
208     M0 = float(M[0])
209     CN = C[:, -1]
210     Ca = np.array([float(Ca_fun(t)) for t in times])
211     I = float(np.trapezoid(HM * (CN - Ca), times))
212     rhs = M0 - R * I
213     Mt = float(M[-1])
214     denom = max(abs(M0 - Mt), abs(R * I), 1e-18)
215     return {"M0": M0, "M_end": Mt, "rhs": rhs, "relative_residual": abs(Mt - rhs) / denom}
216
217
218 def conservation_heat(fvm, times, Y, Ta_fun) -> dict:
219     n = fvm.n_nodes
220     T = Y[:, :n]
221     C = Y[:, n:]
222     Bn = np.asarray(heat_capacity(C), dtype=float)
223     Bp = np.asarray(heat_capacity_deriv(C), dtype=float)
224     E = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bn * (T - TREF), axis=1)
225     TN = T[:, -1]
226     Ta = np.array([float(Ta_fun(t)) for t in times])
227     Q = 2.0 * np.pi * R * L * H * float(np.trapezoid(Ta - TN, times))
228     dC = np.gradient(C, times, axis=0)
229     corr = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bp * (T - TREF) * dC, axis=1)
230     Icorr = float(np.trapezoid(corr, times))
231     lhs = float(E[-1] - E[0] - Icorr)
232     denom = max(abs(Q), abs(lhs), 1e-12)
233     return {
234         "E_end_minus_E0": float(E[-1] - E[0]),
235         "composition_correction": Icorr,
236         "Q_surface": Q,
237         "relative_residual": abs(lhs - Q) / denom,
238     }
239
240
241 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
242     Ta_max = float(np.max(Ta_fun.y))
243     Ca_min = float(np.min(Ca_fun.y))
244     info = {
245         "T_min": float(T_out.min()),
246         "T_max": float(T_out.max()),
247         "C_min": float(C_out.min()),
248         "C_max": float(C_out.max()),
249         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
250         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),
251         "C_above_C0": float(max(0.0, C_out.max() - C0)),
252         "C_below-Ca_min": float(max(0.0, Ca_min - C_out.min())),
253         "Ta_range": [float(np.min(Ta_fun.y)), Ta_max],
254         "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
255     }
256     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-4
257     info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_below-Ca_min"] <= 1e-8
258     return info
259
260
261 def write_result_workbook(path: Path, T_out, C_out, times):
262     radii_cm = 0.1 * np.arange(21)
263     wb = Workbook()
264     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
265         ws = wb.create_sheet(title)
266         ws["A1"] = " 时间\\到药材中心的距离"
267         for j, rc in enumerate(radii_cm, start=2):

```

```

268         ws.cell(1, j, float(rc))
269     for i, t in enumerate(times):
270         ws.cell(i + 2, 1, int(round(float(t))))
271         for j in range(21):
272             cell = ws.cell(i + 2, j + 2, float(arr[i, j]))
273             cell.number_format = "0.0000"
274 if "Sheet" in wb.sheetnames:
275     del wb["Sheet"]
276 path.parent.mkdir(parents=True, exist_ok=True)
277 wb.save(path)
278
279
280 def write_paper_csv(path: Path, arr_out, times):
281     t_idx = {float(t): i for i, t in enumerate(times)}
282     r_idx = {round(0.1 * j, 10): j for j in range(21)}
283     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
284     for th, t in zip(TABLE_HOURS, TABLE_TIMES):
285         row = [f"{th:g}"]
286         i = t_idx[float(t)]
287         for rc in TABLE_RADII_CM:
288             j = r_idx[float(rc)]
289             row.append(f"{arr_out[i, j]:.8f}")
290         lines.append(",".join(row))
291     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
292
293
294 def setup_mpl():
295     import matplotlib
296     matplotlib.use("Agg")
297     import matplotlib.pyplot as plt
298     plt.rcParams.update({
299         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],
300         "axes.unicode_minus": False,
301         "font.size": 9,
302         "axes.labelsize": 10,
303         "legend.fontsize": 8,
304         "xtick.labelsize": 8,
305         "ytick.labelsize": 8,
306         "axes.linewidth": 0.8,
307         "pdf.fonttype": 42,
308         "ps.fonttype": 42,
309     })
310     return plt
311
312
313 def plot_figures(plt, fvm, times, Y, T_out, C_out, Ta_fun, Ca_fun):
314     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
315     n = fvm.n_nodes
316     r_cm = fvm.r * 100.0
317     idx = {float(t): i for i, t in enumerate(times)}
318     marks = TABLE_TIMES
319
320     fig, ax = plt.subplots(figsize=(6.4, 3.8))
321     for t in marks:
322         ax.plot(r_cm, Y[idx[float(t)], :n], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{h}}$")
323     ax.set_xlabel(" 半径 $r$ (cm)")
324     ax.set_ylabel(" 温度 $T$ (°C)")
325     ax.set_xlim(0, 2)
326     ax.legend(frameon=False, ncol=3, loc="lower center", bbox_to_anchor=(0.5, 1.02))
327     fig.tight_layout()
328     fig.savefig(FIGURES_DIR / "q2_T_profiles.pdf")
329     plt.close(fig)
330
331     fig, ax = plt.subplots(figsize=(6.4, 3.8))
332     for t in marks:

```

```

333     ax.plot(r_cm, Y[idx[float(t)], n:], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{{h}}}$")
334 ax.set_xlabel(" 半径 $r$ (cm)")
335 ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
336 ax.set_xlim(0, 2)
337 ax.legend(frameon=False, ncol=2)
338 fig.tight_layout()
339 fig.savefig(FIGURES_DIR / "q2_C_profiles.pdf")
340 plt.close(fig)
341
342 t_plot = times
343 fig, ax = plt.subplots(figsize=(6.4, 3.8))
344 ax.plot(t_plot / 3600.0, [Ta_fun(t) for t in t_plot], lw=1.0, ls="--", label=" 烘房 $T_a$")
345 ax.plot(t_plot / 3600.0, T_out[:, -1], lw=1.2, label=" 表面")
346 ax.plot(t_plot / 3600.0, T_out[:, 0], lw=1.2, label=" 中心")
347 ax.set_xlabel(" 时间 $t$ (h)")
348 ax.set_ylabel(" 温度 ($^{\circ}$C)")
349 ax.set_xlim(0, 3)
350 ax.legend(frameon=False)
351 fig.tight_layout()
352 fig.savefig(FIGURES_DIR / "q2_T_center_surface.pdf")
353 plt.close(fig)
354
355 fig, ax = plt.subplots(figsize=(6.4, 3.8))
356 ax.plot(t_plot / 3600.0, C_out[:, -1], lw=1.2, label=" 表面")
357 ax.plot(t_plot / 3600.0, C_out[:, 0], lw=1.2, label=" 中心")
358 ax.set_xlabel(" 时间 $t$ (h)")
359 ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
360 ax.set_xlim(0, 3)
361 ax.legend(frameon=False)
362 fig.tight_layout()
363 fig.savefig(FIGURES_DIR / "q2_C_center_surface.pdf")
364 plt.close(fig)
365
366 D_c = moisture_diffusivity_q2(C_out[:, 0], T_out[:, 0])
367 D_s = moisture_diffusivity_q2(C_out[:, -1], T_out[:, -1])
368 fig, ax = plt.subplots(figsize=(6.4, 3.6))
369 ax.plot(t_plot / 3600.0, D_s, lw=1.2, label=" 表面 $D_s$")
370 ax.plot(t_plot / 3600.0, D_c, lw=1.2, label=" 中心 $D_s$")
371 ax.set_xlabel(" 时间 $t$ (h)")
372 ax.set_ylabel(r"$D$ (m$^2$/s)")
373 ax.set_xlim(0, 3)
374 ax.legend(frameon=False)
375 fig.tight_layout()
376 fig.savefig(FIGURES_DIR / "q2_D_center_surface.pdf")
377 plt.close(fig)
378
379
380 def run_mms(level: int = 4) -> dict:
381     r = graded_radial_nodes(level)
382     fvm = CoupledRadialFVM(r)
383     T0f, C0f = manufactured_fields(fvm.r, 0.0)
384     y0 = fvm.pack(T0f, C0f)
385     Ta_fun = ManufacturedOven("T")
386     Ca_fun = ManufacturedOven("C")
387     times, Y = integrate_coupled(
388         fvm, y0, 100.0, Ta_fun, Ca_fun,
389         rtol=1e-8, atol=1e-9, max_step=0.5,
390         source=manufactured_sources,
391         t_eval=np.arange(1.0, 100.5, 1.0),
392     )
393     n = fvm.n_nodes
394     T_num, C_num = Y[:, :n], Y[:, n:]
395     T_ex = np.stack([manufactured_fields(fvm.r, t)[0] for t in times])
396     C_ex = np.stack([manufactured_fields(fvm.r, t)[1] for t in times])
397     return {

```

```

398     "level": level,
399     "n_nodes": n,
400     "T_max_abs": float(np.max(np.abs(T_num - T_ex))),
401     "C_max_abs": float(np.max(np.abs(C_num - C_ex))),
402     "T_end_max_abs": float(np.max(np.abs(T_num[-1] - T_ex[-1]))),
403     "C_end_max_abs": float(np.max(np.abs(C_num[-1] - C_ex[-1]))),
404 }
405
406
407 def main():
408     t_wall0 = time.perf_counter()
409     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
410     Ta_fun, Ca_fun, knots = load_q2_inputs()
411     print("oven points", knots.size, "kind", Ta_fun.kind, "Ta(10800)=", Ta_fun(T_END), "Ca(10800)=",
412           ↪ Ca_fun(T_END))
413     D0 = float(moisture_diffusivity_q2(C0, T0))
414     print("initial D", D0, "k", float(conductivity(C0)), "B", float(heat_capacity(C0)))
415
416     print("manufactured-solution probe")
417     mms = run_mms(4)
418     print("MMS", mms)
419
420     mesh_levels = [2, 4]
421     rtol, atol_T, atol_C, max_step = 1e-9, 1e-8, 1e-10, 1.0
422     solutions = {}
423     for level in mesh_levels:
424         r = graded_radial_nodes(level)
425         fvm = CoupledRadialFVM(r)
426         tag = f"G{level}"
427         print(f"\n=== {tag} nodes={fvm.n_nodes} ===")
428         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
429         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
430         times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
431         times_pos = times[times > 0]
432         Y_pos = Y[times > 0]
433         T_out, C_out = extract_output(fvm, Y_pos)
434         solutions[level] = dict(
435             tag=tag, fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
436             T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
437         )
438         print("done", tag, "T", T_out.min(), T_out.max(), "C", C_out.min(), C_out.max(),
439               ↪ "C(t=1,s)", C_out[0, -1], "C(3h,s)", C_out[-1, -1], "C(3h,c)", C_out[-1, 0])
440
441     t = solutions[4]["times"]
442     conv = {
443         "T_G2_vs_G4": compare_on_output(solutions[2]["T_out"], solutions[4]["T_out"], t),
444         "C_G2_vs_G4": compare_on_output(solutions[2]["C_out"], solutions[4]["C_out"], t),
445     }
446     fourdec = {
447         "T_G2_vs_G4": four_decimal_stats(solutions[2]["T_out"], solutions[4]["T_out"], t),
448         "C_G2_vs_G4": four_decimal_stats(solutions[2]["C_out"], solutions[4]["C_out"], t),
449     }
450     print("conv", conv)
451     print("4dp", fourdec)
452     paper_ok = fourdec["T_G2_vs_G4"]["paper_changed_cells"] == 0 and
453     ↪ fourdec["C_G2_vs_G4"]["paper_changed_cells"] == 0
454     abs_ok = conv["T_G2_vs_G4"]["max_abs"] <= 2e-5 and conv["C_G2_vs_G4"]["max_abs"] <= 2e-5
455     need_g8 = not (paper_ok and abs_ok)
456     if need_g8:
457         print("G2 vs G4 not enough; running G8")
458         level = 8
459         r = graded_radial_nodes(level)
460         fvm = CoupledRadialFVM(r)
461         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
462         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])

```

```

461     times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
462     times_pos = times[times > 0]
463     Y_pos = Y[times > 0]
464     T_out, C_out = extract_output(fvm, Y_pos)
465     solutions[8] = dict(
466         tag="G8", fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
467         T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
468     )
469     t = solutions[8]["times"]
470     conv["T_G4_vs_G8"] = compare_on_output(solutions[4]["T_out"], solutions[8]["T_out"], t)
471     conv["C_G4_vs_G8"] = compare_on_output(solutions[4]["C_out"], solutions[8]["C_out"], t)
472     fourdec["T_G4_vs_G8"] = four_decimal_stats(solutions[4]["T_out"], solutions[8]["T_out"], t)
473     fourdec["C_G4_vs_G8"] = four_decimal_stats(solutions[4]["C_out"], solutions[8]["C_out"], t)
474     print("conv G4 G8", conv["T_G4_vs_G8"], conv["C_G4_vs_G8"])
475     pub_level = 8
476     paper_ok = fourdec["T_G4_vs_G8"]["paper_changed_cells"] == 0 and
477     ↪ fourdec["C_G4_vs_G8"]["paper_changed_cells"] == 0
478     abs_ok = conv["T_G4_vs_G8"]["max_abs"] <= 2e-5 and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
479 else:
480     pub_level = 4 if fourdec["C_G2_vs_G4"]["changed_cells"] or fourdec["T_G2_vs_G4"]["changed_cells"]
481     ↪ else 2
482     if pub_level == 2:
483         pub_level = 4
484     pub = solutions[pub_level]
485     fvm = pub["fvm"]
486     times = pub["times"]
487     print("publishing", pub["tag"], "paper_ok", paper_ok, "abs_ok", abs_ok)
488
489     print("time refinement")
490     y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
491     atol = np.concatenate([np.full(fvm.n_nodes, atol_T / 5), np.full(fvm.n_nodes, atol_C / 5)])
492     t2, Y2 = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol / 5, atol, 0.5)
493     T2, C2 = extract_output(fvm, Y2[t2 > 0])
494     time_sens = {"T": compare_on_output(pub["T_out"], T2, times), "C": compare_on_output(pub["C_out"], C2,
495     ↪ times)}
496     time_4dp = {"T": four_decimal_stats(pub["T_out"], T2, times), "C": four_decimal_stats(pub["C_out"], C2,
497     ↪ times)}
498     print("time sens", time_sens)
499
500     print("frozen-D moisture probe")
501     D_freeze = D0
502     Bi_m = HM * R / D_freeze
503     C_ref, moisture_truncation = converged_reference(Bi_m, D_freeze, R, OUTPUT_RADII_M, times, Ca_fun, C0)
504     y0f = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
505     atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
506     tf, Yf = integrate_coupled(fvm, y0f, T_END, Ta_fun, Ca_fun, rtol, atol, max_step, D_const=D_freeze)
507     _, C_frozen = extract_output(fvm, Yf[tf > 0])
508     frozen = compare_on_output(C_frozen, C_ref, times)
509     nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
510     print("frozen vs analytic", frozen)
511     print("nonlinear vs frozen", nl_vs_frozen)
512
513     n = fvm.n_nodes
514     D_field = moisture_diffusivity_q2(pub["Y"][:, n:], pub["Y"][:, :n])
515     D_stats = {
516         "D0": D0,
517         "D_min": float(np.min(D_field)),
518         "D_max": float(np.max(D_field)),
519         "D_min_over_D0": float(np.min(D_field) / D0),
520         "D_max_over_D0": float(np.max(D_field) / D0),
521         "center_C_end": float(pub["C_out"][-1, 0]),
522         "surface_C_end": float(pub["C_out"][-1, -1]),
523         "center_T_end": float(pub["T_out"][-1, 0]),
524         "surface_T_end": float(pub["T_out"][-1, -1]),
525     }
526 }

```

```

522     print("D stats", D_stats)
523
524     bounds = assert_physical(pub["Y_full"][:, :n], pub["Y_full"][:, n:], Ta_fun, Ca_fun)
525     cons_C = conservation_moisture(fvm, pub["times_full"], pub["Y_full"], Ca_fun)
526     cons_T = conservation_heat(fvm, pub["times_full"], pub["Y_full"], Ta_fun)
527     print("bounds", bounds)
528     print("conservation C", cons_C)
529     print("conservation T", cons_T)
530
531     delivery = make_gate({
532         "space_abs": abs_ok,
533         "space_paper": paper_ok,
534         "all_finite": np.isfinite(pub["Y_full"]).all(),
535         "manufactured_solution": mms["T_max_abs"] <= 2e-5 and mms["C_max_abs"] <= 2e-5,
536         "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
537         "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
538             ↪ 0,
539         "temperature_bounds": bounds["T_bounds_ok"],
540         "moisture_bounds": bounds["C_bounds_ok"],
541         "moisture_reference_truncation": moisture_truncation["passed"],
542         "frozen_reference_error": frozen["max_abs"] <= 2e-5,
543         "heat_balance": cons_T["relative_residual"] <= 1e-6,
544         "moisture_balance": cons_C["relative_residual"] <= 1e-6,
545     })
546     print("delivery", delivery)
547
548     paper_T = {f"h{th:g}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
549         for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
550     paper_C = {f"h{th:g}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
551         for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
552
553     validation = {
554         "result_version": "q2-closeout-v1",
555         "interpolation": Ta_fun.kind,
556         "published_mesh": pub["tag"],
557         "published_n_nodes": pub["n_nodes"],
558         "mesh_levels": list(solutions.keys()),
559         "solver": "coupled-FVM+BDF",
560         "rtol": rtol, "atol_T": atol_T, "atol_C": atol_C, "max_step": max_step,
561         "mms": mms,
562         "grid_convergence": conv,
563         "four_decimal_grid": fourdec,
564         "time_sensitivity": time_sens,
565         "four_decimal_time": time_4dp,
566         "delivery_gate": delivery,
567         "reference_truncation_C": moisture_truncation,
568         "frozen_D_moisture_vs_analytic": frozen,
569         "nonlinear_C_vs_frozen_D": nl_vs_frozen,
570         "D_variation": D_stats,
571         "conservation_T_10800s": cons_T,
572         "conservation_C_10800s": cons_C,
573         "physical_bounds": bounds,
574         "paper_table_T": paper_T,
575         "paper_table_C": paper_C,
576         "elapsed_s": time.perf_counter() - t_wall0,
577         "python": sys.version,
578         "end_face_2d": "skipped_by_plan",
579     }
580     require_delivery(validation, RESULTS_DIR / "diagnostics/q2_delivery/latest.json")
581     np.savez(
582         RESULTS_DIR / "q2_solution.npz",
583         mesh_level=pub_level,
584         n_nodes=pub["n_nodes"],
585         times=times,
586         r=fvm.r,

```

```

586     output_radai_m=OUTPUT_RADII_M,
587     checkpoint_times=np.r_[0., 1., 60., 100., TABLE_TIMES],
588     T_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), :n],
589     C_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), n:],
590     T_out=pub["T_out"],
591     C_out=pub["C_out"],
592     interpolation=np.array(Ta_fun.kind),
593     rtol=rtol, atol_T=atol_T, atol_C=atol_C, max_step=max_step,
594 )
595 write_result_workbook(RESULTS_DIR / "result2.xlsx", pub["T_out"], pub["C_out"], times)
596 write_paper_csv(RESULTS_DIR / "q2_table_temperature.csv", pub["T_out"], times)
597 write_paper_csv(RESULTS_DIR / "q2_table_moisture.csv", pub["C_out"], times)
598 plt = setup_mpl()
599 plot_figures(plt, fvm, times, pub["Y"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
600
601 (RESULTS_DIR / "q2_validation.json").write_text(
602     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
603     encoding="utf-8",
604 )
605 print("elapsed_s", validation["elapsed_s"])
606 print("wrote results to", RESULTS_DIR)
607
608
609
610 if __name__ == "__main__":
611     main()

```

B.9 code/problem3.py

```

1 """Problem 3: full coupled drying until every interior node is below 0.15 kg/kg."""
2 from __future__ import annotations
3
4 import argparse
5 import json
6 import sys
7 import time
8 from pathlib import Path
9
10 import numpy as np
11 from scipy.integrate import solve_ivp
12 from scipy.optimize import brentq
13
14 CODE_DIR = Path(__file__).resolve().parent
15 if str(CODE_DIR) not in sys.path:
16     sys.path.insert(0, str(CODE_DIR))
17
18 from appendix3 import moisture_diffusivity_q2
19 from delivery import make_gate, require_delivery
20 from problem2 import conservation_heat, conservation_moisture, extract_output, setup_mpl
21 from radial_coupled import CoupledRadialFVM
22 from radial_fvm import graded_radial_nodes
23 from utils import (
24     C0,
25     FIGURES_DIR,
26     RESULTS_DIR,
27     T0,
28     PchipOven,
29     load_oven_table,
30 )
31
32 T_SWITCH = 14400.0
33 TA_AFTER = 50.0
34 CA_AFTER = 0.05
35 C_THRESH = 0.15
36 T_HARD_CAP = 21.0 * 24.0 * 3600.0

```

```

37 OUTPUT_RADII_M = 0.001 * np.arange(21)
38 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
39 EXCEL_DT = 60.0
40 TABLE_DT_H = 6.0
41 OVEN_N_EXPECTED = 241
42
43 # First-round baseline; recorded after the run, not treated as a prior user decision.
44 RTOL = 1e-9
45 ATOL_T = 1e-8
46 ATOL_C = 1e-10
47 MAX_STEP_EARLY = 2.0
48 MAX_STEP_LATE = 60.0
49
50
51 class ExtendedOven:
52     """PCHIP through attachment 1 on [0, 4 h]; constant value strictly after 4 h."""
53
54     def __init__(self, pchip: PchipOven, after_value: float, t_switch: float = T_SWITCH):
55         self.pchip = pchip
56         self.after = float(after_value)
57         self.t_switch = float(t_switch)
58         self.kind = pchip.kind
59         self.t = pchip.t
60         self.y = pchip.y
61
62     def __call__(self, t):
63         if np.ndim(t) == 0:
64             tt = float(t)
65             if tt <= self.t_switch:
66                 return float(self.pchip(tt))
67             return self.after
68         tt = np.asarray(t, dtype=float)
69         out = np.empty_like(tt, dtype=float)
70         inside = tt <= self.t_switch
71         if np.any(inside):
72             out[inside] = np.asarray(self.pchip(tt[inside]), dtype=float)
73         out[~inside] = self.after
74         return out
75
76
77 def load_q3_inputs() -> tuple[ExtendedOven, ExtendedOven, np.ndarray]:
78     t, Ta, Ca = load_oven_table()
79     if t.size != OVEN_N_EXPECTED or abs(t[-1] - T_SWITCH) > 1e-12:
80         raise ValueError(f"expected {OVEN_N_EXPECTED} oven records ending at {T_SWITCH}, got {t.size},
81             ↳ t_end={t[-1]}")
82     if abs(Ta[0] - T0) > 1e-12:
83         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
84     Ta_pchip = PchipOven(t, Ta)
85     Ca_pchip = PchipOven(t, Ca)
86     for tj, yj in zip(t, Ta):
87         if abs(float(Ta_pchip(tj)) - float(yj)) > 1e-12:
88             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
89     for tj, yj in zip(t, Ca):
90         if abs(float(Ca_pchip(tj)) - float(yj)) > 1e-12:
91             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
92     Ta_fun = ExtendedOven(Ta_pchip, TA_AFTER)
93     Ca_fun = ExtendedOven(Ca_pchip, CA_AFTER)
94     jump = {
95         "Ta_at_4h": float(Ta_fun(T_SWITCH)),
96         "Ca_at_4h": float(Ca_fun(T_SWITCH)),
97         "Ta_after": float(Ta_fun(T_SWITCH + 1e-9)),
98         "Ca_after": float(Ca_fun(T_SWITCH + 1e-9)),
99     }
100     print("oven 4h jump", jump, flush=True)
101     return Ta_fun, Ca_fun, t

```

```

101
102
103 def atol_vector(n: int, atol_T: float = ATOL_T, atol_C: float = ATOL_C) -> np.ndarray:
104     return np.concatenate([np.full(n, float(atol_T)), np.full(n, float(atol_C))])
105
106
107 def max_moisture_event(n_nodes: int):
108     def event(t, y):
109         return float(np.max(y[n_nodes:]) - C_THRESH)
110
111     event.terminal = True
112     event.direction = -1.0
113     return event
114
115
116 def integrate_segment(
117     fvm: CoupledRadialFVM,
118     y0,
119     t0: float,
120     t1: float,
121     Ta_fun,
122     Ca_fun,
123     rtol: float,
124     atol: np.ndarray,
125     max_step: float,
126     *,
127     t_eval=None,
128     events=None,
129     dense_output: bool = False,
130     first_step=None,
131     label: str = "",
132     balance_audit: bool = False,
133 ):
134     n = fvm.n_nodes
135     y0 = np.asarray(y0, dtype=float)
136     t0 = float(t0)
137     t1 = float(t1)
138     if t1 <= t0 + 1e-15:
139         raise ValueError(f"empty interval [{t0}, {t1}]")
140     nfev = [0]
141     t_print = [t0]
142     t_wall0 = time.perf_counter()
143
144     def rhs(t, y):
145         nfev[0] += 1
146         interval = 3600.0 if t < T_SWITCH + 12.0 * 3600.0 else 6.0 * 3600.0
147         if t - t_print[0] >= interval:
148             C = y[n:]
149             imax = int(np.argmax(C))
150             elapsed = time.perf_counter() - t_wall0
151             print(
152                 f"    {label}t={t/3600.0:.3f} h    maxC={float(C[imax]):.6f} "
153                 f"rmax={fvm.r[imax]*100:.4f} cm    nfev={nfev[0]}    wall={elapsed:.1f}s",
154                 flush=True,
155             )
156             t_print[0] = float(t)
157         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)))
158
159     kwargs = dict(
160         fun=rhs,
161         t_span=(t0, t1),
162         y0=y0,
163         method="BDF",
164         rtol=rtol,
165         atol=np.asarray(atol, dtype=float),

```

```

166         max_step=max_step,
167         dense_output=dense_output or balance_audit,
168         vectorized=False,
169         jac_sparsity=fvm.jac_sparsity(),
170     )
171     if t_eval is not None:
172         te = np.asarray(t_eval, dtype=float)
173         te = te[(te > t0 + 1e-14) & (te <= t1 + 1e-12)]
174         if te.size:
175             kwargs["t_eval"] = te
176     if events is not None:
177         kwargs["events"] = events
178     if first_step is not None:
179         kwargs["first_step"] = min(float(first_step), 0.5 * max_step, t1 - t0)
180
181     sol = solve_ivp(**kwargs)
182     if not sol.success:
183         raise RuntimeError(f"coupled BDF failed on [{t0}, {t1}] {label}: {sol.message}")
184     if balance_audit:
185         from q3_balance import audit_segment
186         sol.balance_audit = audit_segment(fvm, sol, Ta_fun, Ca_fun)
187     if not dense_output:
188         sol.sol = None
189     return sol
190
191
192 def excel_times(t_rep: float) -> np.ndarray:
193     t_rep = float(t_rep)
194     grid = np.arange(EXCEL_DT, np.floor(t_rep / EXCEL_DT) * EXCEL_DT + 0.5 * EXCEL_DT, EXCEL_DT)
195     if grid.size == 0 or grid[-1] < t_rep - 1e-12:
196         times = np.concatenate([grid, [t_rep]]) if grid.size else np.array([t_rep])
197     elif abs(grid[-1] - t_rep) <= 1e-9:
198         times = grid
199     else:
200         times = np.concatenate([grid, [t_rep]])
201     return times
202
203
204 def table_hours(t_rep_h: float) -> np.ndarray:
205     t_rep_h = float(t_rep_h)
206     hrs = np.arange(TABLE_DT_H, t_rep_h + 1e-12, TABLE_DT_H)
207     if hrs.size == 0 or abs(hrs[-1] - t_rep_h) > 1e-10:
208         hrs = np.concatenate([hrs, [t_rep_h]]) if hrs.size else np.array([t_rep_h])
209     return hrs
210
211
212 def monitor_from_Y(fvm: CoupledRadialFVM, times: np.ndarray, Y: np.ndarray) -> dict:
213     n = fvm.n_nodes
214     C = Y[:, n:]
215     imax = np.argmax(C, axis=1)
216     M = C[np.arange(C.shape[0]), imax]
217     rmax = fvm.r[imax]
218     dC = np.diff(C, axis=1)
219     # Allow tiny positive bumps from rounding; count clear interior increases.
220     n_increase = int(np.sum(dC > 1e-10))
221     center_is_max = imax == 0
222     return {
223         "times": times,
224         "M": M,
225         "rmax_m": rmax,
226         "imax": imax,
227         "center_is_max_frac": float(np.mean(center_is_max)),
228         "n_radial_increases": n_increase,
229         "C_center": C[:, 0].copy(),
230         "C_surface": C[:, -1].copy(),

```

```

231         "T_center": Y[:, 0].copy(),
232         "T_surface": Y[:, n - 1].copy(),
233     }
234
235
236 def locate_threshold(
237     fvm: CoupledRadialFVM,
238     Ta_fun,
239     Ca_fun,
240     t_left: float,
241     y_left: np.ndarray,
242     t_right: float,
243     rtol: float,
244     atol: np.ndarray,
245 ):
246     """Bracketed root of max_i C_i(t) - 0.15 on a locally refined dense interpolant."""
247     n = fvm.n_nodes
248     event = max_moisture_event(n)
249     t_pad = max(30.0, 2.0 * (t_right - t_left))
250     t_hi = t_right + t_pad
251     sol = integrate_segment(
252         fvm, y_left, t_left, t_hi, Ta_fun, Ca_fun, rtol, atol,
253         max_step=0.5, events=event, dense_output=True, first_step=0.05,
254         label="zoom ",
255     )
256     if sol.t_events is None or len(sol.t_events[0]) == 0:
257         C_end = sol.y[n:, -1]
258         raise RuntimeError(
259             f"zoom missed threshold on [{t_left}, {t_hi}]; "
260             f"maxC(end)={float(np.max(C_end))}"
261         )
262     t_solver = float(sol.t_events[0][0])
263     y_solver = np.asarray(sol.y_events[0][0], dtype=float)
264
265     def g(t):
266         return float(np.max(sol.sol(float(t))[n:]) - C_THRESH)
267
268     g_left = g(t_left)
269     t_lo = t_left
270     t_hi_root = float(sol.t[-1])
271     g_hi = g(t_hi_root)
272     if g_left <= 0.0:
273         raise RuntimeError(f"zoom left end already <= threshold: t={t_left}, g={g_left}")
274     if g_hi > 0.0 and abs(g_hi) <= 1e-8:
275         t_star = t_solver
276     elif g_hi > 0.0:
277         raise RuntimeError(f"zoom right end still above threshold: t={t_hi_root}, g={g_hi}")
278     else:
279         t_star = float(brentq(g, t_lo, t_hi_root, xtol=1e-4, rtol=1e-12, maxiter=200))
280     y_star = np.asarray(sol.sol(t_star), dtype=float)
281     C_star = y_star[n:]
282     imax = int(np.argmax(C_star))
283     info = {
284         "t_star_s": t_star,
285         "t_star_h": t_star / 3600.0,
286         "t_solver_event_s": t_solver,
287         "event_solver_minus_brent_s": t_solver - t_star,
288         "M_star": float(C_star[imax]),
289         "g_star": float(C_star[imax] - C_THRESH),
290         "rmax_star_m": float(fvm.r[imax]),
291         "rmax_star_cm": float(fvm.r[imax] * 100.0),
292         "imax_star": imax,
293         "center_is_max": bool(imax == 0),
294         "C_center_star": float(C_star[0]),
295         "C_surface_star": float(C_star[-1]),

```

```

296         "T_center_star": float(y_star[0]),
297         "T_surface_star": float(y_star[n - 1]),
298         "bracket_s": [float(t_left), float(t_hi_root)],
299         "zoom_nfev": int(sol.nfev),
300     }
301     return t_star, y_star, sol, info, y_solver
302
303
304 def find_strict_report(eval_M, t_star: float, t_hi: float, margin: float = 0.0) -> dict:
305     target = C_THRESH - float(margin)
306     t_star = float(t_star)
307     t_hi = float(t_hi)
308     if not (np.isfinite([t_star, t_hi, margin]).all() and 0 <= margin < C_THRESH and t_hi > t_star):
309         raise ValueError('invalid strict-report interval or margin')
310
311     original_eval = eval_M
312     def eval_M(t):
313         if not t_star <= t <= t_hi:
314             raise RuntimeError('strict report requested outside verified continuation')
315         value = float(original_eval(t))
316         if not np.isfinite(value):
317             raise RuntimeError('nonfinite moisture in strict report')
318         return value
319
320     def h(t):
321         return float(eval_M(t) - target)
322
323     h0 = h(t_star)
324     h1 = h(t_hi)
325     if h1 >= 0.0:
326         raise RuntimeError(f"M(t_hi)={eval_M(t_hi)} still >= {target}")
327     if h0 < 0.0:
328         t_cross = t_star
329     else:
330         t_cross = float(brentq(h, t_star, t_hi, xtol=1e-4, rtol=1e-12, maxiter=200))
331     t_rep = float(np.floor(max(t_cross, t_star)) + 1.0)
332     # Walk forward in seconds until unrounded moisture is strictly below the target.
333     for _ in range(10000):
334         if float(eval_M(t_rep)) < target:
335             break
336         t_rep += 1.0
337     else:
338         raise RuntimeError("failed to find integer-second report time below threshold")
339     t_hour4 = np.ceil((t_rep / 3600.0) * 10000.0) / 10000.0
340     t_hour4_s = 3600.0 * t_hour4
341     if float(eval_M(t_hour4_s)) >= target:
342         raise RuntimeError("four-decimal hour report does not satisfy the target")
343     t_grid60 = EXCEL_DT * np.ceil(t_star / EXCEL_DT)
344     if t_grid60 < t_rep:
345         t_grid60 = EXCEL_DT * np.ceil(t_rep / EXCEL_DT)
346     return {
347         "t_cross_s": t_cross,
348         "t_rep_s": float(t_rep),
349         "t_rep_h": float(t_rep) / 3600.0,
350         "M_rep": float(eval_M(t_rep)),
351         "t_hour4_h": float(t_hour4),
352         "t_hour4_s": float(t_hour4_s),
353         "M_hour4": float(eval_M(t_hour4_s)),
354         "t_grid60_s": float(t_grid60),
355         "M_grid60": float(eval_M(t_grid60)),
356         "margin": float(margin),
357         "strict_below_rep": bool(float(eval_M(t_rep)) < C_THRESH),
358         "strict_below_hour4": bool(float(eval_M(t_hour4_s)) < C_THRESH),
359     }
360

```

```

361
362 def slope_at(eval_M, t: float, h: float = 1.0) -> float:
363     return float(eval_M(t + h) - eval_M(t - h)) / (2.0 * h)
364
365
366 def run_case(
367     level: int,
368     Ta_fun,
369     Ca_fun,
370     t_max: float,
371     rtol: float,
372     atol_T: float,
373     atol_C: float,
374     max_step_early: float,
375     max_step_late: float,
376     tag: str,
377     audit_balances: bool = False,
378 ):
379     r = graded_radial_nodes(level)
380     fvm = CoupledRadialFVM(r)
381     n = fvm.n_nodes
382     print(f"\n=== {tag} nodes={n} t_max={t_max/3600:.2f} h ===", flush=True)
383     y0 = fvm.pack(np.full(n, T0), np.full(n, C0))
384     atol = atol_vector(n, atol_T, atol_C)
385     t_eval_early = np.arange(EXCEL_DT, T_SWITCH + 0.5 * EXCEL_DT, EXCEL_DT)
386     t_wall0 = time.perf_counter()
387     sol1 = integrate_segment(
388         fvm, y0, 0.0, T_SWITCH, Ta_fun, Ca_fun, rtol, atol,
389         max_step=max_step_early, t_eval=t_eval_early, first_step=0.05, label=f"{tag} early ",
390         ↪ balance_audit=audit_balances,
391     )
392     y_switch = sol1.y[:, -1]
393     times1 = np.concatenate([[0.0], sol1.t])
394     Y1 = np.vstack([y0, sol1.y.T])
395     M_switch = float(np.max(y_switch[n:]))
396     print(f" {tag} 4h maxC={M_switch:.6f} C_center={y_switch[n]:.6f}
397     ↪ wall={time.perf_counter()-t_wall0:.1f}s", flush=True)
398     if M_switch <= C_THRESH:
399         raise RuntimeError("threshold already reached at 4 h; unexpected for this model")
400
401     event = max_moisture_event(n)
402     t_eval_late = np.unique(np.r_[np.arange(T_SWITCH + EXCEL_DT, t_max, EXCEL_DT), t_max])
403     sol2 = integrate_segment(
404         fvm, y_switch, T_SWITCH, t_max, Ta_fun, Ca_fun, rtol, atol,
405         max_step=max_step_late, t_eval=t_eval_late, events=event,
406         first_step=0.2, label=f"{tag} late ", dense_output=audit_balances,
407     )
408     if sol2.t_events is None or len(sol2.t_events[0]) == 0:
409         M_end = float(np.max(sol2.y[n:, -1]))
410         print(f" {tag} no event by {t_max/3600:.2f} h, maxC={M_end:.6f}; extending to hard cap",
411             ↪ flush=True)
412         if t_max >= T_HARD_CAP - 1.0:
413             raise RuntimeError(f"{tag} still above threshold at {T_HARD_CAP/3600:.1f} h, maxC={M_end}")
414         y_cont = sol2.y[:, -1]
415         t_cont = float(sol2.t[-1])
416         t_eval_cap = np.arange(t_cont + EXCEL_DT, T_HARD_CAP + 0.5 * EXCEL_DT, EXCEL_DT)
417         sol2b = integrate_segment(
418             fvm, y_cont, t_cont, T_HARD_CAP, Ta_fun, Ca_fun, rtol, atol,
419             max_step=max_step_late, t_eval=t_eval_cap, events=event,
420             first_step=0.2, label=f"{tag} cap ", dense_output=audit_balances,
421         )
422         if sol2b.t_events is None or len(sol2b.t_events[0]) == 0:
423             raise RuntimeError(
424                 f"{tag} not dry at {T_HARD_CAP/3600:.1f} h, maxC={float(np.max(sol2b.y[n:, -1]))}"
425             )

```

```

423     times2 = np.concatenate([sol2.t[sol2.t > T_SWITCH + 1e-12], sol2b.t[sol2b.t > t_cont + 1e-12]])
424     Y2 = np.vstack([sol2.y[:, sol2.t > T_SWITCH + 1e-12].T, sol2b.y[:, sol2b.t > t_cont + 1e-12].T])
425     t_event_raw = float(sol2b.t_events[0][0])
426     y_event_raw = np.asarray(sol2b.y_events[0][0], dtype=float)
427     else:
428         times2 = sol2.t[sol2.t > T_SWITCH + 1e-12]
429         Y2 = sol2.y[:, sol2.t > T_SWITCH + 1e-12].T
430         t_event_raw = float(sol2.t_events[0][0])
431         y_event_raw = np.asarray(sol2.y_events[0][0], dtype=float)
432
433     times = np.concatenate([times1, times2])
434     Y = np.vstack([Y1, Y2])
435     order = np.argsort(times)
436     times, Y = times[order], Y[order]
437     uniq = np.concatenate([[True], np.diff(times) > 1e-12])
438     times, Y = times[uniq], Y[uniq]
439
440     M_hist = np.max(Y[:, n:], axis=1)
441     above = np.where(M_hist > C_THRESH + 1e-14)[0]
442     if above.size == 0:
443         raise RuntimeError(f"{tag} never above threshold")
444     i_left = int(above[-1])
445     t_left = float(times[i_left])
446     y_left = Y[i_left]
447     if t_left >= t_event_raw:
448         i_left = max(0, i_left - 1)
449         t_left = float(times[i_left])
450         y_left = Y[i_left]
451     t_star, y_star, sol_zoom, event_info, y_solver = locate_threshold(
452         fvm, Ta_fun, Ca_fun, t_left, y_left, t_event_raw, rtol, atol,
453     )
454     event_info["t_event_raw_s"] = t_event_raw
455     event_info["M_event_raw"] = float(np.max(y_event_raw[n:]))
456
457     balance_parts = []
458     if audit_balances:
459         from q3_balance import audit_segment
460         # Follow the same trajectory used for reporting, including the zoom restart.
461         balance_parts = [sol1.balance_audit]
462         if 'sol2b' in locals():
463             balance_parts.append(audit_segment(fvm, sol2, Ta_fun, Ca_fun))
464             late_sol = sol2b
465         else:
466             late_sol = sol2
467         balance_parts.append(audit_segment(fvm, late_sol, Ta_fun, Ca_fun, t_end=t_left))
468         balance_parts.append(audit_segment(fvm, sol_zoom, Ta_fun, Ca_fun, t_end=t_star))
469         sol2.sol = None
470         if 'sol2b' in locals():
471             sol2b.sol = None
472
473     t_pad = 3600.0
474     sol_after = integrate_segment(
475         fvm, y_star, t_star, t_star + t_pad, Ta_fun, Ca_fun, rtol, atol,
476         max_step=min(max_step_late, 5.0), dense_output=True, first_step=0.05,
477         label=f"{tag} after ",
478     )
479
480     def eval_M(t):
481         return float(np.max(eval_y(float(t))[n:]))
482
483     def eval_y(t):
484         tt = float(t)
485         if abs(tt - t_star) <= 1e-12:
486             return y_star.copy()
487         if tt < t_star:

```

```

488         if tt < times[0]:
489             raise ValueError(f"state requested before initial time: {tt}")
490         if tt >= t_left:
491             return np.asarray(sol_zoom.sol(tt), dtype=float)
492         k = max(0, int(np.searchsorted(times, tt, side="right"))-1)
493         if abs(times[k]-tt)<1e-8:
494             return Y[k].copy()
495         short = integrate_segment(fvm,Y[k],float(times[k]),tt,Ta_fun,Ca_fun,
496                                 min(rtol,1e-10),atol/10,max_step=1.0,first_step=0.05,label="matched time ")
497         return short.y[:,-1]
498     if tt > float(sol_after.t[-1])+1e-9:
499         raise ValueError(f"state requested beyond continuation: {tt}")
500     return np.asarray(sol_after.sol(tt), dtype=float)
501
502     dMdt = slope_at(eval_M, min(t_star + 2.0, t_star + 0.5 * t_pad), h=1.0)
503     event_info["dMdt_per_s"] = dMdt
504     event_info["dMdt_per_h"] = dMdt * 3600.0
505
506     T_out, C_out = extract_output(fvm, Y)
507     mon = monitor_from_Y(fvm, times, Y)
508     elapsed = time.perf_counter() - t_wall0
509     print(
510         f" {tag} t*={t_star/3600:.6f} h  rmax={event_info['rmax_star_cm']:.4f} cm  "
511         f"M*={event_info['M_star']:.8f}  dM/dt={dMdt:.3e}/s  wall={elapsed:.1f}s",
512         flush=True,
513     )
514     return dict(
515         tag=tag, level=level, fvm=fvm, n_nodes=n, times=times, Y=Y,
516         T_out=T_out, C_out=C_out, t_star=t_star, y_star=y_star,
517         eval_M=eval_M, eval_y=eval_y, sol_after=sol_after, event=event_info,
518         monitor=mon, elapsed_s=elapsed, rtol=rtol, atol_T=atol_T, atol_C=atol_C,
519         max_step_early=max_step_early, max_step_late=max_step_late, t_max=t_max,
520         y_switch=y_switch,
521         balance_parts=balance_parts,
522     )
523
524
525 def run_probe(Ta_fun, Ca_fun) -> dict:
526     """Cheap uniform-mesh estimate of t* to size the production interval."""
527     r = np.linspace(0.0, 0.02, 41)
528     fvm = CoupledRadialFVM(r)
529     n = fvm.n_nodes
530     y0 = fvm.pack(np.full(n, T0), np.full(n, C0))
531     atol = atol_vector(n, 1e-6, 1e-8)
532     print("\n=== probe N=41 ===", flush=True)
533     sol1 = integrate_segment(
534         fvm, y0, 0.0, T_SWITCH, Ta_fun, Ca_fun, 1e-6, atol, max_step=30.0,
535         first_step=0.2, label="probe early ",
536     )
537     event = max_moisture_event(n)
538     sol2 = integrate_segment(
539         fvm, sol1.y[:, -1], T_SWITCH, T_HARD_CAP, Ta_fun, Ca_fun, 1e-6, atol,
540         max_step=120.0, events=event, first_step=1.0, label="probe late ",
541     )
542     if sol2.t_events is None or len(sol2.t_events[0]) == 0:
543         raise RuntimeError(f"probe not dry by 14 d, maxC={float(np.max(sol2.y[n:, -1]))}")
544     t_star = float(sol2.t_events[0][0])
545     y_e = np.asarray(sol2.y_events[0][0], dtype=float)
546     imax = int(np.argmax(y_e[n:]))
547     info = {
548         "n_nodes": n,
549         "t_star_s": t_star,
550         "t_star_h": t_star / 3600.0,
551         "M_star": float(y_e[n + imax]),
552         "rmax_cm": float(fvm.r[imax] * 100.0),

```

```

553         "C_center": float(y_e[n]),
554         "C_surface": float(y_e[-1]),
555         "nfev": int(sol1.nfev + sol2.nfev),
556     }
557     print("probe", info, flush=True)
558     return info
559
560
561 def overlap_q2(case) -> dict:
562     path = RESULTS_DIR / "q2_solution.npz"
563     if not path.exists():
564         return {"available": False}
565     z = np.load(path)
566     fvm = case["fvm"]
567     n = fvm.n_nodes
568     idx = fvm.output_indices()
569     t_check = np.array([1800.0, 3600.0, 7200.0, 9000.0])
570     q2_times = np.asarray(z["times"], dtype=float)
571     T_q2 = np.asarray(z["T_out"], dtype=float)
572     C_q2 = np.asarray(z["C_out"], dtype=float)
573     rows = {}
574     for t in t_check:
575         k = int(np.argmin(np.abs(case["times"] - t)))
576         if abs(case["times"][k] - t) > 1e-8:
577             rows[f"t{int(t)}"] = {"matched": False}
578             continue
579         jq = int(np.argmin(np.abs(q2_times - t)))
580         if abs(q2_times[jq] - t) > 1e-8:
581             rows[f"t{int(t)}"] = {"matched": False}
582             continue
583         T = case["Y"][k, :n][idx]
584         C = case["Y"][k, n:][idx]
585         rows[f"t{int(t)}"] = {
586             "matched": True,
587             "T_max_abs": float(np.max(np.abs(T - T_q2[jq]))),
588             "C_max_abs": float(np.max(np.abs(C - C_q2[jq]))),
589             "T_center_diff": float(T[0] - T_q2[jq, 0]),
590             "C_center_diff": float(C[0] - C_q2[jq, 0]),
591             "C_surface_diff": float(C[-1] - C_q2[jq, -1]),
592         }
593     return {"available": True, "note": "PCHIP stencils differ near 3 h endpoint", "checks": rows}
594
595
596 def sample_state(case, t: float) -> tuple[np.ndarray, np.ndarray]:
597     fvm = case["fvm"]
598     n = fvm.n_nodes
599     y = case["eval_y"](float(t))
600     T, C = y[:n], y[n:]
601     return T, C
602
603
604 def output_row(fvm, C: np.ndarray) -> np.ndarray:
605     return np.asarray(C[fvm.output_indices()], dtype=float)
606
607
608 def build_output_tables(case, t_rep: float) -> dict:
609     fvm = case["fvm"]
610     times_x = excel_times(t_rep)
611     C_excel = np.empty((times_x.size, 21), dtype=float)
612     T_excel = np.empty_like(C_excel)
613     n = fvm.n_nodes
614     idx = fvm.output_indices()
615     t_star = case["t_star"]
616     times = case["times"]
617     Y = case["Y"]

```

```

618     for i, t in enumerate(times_x):
619         if t <= times[-1] + 1e-12 and t < t_star - 1e-9:
620             j = int(np.argmin(np.abs(times - t)))
621             if abs(times[j] - t) > 1.0:
622                 raise RuntimeError(f"missing 60 s sample at t={t}, nearest {times[j]}")
623             T_excel[i] = Y[j, :n][idx]
624             C_excel[i] = Y[j, n:][idx]
625         else:
626             y = case["eval_y"](float(t))
627             T_excel[i] = y[:n][idx]
628             C_excel[i] = y[n:][idx]
629     hrs = table_hours(t_rep / 3600.0)
630     C_table = np.empty((hrs.size, TABLE_RADII_CM.size), dtype=float)
631     paper_map = {}
632     r_idx = {round(0.1 * j, 10): j for j in range(21)}
633     for i, th in enumerate(hrs):
634         t = 3600.0 * float(th)
635         k = int(np.argmin(np.abs(times_x - t)))
636         if abs(times_x[k] - t) > 1.0 and abs(t - t_rep) > 1.0:
637             # 6 h marks lie on the 60 s grid; end time may be the extra row.
638             y = case["eval_y"](t) if t >= t_star - 1e-9 else None
639             if y is None:
640                 j = int(np.argmin(np.abs(times - t)))
641                 row = Y[j, n:][idx]
642             else:
643                 row = y[n:][idx]
644         else:
645             row = C_excel[k]
646         for rc, col in zip(TABLE_RADII_CM, range(5)):
647             C_table[i, col] = float(row[r_idx[float(rc)]]])
648             paper_map[f"h{th:.6g}_r{rc:g}"] = float(row[r_idx[float(rc)]]])
649     return {
650         "excel_times": times_x,
651         "C_excel": C_excel,
652         "T_excel": T_excel,
653         "table_hours": hrs,
654         "C_table": C_table,
655         "paper_C": paper_map,
656     }
657
658
659 def write_result3_xlsx(path: Path, C_out: np.ndarray, times: np.ndarray, *, validation):
660     """All numerical checks must pass before calling the workbook author."""
661     from q3_closeout import export_workbook, require_q3_delivery
662     require_q3_delivery(validation, RESULTS_DIR / "diagnostics/q3_closeout/pre_export.json")
663     export_workbook(path, C_out, times, validation)
664
665
666 def write_table5_csv(path: Path, hours: np.ndarray, C_table: np.ndarray):
667     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
668     for th, row in zip(hours, C_table):
669         bits = [f"{float(th):.8f}"] + [f"{float(v):.10f}" for v in row]
670         lines.append(",".join(bits))
671     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
672
673
674 def physical_bounds(case, Ta_fun, Ca_fun) -> dict:
675     n = case["n_nodes"]
676     T = case["Y"][:, :n]
677     C = case["Y"][:, n:]
678     Ta_max = max(float(np.max(Ta_fun.y)), TA_AFTER)
679     info = {
680         "T_min": float(T.min()),
681         "T_max": float(T.max()),
682         "C_min": float(C.min()),

```

```

683     "C_max": float(C.max()),
684     "T_below_28": float(max(0.0, 28.0 - T.min())),
685     "T_above-Ta_max": float(max(0.0, T.max() - Ta_max)),
686     "C_above_C0": float(max(0.0, C.max() - C0)),
687     "C_negative": float(max(0.0, -C.min())),
688     "Ta_max": Ta_max,
689     "Ca_range_knots": [float(np.min(Ca_fun.y)), float(np.max(Ca_fun.y))],
690 }
691 info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 0.05
692 info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_negative"] <= 1e-12
693 return info
694
695
696 def plot_figures(plt, pub, tables, t_rep: float, Ta_fun):
697     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
698     fvm = pub["fvm"]
699     n = fvm.n_nodes
700     mon = pub["monitor"]
701     t_h = mon["times"] / 3600.0
702     t_star_h = pub["t_star"] / 3600.0
703     t_rep_h = t_rep / 3600.0
704
705     fig, ax = plt.subplots(figsize=(6.4, 3.8))
706     ax.axhline(C_THRESH, color="0.4", lw=1.0, ls="--", label=r" 阈值 $0.15$")
707     ax.plot(t_h, mon["M"], lw=1.2, label=r" 全网格 $\max C$")
708     ax.plot(t_h, mon["C_center"], lw=1.2, ls="-. ", label=" 中心")
709     ax.plot(t_h, mon["C_surface"], lw=1.2, ls=":", label=" 表面")
710     ax.axvline(t_star_h, color="0.2", lw=0.8, ls=":")
711     ax.set_xlabel(" 时间 $t$ (h)")
712     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
713     ax.set_xlim(0, t_rep_h)
714     ax.legend(frameon=False)
715     fig.tight_layout()
716     fig.savefig(FIGURES_DIR / "q3_C_max_history.pdf")
717     plt.close(fig)
718
719     fig, ax = plt.subplots(figsize=(6.4, 3.6))
720     ax.plot(t_h, mon["rmax_m"] * 100.0, lw=1.2)
721     ax.set_xlabel(" 时间 $t$ (h)")
722     ax.set_ylabel(r" 最大含水率位置 $r$ (cm)")
723     ax.set_xlim(0, t_rep_h)
724     ax.set_ylim(-0.02, 2.05)
725     fig.tight_layout()
726     fig.savefig(FIGURES_DIR / "q3_max_location.pdf")
727     plt.close(fig)
728
729     fig, ax = plt.subplots(figsize=(6.4, 3.8))
730     r_cm = fvm.r * 100.0
731     hrs = tables["table_hours"]
732     t_end_h = float(hrs[-1])
733     if t_end_h > 72.0:
734         show = np.arange(24.0, t_end_h + 1e-9, 24.0)
735         if abs(show[-1] - t_end_h) > 1e-6:
736             show = np.concatenate([show, [t_end_h]])
737     elif hrs.size > 10:
738         show = np.unique(np.concatenate([hrs[:,2], hrs[-1:]]))
739     else:
740         show = hrs
741     for th in show:
742         t = 3600.0 * float(th)
743         if t < pub["t_star"] - 1e-9:
744             j = int(np.argmin(np.abs(pub["times"] - t)))
745             C = pub["Y"][j, n:]
746         else:
747             C = pub["eval_y"](t)[n:]

```

```

748     ax.plot(r_cm, C, lw=1.2, label=fr"$t={th:g}\,\mathrm{{{h}}}$")
749     ax.set_xlabel(" 半径 $r$ (cm)")
750     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
751     ax.set_xlim(0, 2)
752     ax.legend(frameon=False, ncol=2)
753     fig.tight_layout()
754     fig.savefig(FIGURES_DIR / "q3_C_profiles.pdf")
755     plt.close(fig)
756
757     fig, ax = plt.subplots(figsize=(6.4, 3.6))
758     mask = mon["times"] >= pub["t_star"] - 6 * 3600.0
759     if np.count_nonzero(mask) < 5:
760         mask = np.ones_like(mon["times"], dtype=bool)
761     ax.axhline(C_THRESH, color="0.4", lw=1.0, ls="--", label=r" 阈值 $0.15$")
762     ax.plot(t_h[mask], mon["M"][mask], lw=1.2, label=r"$\max C$")
763     ax.axvline(t_star_h, color="0.2", lw=0.8, ls=":", label=fr"$t_*={t_star_h:.3f}\,\mathrm{{{h}}}$")
764     ax.axvline(t_rep_h, color="0.2", lw=0.8, ls="-.",
765         ↪ label=fr"$t_{\mathrm{{{rep}}}}={t_rep_h:.3f}\,\mathrm{{{h}}}$")
766     ax.set_xlabel(" 时间 $t$ (h)")
767     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
768     ax.legend(frameon=False)
769     fig.tight_layout()
770     fig.savefig(FIGURES_DIR / "q3_event_zoom.pdf")
771     plt.close(fig)
772
773     fig, ax = plt.subplots(figsize=(6.4, 3.6))
774     ax.plot(t_h, [Ta_fun(t) for t in mon["times"]], lw=1.0, ls="--", label=" 烘房 $T_a$")
775     ax.plot(t_h, mon["T_surface"], lw=1.2, label=" 表面")
776     ax.plot(t_h, mon["T_center"], lw=1.2, label=" 中心")
777     ax.set_xlabel(" 时间 $t$ (h)")
778     ax.set_ylabel(" 温度 (°C)")
779     ax.set_xlim(0, t_rep_h)
780     ax.legend(frameon=False)
781     fig.tight_layout()
782     fig.savefig(FIGURES_DIR / "q3_T_center_surface.pdf")
783     plt.close(fig)
784
785     def json_safe(obj):
786         if isinstance(obj, dict):
787             return {str(k): json_safe(v) for k, v in obj.items()}
788         if isinstance(obj, (list, tuple)):
789             return [json_safe(v) for v in obj]
790         if isinstance(obj, np.ndarray):
791             return obj.tolist()
792         if isinstance(obj, (np.floating, np.integer)):
793             return obj.item()
794         if isinstance(obj, (float, int, str, bool)) or obj is None:
795             return obj
796         return str(obj)
797
798
799     def main(argv=None):
800         from q3_closeout import main as closeout_main
801         return closeout_main(argv)
802
803
804     if __name__ == '__main__':
805         main()

```

B.10 code/q3_balance.py

```

1 """Continuous-solution balance integrals for the accepted Q3 effective model."""
2 import numpy as np
3 from numpy.polynomial.legendre import leggauss

```

```

4 from appendix3 import heat_capacity, heat_capacity_deriv, moisture_diffusivity_q2
5 from utils import R, L, H, HM
6
7
8 def audit_segment(fvm, sol, Ta_fun, Ca_fun, t_end=None, orders=(4,8)):
9     if sol.sol is None:
10         raise ValueError("balance audit requires a dense solution")
11     n = fvm.n_nodes
12     start = float(sol.sol.ts[0])
13     end = float(sol.sol.ts[-1]) if t_end is None else float(t_end)
14     if not start < end <= sol.sol.ts[-1]+1e-9:
15         raise ValueError("audit outside dense-solution interval")
16     edges = np.r_[sol.sol.ts[sol.sol.ts < end],end]
17     edges = np.unique(edges)
18     z0, z1 = sol.sol(start), sol.sol(end)
19     factor = 2*np.pi*L
20     def energy(z):
21         return float(factor*np.dot(fvm.W,heat_capacity(z[n:]))*(z[:n]-28.)))
22     def quadrature(order):
23         gx,gw=leggauss(order)
24         total=np.zeros(3)
25         for j in range(0,len(edges)-1,64):
26             aa,bb=edges[j:-1][:64],edges[j+1:][:64]
27             tt=((aa+bb)[:None]/2+(bb-aa)[:None]/2*gx).ravel()
28             Y=sol.sol(tt);T,C=Y[:n],Y[n:]
29             Ta,Ca=np.asarray(Ta_fun(tt)),np.asarray(Ca_fun(tt))
30             D=moisture_diffusivity_q2(C,T)
31             Df=2*D[: -1]*D[1:]/np.maximum(D[: -1]+D[1:],1e-300)
32             F=-fvm.r_half[:None]*Df*np.diff(C,axis=0)/fvm.dr_face[:None]
33             flows=np.vstack([np.zeros((1,len(tt))),F,R*HM*(C[-1]-Ca)])
34             Cdot=-np.diff(flows,axis=0)/fvm.W[:None]
35             vals=np.vstack([R*HM*(C[-1]-Ca),factor*R*H*(Ta-T[-1]),
36                             factor*np.sum(fvm.W[:None]*heat_capacity_deriv(C)*(T-28.)*Cdot,axis=0)])
37             weights=((bb-aa)[:None]/2*gw).ravel()
38             total+=vals@weights
39         return total
40     integrals={str(o):quadrature(o).tolist() for o in orders}
41     return {"start_s":start,"end_s":end,"accepted_intervals":len(edges)-1,
42           "M_start":float(fvm.W@z0[n:]),"M_end":float(fvm.W@z1[n:]),
43           "E_start":energy(z0),"E_end":energy(z1),"integrals_by_order":integrals}
44
45
46 def combine(parts):
47     if not parts:
48         raise ValueError('no balance segments')
49     for left, right in zip(parts, parts[1:]):
50         if abs(left['end_s']-right['start_s']) > 1e-9:
51             raise ValueError('balance segments have a time gap or overlap')
52         for field in ('M', 'E'):
53             if not np.isclose(left[field+'_end'], right[field+'_start'], rtol=1e-12, atol=1e-14):
54                 raise ValueError('balance segments have a state discontinuity')
55     I=np.sum([p['integrals_by_order']['8'] for p in parts],axis=0)
56     I4=np.sum([p['integrals_by_order']['4'] for p in parts],axis=0)
57     M0,M1=parts[0]['M_start'],parts[-1]['M_end']
58     E0,E1=parts[0]['E_start'],parts[-1]['E_end']
59     mass_denom=max(abs(M0-M1),abs(I[0]),1e-18)
60     heat_denom=max(abs(E1-E0-I[2]),abs(I[1]),1e-12)
61     return {
62         'moisture':{'M0':M0,'M_end':M1,'surface_integral':float(I[0]),
63                     'relative_residual':float(abs(M1-M0+I[0])/mass_denom)},
64         'heat':{'E_end_minus_E0':E1-E0,'composition_correction':float(I[2]),'Q_surface':float(I[1]),
65                 'relative_residual':float(abs(E1-E0-I[2]-I[1])/heat_denom)},
66         'quadrature_relative_difference':{'moisture':float(abs(I[0]-I4[0])/mass_denom),
67                                           'heat_and_composition':float((abs(I[1]-I4[1])+abs(I[2]-I4[2]))/heat_denom)},
68         'parts':parts,

```

```

69         'interpretation':'Integrated effective equation with B_prime(C)*(T-28)*Cdot correction; not full
        ↪ multiphase energy conservation.'}

```

B.11 code/q3_closeout.py

```

1  """Q3 closeout: matched-time convergence, continuous balances and gated delivery."""
2  from __future__ import annotations
3  import argparse
4  import hashlib
5  import json
6  import math
7  import subprocess
8  import sys
9  import time
10 from pathlib import Path
11 import numpy as np
12 from delivery import make_gate,require_delivery
13 from q3_balance import audit_segment,combine
14 from utils import RESULTS_DIR,REPORTS_DIR,FIGURES_DIR,ROOT
15
16 DIAG=RESULTS_DIR/'diagnostics/q3_closeout'
17 NODE=Path(__import__('shutil').which('node') or 'node')
18 BUILDER=ROOT/'tmp/q3_export/build.mjs'
19 REQUIRED_CHECKS = frozenset({
20     'event_found', 'finite', 'strict_below_all_verified_grids',
21     'strict_below_with_empirical_margin', 'moisture_bounds', 'temperature_bounds',
22     'moisture_balance_1e-6', 'heat_balance_1e-6', 'quadrature_convergence_1e-7',
23     'space_event_60s', 'time_event_60s', 'space_full_fields_2e-5',
24     'time_full_fields_2e-5', 'table_four_decimals', 'q2_overlap',
25 })
26
27
28 def require_q3_delivery(validation, diagnostic_path):
29     require_delivery(validation, diagnostic_path)
30     if validation.get('result_version') != 'q3-closeout-v1' or not
31     ↪ REQUIRED_CHECKS.issubset(validation['delivery_gate']['checks']):
32         raise RuntimeError('Delivery blocked: missing Q3 checks or wrong result version')
33
34 def export_workbook(path,C_out,times,validation):
35     require_q3_delivery(validation,DIAG/'pre_export.json')
36     data={'sheet':' 水分浓度','headers':[' 时间\\到药材中心的距离']+[round(.1*j,1) for j in range(21)],
37         'rows':[[float(t)]+list(map(float,row)) for t,row in
38     ↪ zip(times,C_out)],'path':str(path.resolve())}
39     (DIAG/'excel_payload.json').write_text(json.dumps(data,ensure_ascii=False,allow_nan=False),encoding='u'
40     ↪ tf-8')
41     subprocess.run([str(NODE),str(BUILDER)],cwd=ROOT,check=True)
42
43 def save_case(case):
44     f=case['fvm'];n=f.n_nodes
45     np.savez_compressed(DIAG/(case['tag']+'.npz'),r=f.r,times=case['times'],Y=case['Y'],
46         y_star=case['y_star'],t_star=case['t_star'])
47     info={k:case[k] for k in ('tag','level','n_nodes','rtol','atol_T','atol_C','max_step_early','max_step_
48     ↪ late','elapsed_s','event')}
49     (DIAG/(case['tag']+'.json')).write_text(json.dumps(info,indent=2),encoding='utf-8')
50
51 def compare_fields(a,b):
52     """Shared physical nodes and exactly matching stored times."""
53     times,ia,ib=np.intersect1d(a['times'],b['times'],return_indices=True)
54     ra,rb=a['fvm'].r,b['fvm'].r
55     j=np.searchsorted(rb,ra)
56     if np.max(abs(rb[j]-ra))>1e-13:
57         raise RuntimeError('spatial meshes are not nested')

```

```

57     na,nb=a['n_nodes'],b['n_nodes']
58     # Batch to bound temporary array sizes for fine meshes.
59     maxT=maxC=0.
60     for start in range(0,len(times),128):
61         ai,bi=ia[start:start+128],ib[start:start+128]
62         maxT=max(maxT,float(np.max(abs(a['Y'][ai,:na]-b['Y'][bi,nb][:,j]))))
63         maxC=max(maxC,float(np.max(abs(a['Y'][ai,na:]-b['Y'][bi,nb][:,j]))))
64     return {'matched_times':len(times), 'max_T':maxT, 'max_C':maxC}
65
66
67 def main(argv=None):
68     from problem3 import (load_q3_inputs,run_case,run_probe,RTOL,ATOL_T,ATOL_C,MAX_STEP_EARLY,
69         MAX_STEP_LATE,find_strict_report,build_output_tables,physical_bounds,overlap_q2,
70         write_result3_xlsx,write_table5_csv,plot_figures,setup_mpl,conservation_heat,conservation_moisture)
71     parser=argparse.ArgumentParser();parser.add_argument('--probe-only',action='store_true')
72     parser.add_argument('--no-xlsx',action='store_true',help='Save gated numerical evidence; export
73     ↪ separately later.')
74     args=parser.parse_args(argv)
75     DIAG.mkdir(parents=True,exist_ok=True)
76     start=time.perf_counter()
77     Ta,Ca,knots=load_q3_inputs()
78     if args.probe_only:
79         print(json.dumps(run_probe(Ta,Ca)));return
80     cases={}
81     for level in (4,8,16):
82         cases[level]=run_case(level,Ta,Ca,72*3600.,RTOL,ATOL_T,ATOL_C,MAX_STEP_EARLY,MAX_STEP_LATE,
83             tag=f'G{level}',audit_balances=(level==8))
84         save_case(cases[level])
85     pub,ref=cases[8],cases[16]
86     tight=run_case(8,Ta,Ca,72*3600.,RTOL/5,ATOL_T/5,ATOL_C/5,MAX_STEP_EARLY/2,MAX_STEP_LATE/2,tag='G8t')
87     save_case(tight)
88     space=compare_fields(pub,ref);temporal=compare_fields(pub,tight)
89     coarse_space=compare_fields(cases[4],pub)
90     critical=pub['t_star']
91     same={name:float(case['eval_M'](critical)) for name,case in
92     ↪ [(('G4',cases[4]),('G8',pub),('G16',ref),('G8t',tight))]}
93     # The maximum differences are measured at the same physical time.
94     err=max(abs(same['G8']-same['G16']),abs(same['G8']-same['G8t']))
95     margin=2*err+1e-8
96     zero_report=find_strict_report(pub['eval_M'],critical,critical+3500,margin=0.)
97     report=find_strict_report(pub['eval_M'],critical,critical+3500,margin=margin)
98     t_rep=report['t_hour4_s'] # Upward four-decimal-hour rounding retains the safety margin.
99     report_M={name:float(case['eval_M'](t_rep)) for name,case in [(('G8',pub),('G16',ref),('G8t',tight))]}
100     tables=build_output_tables(pub,t_rep);other=build_output_tables(ref,t_rep)
101     tight_tables=build_output_tables(tight,t_rep)
102     table_changed=int(np.count_nonzero(np.round(tables['C_table'],4)!=np.round(other['C_table'],4)))
103     table_time_changed=int(np.count_nonzero(np.round(tables['C_table'],4)!=np.round(tight_tables['C_table']
104     ↪ ),4)))
105     output_space=float(np.max(abs(tables['C_excel']-other['C_excel'])))
106     tail=audit_segment(pub['fvm'],pub['sol_after'],Ta,Ca,t_end=t_rep)
107     balances=combine(pub['balance_parts']+tail)
108     checked_states = np.vstack([pub['Y'], pub['y_star'], pub['eval_y'](t_rep)])
109     bounds=physical_bounds(dict(pub,Y=checked_states),Ta,Ca);overlap=overlap_q2(pub)
110     overlap_ok=overlap.get('available',False) and all(c.get('matched',False) and c['T_max_abs']<2e-5 and
111     ↪ c['C_max_abs']<2e-5 for c in overlap['checks'].values())
112     event_space=abs(pub['t_star']-ref['t_star']);event_time=abs(pub['t_star']-tight['t_star'])
113     gates={
114         'event_found':abs(pub['event']['g_star'])<1e-8,
115         'finite':bool(np.isfinite(checked_states).all() and np.isfinite(tables['C_excel']).all()
116             and np.isfinite(tables['T_excel']).all()),
117         'strict_below_all_verified_grids':all(v<.15 for v in report_M.values()),
118         'strict_below_with_empirical_margin':report_M['G8']+margin<.15,
119         'moisture_bounds':bounds['C_bounds_ok'],'temperature_bounds':bounds['T_bounds_ok'],
120         'moisture_balance_1e-6':balances['moisture']['relative_residual']<1e-6,
121         'heat_balance_1e-6':balances['heat']['relative_residual']<1e-6,

```

```

118     'quadrature_convergence_1e-7':max(balances['quadrature_relative_difference'].values())<1e-7,
119     'space_event_60s':event_space<=60., 'time_event_60s':event_time<=60.,
120     'space_full_fields_2e-5':max(space['max_T'],space['max_C'],output_space)<2e-5,
121     'time_full_fields_2e-5':max(temporal['max_T'],temporal['max_C'])<2e-5,
122     'table_four_decimals':table_changed==0 and table_time_changed==0,
123     'q2_overlap':overlap_ok,
124 }
125 v={'result_version':'q3-closeout-v1','model':'q3-model-v1','result_status':'PENDING',
126   'published_mesh':'G8','published_n_nodes':pub['n_nodes'],'rtol':RTOL,'atol_T':ATOL_T,'atol_C':ATOL_
    ↪ C,
127   'max_step_early':MAX_STEP_EARLY,'max_step_late':MAX_STEP_LATE,
128   't_star_s':critical,'t_star_h':critical/3600., 't_rep_s':t_rep,'t_rep_h':t_rep/3600.,
129   'M_star':pub['event']['M_star'],'M_rep':report_M['G8'],'report_maxima':report_M,
130   'matched_critical_time_maxima':same,'moisture_margin':margin,'margin_type':'empirical, not a
    ↪ rigorous PDE error bound',
131   'report_zero_margin':zero_report,'report_with_margin':report,
132   'space_fields':space,'time_fields':temporal,'coarse_space_fields':coarse_space,'output_space_max_C'
    ↪ :output_space,
133   'dt_space_s':event_space,'dt_time_s':event_time,
134   'conservation_C':balances['moisture'],'conservation_T':balances['heat'],'continuous_balances':balan
    ↪ ces,
135   'old_sampled_conservation_C':conservation_moisture(pub['fvm'],pub['times'],pub['Y'],Ca),
136   'old_sampled_conservation_T':conservation_heat(pub['fvm'],pub['times'],pub['Y'],Ta),
137   'physical_bounds':bounds,'q2_overlap':overlap,'table_four_decimal_changes':table_changed,
138   'events':{f'G{k}':c['event'] for k,c in cases.items()}|{'G8t':tight['event']}},
139   'delivery_gate':make_gate(gates),'table_hours':tables['table_hours'].tolist(),
140   'excel_nrows':len(tables['excel_times']), 'paper_table_C':tables['paper_C'],
141   'max_location_cm':pub['event']['rmax_star_cm'],'center_is_max_at_event':pub['event']['center_is_max
    ↪ '],
142   'n_radial_increases':pub['monitor']['n_radial_increases'],
143   'elapsed_s':time.perf_counter()-start,'python':sys.version,
144   'source_sha256':{p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest() for p in
145     ['code/problem3.py','code/q3_balance.py','code/q3_closeout.py','code/delivery.py',
146     'tmp/q3_export/build.mjs','tmp/q3_export/gate.mjs',
147     'code/radial_coupled.py','code/appendix3.py','data/附件/附件 1.xlsx']}}
148 print('CLOSEOUT',json.dumps({k:v[k] for k in ('t_star_h','t_rep_h','M_rep','dt_space_s','dt_time_s','c
    ↪ nservation_C','conservation_T','delivery_gate')},ensure_ascii=False),flush=True)
149 publish_results(pub, tables, v, Ta, no_xlsx=args.no_xlsx)
150 print('Q3 CLOSEOUT NUMERICAL PASS',flush=True)
151
152
153 def publish_results(pub, tables, v, Ta, *, no_xlsx=False):
154     from problem3 import write_result3_xlsx, write_table5_csv, plot_figures, setup_mpl
155     require_q3_delivery(v,DIAG/'latest.json')
156     t_rep, critical = v['t_rep_s'], v['t_star_s']
157     # Commit production outputs only after every required numerical check passed.
158     yend=pub['eval_y'](t_rep)
159     np.savez_compressed(RESULTS_DIR/'q3_solution.npz',mesh_level=8,n_nodes=pub['n_nodes'],r=pub['fvm'].r,
160       times=pub['times'],Y=pub['Y'],excel_times=tables['excel_times'],T_out=tables['T_excel'],C_out=tabl
    ↪ es['C_excel'],
161     table_hours=tables['table_hours'],C_table=tables['C_table'],t_star=critical,t_rep=t_rep,
162     y_star=pub['y_star'],y_end=yend,M_hist=pub['monitor']['M'],rmax_m=pub['monitor']['rmax_m'],
163     C_center=pub['monitor']['C_center'],C_surface=pub['monitor']['C_surface'],
164     T_center=pub['monitor']['T_center'],T_surface=pub['monitor']['T_surface'],
165     rtol=v['rtol'],atol_T=v['atol_T'],atol_C=v['atol_C'],max_step_early=v['max_step_early'],max_step_l
    ↪ ate=v['max_step_late'],
166     interpolation=np.array('pchip_then_constant'))
167     (RESULTS_DIR/'q3_validation.json').write_text(json.dumps(v,ensure_ascii=False,indent=2),encoding='utf-
    ↪ 8')
168     write_table5_csv(RESULTS_DIR/'q3_table_moisture.csv',tables['table_hours'],tables['C_table'])
169     if not no_xlsx:
170         write_result3_xlsx(RESULTS_DIR/'result3.xlsx',tables['C_excel'],tables['excel_times'],validation=v)
171     plot_figures(setup_mpl(),pub,tables,t_rep,Ta)
172

```

```

173
174 if __name__ == '__main__':
175     main()

```

B.12 code/problem4.py

```

1  """Q4: prescribed radial shrinkage, material-coordinate FVM and BDF.
2
3  Run a bounded, source-preserving baseline with --level 2/4/8 [--tight].
4  No radius extrapolation; output here is internal numerical evidence, not Excel.
5  """
6  from __future__ import annotations
7
8  import argparse
9  import hashlib
10 import json
11 import platform
12 import time
13 from pathlib import Path
14
15 import numpy as np
16 import scipy
17 from numpy.polynomial.legendre import leggauss
18 from openpyxl import load_workbook
19 from scipy.integrate import solve_ivp
20 from scipy.interpolate import PchipInterpolator
21 from scipy.sparse import lil_matrix
22
23 from radial_fvm import graded_radial_nodes, harmonic_mean
24 from utils import ATTACH_DIR, RESULTS_DIR, C0, T0, H, HM, R, load_oven_table
25
26 CAP = 259200.0
27 SWITCH = 14400.0
28 THRESHOLD = 0.15
29 OUT = RESULTS_DIR / "q4_baseline"
30
31
32 def properties(C, T):
33     C, T = np.asarray(C), np.asarray(T)
34     # Only protect nonphysical implicit-iteration probes; stored states are audited.
35     Cp = np.maximum(C, 1e-12)
36     rho = 760.0 + 90.0 * Cp
37     cp = 1850.0 + 2150.0 * Cp / (1.0 + Cp)
38     k = 0.12 + 0.20 * Cp / (1.0 + Cp)
39     D = 4.2e-4 * np.exp(-0.30 / Cp) * np.exp(-3850.0 / (T + 273.15))
40     return rho * cp, k, D
41
42
43 class Inputs:
44     def __init__(self):
45         t, Ta, Ca = load_oven_table()
46         if t.size != 241 or t[0] != 0 or t[-1] != SWITCH:
47             raise ValueError("unexpected oven input coverage")
48         self.Ta = PchipInterpolator(t, Ta, extrapolate=False)
49         self.Ca = PchipInterpolator(t, Ca, extrapolate=False)
50         wb = load_workbook(ATTACH_DIR / "附件 2.xlsx", read_only=True, data_only=True)
51         radius_rows = list(wb.active.values)
52         wb.close()
53         a = np.asarray(radius_rows[1:], dtype=float)
54         self.rt, self.rv = a[:, 0], a[:, 1] / 100.0
55         if (a.shape != (145, 2) or not np.all(np.isfinite(a))
56             or np.any(np.diff(self.rt) <= 0) or np.any(np.diff(self.rv) > 0)
57             or self.rt[0] != 0 or self.rt[-1] != CAP or self.rv[0] != R):
58             raise ValueError("unexpected radius input")
59         self.radius = PchipInterpolator(self.rt, self.rv, extrapolate=False)

```

```

60     self.audit = {"radius_records": len(a), "radius_range_m": [float(self.rv[0]), float(self.rv[-1])],
61                  "radius_cap_s": CAP, "oven_records": len(t), "radius_input_headers":
        ↳ list(radius_rows[0]),
62                  "radius_node_error_m": float(np.max(abs(self.radius(self.rt) - self.rv))),
63                  "oven_node_error": float(max(np.max(abs(self.Ta(t)-Ta)),
        ↳ np.max(abs(self.Ca(t)-Ca))))}
64     self.audit["source_sha256"] = {
65         name: hashlib.sha256((ATTACH_DIR / name).read_bytes()).hexdigest()
66         for name in (" 附件 1.xlsx", " 附件 2.xlsx")}
67
68     def environment(self, t, late=False):
69         if late or t > SWITCH:
70             return 50.0, 0.05
71         return float(self.Ta(t)), float(self.Ca(t))
72
73
74 class ShrinkingFVM:
75     def __init__(self, xi):
76         self.xi = np.asarray(xi, dtype=float)
77         if self.xi[0] != 0 or self.xi[-1] != 1 or np.any(np.diff(self.xi) <= 0):
78             raise ValueError("invalid material mesh")
79         self.n = len(xi)
80         self.faces = (self.xi[:-1] + self.xi[1:]) / 2
81         edges = np.r_[0., self.faces, 1.]
82         self.w = np.diff(edges**2) / 2
83         self.dx = np.diff(self.xi)
84
85     def sparsity(self):
86         J = lil_matrix((2*self.n, 2*self.n))
87         for i in range(self.n):
88             for j in range(max(0, i-1), min(self.n, i+2)):
89                 J[i, j] = J[i, self.n+j] = J[self.n+i, j] = J[self.n+i, self.n+j] = 1
90         return J.tocsc()
91
92     def rhs(self, y, radius, Ta, Ca, hm=HM, ht=H):
93         T, C = y[:self.n], y[self.n:]
94         B, k, D = properties(C, T)
95         FT = np.r_[0., -self.faces*harmonic_mean(k[:-1], k[1:])*np.diff(T)/self.dx,
96                  radius*ht*(T[-1]-Ta)]
97         FC = np.r_[0., -self.faces*harmonic_mean(D[:-1], D[1:])*np.diff(C)/self.dx,
98                  radius*hm*(C[-1]-Ca)]
99         Tdot = -np.diff(FT) / (radius**2 * self.w * B)
100         Cdot = -np.diff(FC) / (radius**2 * self.w)
101         return np.r_[Tdot, Cdot]
102
103
104 def model_checks(inputs):
105     fvm = ShrinkingFVM(np.linspace(0., 1., 41))
106     x = fvm.xi
107     y = np.r_[35.+10*x*x, 1.7-1.2*x*x]
108     rr = float(inputs.radius(21600.))
109     dy = fvm.rhs(y, rr, 50., 0.05)
110     mass_error = abs(2*np.dot(fvm.w, dy[fvm.n:])) + 2*HM/rr*(y[-1]-0.05)
111     B, k, D = properties(y[fvm.n:], y[fvm.n:])
112     heat_error = abs(np.dot(fvm.w*B, dy[:fvm.n])) + H/rr*(y[fvm.n-1]-50.)
113     # Independent dimensional-volume assembly on the same physical annuli.
114     rp = rr*x
115     rf = (rp[:-1]+rp[1:])/2
116     volumes = np.diff(np.r_[0., rf, rr]**2)/2
117     flows_T = np.r_[0., -rf*harmonic_mean(k[:-1], k[1:])*np.diff(y[:fvm.n])/np.diff(rp),
        ↳ rr*H*(y[fvm.n-1]-50.)]
118     flows_C = np.r_[0., -rf*harmonic_mean(D[:-1], D[1:])*np.diff(y[fvm.n:])/np.diff(rp),
        ↳ rr*HM*(y[-1]-0.05)]
119     reference = np.r_[ -np.diff(flows_T)/(volumes*B), -np.diff(flows_C)/volumes]
120     fixed_error = float(np.max(abs(dy-reference)))

```

```

121 uniform = np.r_[np.full(fvm.n, T0), np.full(fvm.n, C0)]
122 pure = solve_ivp(lambda t,z: fvm.rhs(z, float(inputs.radius(t)), T0, C0, hm=0., ht=0.),
123                 (0., CAP), uniform, method="BDF", jac_sparsity=fvm.sparsity(),
124                 rtol=1e-9, atol=1e-11)
125 pure_error = float(np.max(abs(pure.y-uniform[:,None])))
126 checks = {"instant_mass_abs": float(mass_error), "instant_heat_abs": float(heat_error),
127          "fixed_physical_operator_abs": fixed_error, "pure_shrinkage_uniform_abs": pure_error,
128          "pure_shrinkage_success": bool(pure.success)}
129 checks["pass"] = bool(mass_error < 1e-12 and heat_error < 1e-6 and fixed_error < 1e-10
130                    and pure_error < 1e-10 and pure.success)
131 return checks
132
133
134 def run(level=2, tight=False):
135     OUT.mkdir(parents=True, exist_ok=True)
136     inputs = Inputs()
137     basic = model_checks(inputs)
138     if not basic["pass"]:
139         raise RuntimeError(f"model checks failed: {basic}")
140     fvm = ShrinkingFVM(graded_radial_nodes(level)/R)
141     n = fvm.n
142     rtol = 1e-9 if tight else 1e-8
143     atol = np.r_[np.full(n, 1e-9 if tight else 1e-8), np.full(n, 1e-11 if tight else 1e-10)]
144     label = f"G{level}" + ("_tight" if tight else "")
145     y0 = np.r_[np.full(n, T0), np.full(n, C0)]
146     times, states = [np.array([0.])], [y0[None,:]]
147     segments, events, event_states = [], [], []
148     mass_loss = 0.
149     max_mass_error = 0.
150     bounds = [float("inf"), -float("inf"), float("inf"), -float("inf")]
151     radial_increase = 0.
152     heat_residual_max = 0.
153     wall = time.perf_counter()
154     event = lambda t,y: float(np.max(y[n:])-THRESHOLD)
155     event.direction = -1.
156     event.terminal = False
157     breaks = np.unique(np.r_[0., SWITCH, np.arange(21600., CAP+1., 21600.)])
158     gx, gw = leggauss(4)
159     for a,b in zip(breaks[:-1], breaks[1:]):
160         late = a >= SWITCH
161         def rhs(t,y):
162             Ta,Ca = inputs.environment(t, late=late)
163             return fvm.rhs(y, float(inputs.radius(t)), Ta, Ca)
164         max_step = (5. if tight else 10.) if not late else (30. if tight else 60.)
165         sol = solve_ivp(rhs, (a,b), y0, method="BDF", rtol=rtol, atol=atol,
166                       jac_sparsity=fvm.sparsity(), max_step=max_step,
167                       dense_output=True, events=event)
168         if not sol.success:
169             raise RuntimeError(sol.message)
170         ts = np.arange(a+60., b+0.1, 60.)
171         ys = sol.sol(ts).T
172         times.append(ts)
173         states.append(ys)
174         if len(sol.t_events[0]):
175             events.extend(sol.t_events[0].tolist())
176             event_states.extend(sol.y_events[0].tolist())
177         # Quadrature on accepted BDF step intervals; 4-point Gaussian rule.
178         integral = 0.
179         for start in range(0, len(sol.t)-1, 128):
180             aa, bb = sol.t[start:-1][:128], sol.t[start+1:][:128]
181             qq = ((aa+bb)[:None]/2 + (bb-aa)[:None]/2*gx).ravel()
182             surface = sol.sol(qq)[-1]
183             ca = np.full_like(qq, 0.05) if late else inputs.Ca(qq)
184             loss = 2*HM/inputs.radius(qq)*(surface-ca)
185             integral += float(np.sum(loss.reshape(-1,4)*gw*(bb-aa)[:None]/2))

```

```

186     mass_loss += integral
187     y0 = sol.y[:,-1]
188     mass_error = abs(float(2*np.dot(fvm.w,y0[n:])-C0+mass_loss))
189     max_mass_error = max(max_mass_error, mass_error)
190     TT, CC = sol.y[:n], sol.y[n:]
191     bounds = [min(bounds[0],float(TT.min())), max(bounds[1],float(TT.max())),
192               min(bounds[2],float(CC.min())), max(bounds[3],float(CC.max()))]
193     radial_increase = max(radial_increase,float(np.max(np.diff(CC,axis=0))))
194     # Audit sampled heat-equation weighted residual, not d(rho*cp*T)/dt.
195     for tcheck in (a,(a+b)/2,b):
196         z = sol.sol(tcheck)
197         dz = rhs(tcheck,z)
198         B,_,_ = properties(z[n:],z[:n])
199         Ta,_ = inputs.environment(tcheck,late=late)
200         heat_error = abs(float(np.dot(fvm.w*dz[:n])+H/float(inputs.radius(tcheck))*(z[n-1]-Ta)))
201         heat_residual_max = max(heat_residual_max,heat_error)
202         segments.append({"start_s":float(a),"end_s":float(b),"steps":len(sol.t)-1,
203                         "nfev":sol.nfev,"nlu":sol.nlu,"mass_abs_error":mass_error})
204     print(f"{label}: {b/3600:g}h maxC={y0[n:].max():.8f} mass_err={mass_error:.2e}
205     ↪ wall={time.perf_counter()-wall:.1f}s",flush=True)
206     tt, yy = np.concatenate(times), np.vstack(states)
207     meta = {"label":label,"level":level,"tight":tight,"nodes":n,"rtol":rtol,
208            "atol_T":float(atol[0]),"atol_C":float(atol[n]),"early_max_step_s":5. if tight else 10.,
209            "late_max_step_s":30. if tight else 60.,"event_s":events[0] if events else None,
210            "event_h":events[0]/3600 if events else None,"end_maxC":float(yy[-1,n:].max()),
211            "max_mass_abs_error":max_mass_error,"mass_relative_error":max_mass_error/C0,
212            "heat_equation_residual_max":heat_residual_max,"bounds_Tmin_Tmax_Cmin_Cmax":bounds,
213            "max_outward_C_increase":radial_increase,"segments":segments,"input_audit":inputs.audit,
214            "model_checks":basic,"wall_s":time.perf_counter()-wall,
215            "python":platform.python_version(),"numpy":np.__version__,"scipy":scipy.__version__,
216            "model":"q4-model-v1","radius_extrapolation":False,
217            "source_code_sha256":hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
218     np.savez_compressed(OUT/f"{label}.npz",times_s=tt,Y=yy,xi=fvm.xi,w=fvm.w,
219                       radius_m=inputs.radius(tt),events_s=np.array(events),event_states=np.array(event_s)
220                       ↪ tates))
221     (OUT/f"{label}.json").write_text(json.dumps(meta,ensure_ascii=False,indent=2),encoding="utf-8")
222     print(json.dumps({k:meta[k] for k in
223                       ↪ ("label","event_h","end_maxC","mass_relative_error","wall_s")}),flush=True)
224     return meta
225
226
227 if __name__ == "__main__":
228     parser = argparse.ArgumentParser()
229     parser.add_argument("--level",type=int,default=2)
230     parser.add_argument("--tight",action="store_true")
231     args = parser.parse_args()
232     run(args.level,args.tight)

```

B.13 code/verify_q4_results.py

```

1  """Validate Q4 at matched times; produce source tables and figures after gates."""
2  from __future__ import annotations
3  import csv
4  import json
5  import math
6  from pathlib import Path
7  import numpy as np
8  from scipy.integrate import solve_ivp
9  from scipy.interpolate import PchipInterpolator
10 from problem4 import OUT, Inputs, ShrinkingFVM, THRESHOLD, SWITCH
11 from problem2 import setup_mpl
12 from utils import RESULTS_DIR, FIGURES_DIR, REPORTS_DIR
13
14

```

```

15 def load(label):
16     return np.load(OUT/f"{label}.npz"), json.loads((OUT/f"{label}.json").read_text(encoding="utf-8"))
17
18
19 def state_at(data, meta, t, inputs):
20     """Match a new time via short BDF restart from a saved full state, not nearest-time comparison."""
21     times = data["times_s"]
22     i = int(np.searchsorted(times,t,side="right")-1)
23     if abs(times[i]-t) < 1e-8:
24         return data["Y"][i].copy()
25     f = ShrinkingFVM(data["xi"])
26     n = f.n
27     assert times[i] >= SWITCH # New event/report times are in the late constant environment.
28     sol = solve_ivp(lambda tt,y:f.rhs(y,float(inputs.radius(tt)),50.,0.05),
29                     (times[i],t),data["Y"][i],method="BDF",jac_sparsity=f.sparsity(),
30                     rtol=min(meta["rtol"],1e-10),atol=np.r_[np.full(n,1e-10),np.full(n,1e-12)],
31                     max_step=1.)
32     if not sol.success:
33         raise RuntimeError(sol.message)
34     return sol.y[:,-1]
35
36
37 def physical_values(xi, Y, times, inputs, spacing=.001):
38     n = len(xi)
39     fixed = np.arange(21)*.001 if spacing == .001 else np.arange(5)*.005
40     radii = inputs.radius(times)
41     out = np.full((len(times),len(fixed)+1),np.nan)
42     for i,(row,rr) in enumerate(zip(Y,radii)):
43         good = fixed <= rr+1e-14
44         out[i,:len(fixed)][good] =
45             ↳ PchipInterpolator(xi,row[n:],extrapolate=False)(np.minimum(fixed[good]/rr,1.))
46     out[i,-1] = row[-1]
47     return out, radii, fixed
48
49 def main():
50     inputs = Inputs()
51     labels = ["G2","G4","G8","G4_tight","G16","G8_tight"]
52     datasets = {k:load(k) for k in labels}
53     production, reference, tightened = "G8", "G16", "G8_tight"
54     a,ma = datasets[production]
55     b,mb = datasets[reference]
56     c,mc = datasets[tightened]
57     n,m = len(a["xi"]),len(b["xi"])
58     if not np.array_equal(a["times_s"],b["times_s"]) or not
59         ↳ np.allclose(a["xi"],b["xi"][:2],atol=1e-14,rtol=0):
60         raise RuntimeError("grids or sampling times are not nested")
61     spatial_T = float(np.max(abs(a["Y"][:,n]-b["Y"][:,m:2])))
62     spatial_C = float(np.max(abs(a["Y"][:,n]-b["Y"][:,m:2])))
63     temporal_T = float(np.max(abs(a["Y"][:,n]-c["Y"][:,n])))
64     temporal_C = float(np.max(abs(a["Y"][:,n]-c["Y"][:,n])))
65     # Production was refined after one table cell straddled a rounding boundary.
66     critical = ma["event_s"]
67     t_rep = math.ceil((max(ma["event_s"],mb["event_s"],mc["event_s"])+2.)/.36)*.36
68     checked = (production,reference,tightened)
69     at_critical = {k:state_at(d,md,critical,inputs) for k,(d,md) in datasets.items() if k in checked}
70     at_report = {k:state_at(d,md,t_rep,inputs) for k,(d,md) in datasets.items() if k in checked}
71     maxima = {k:float(z[len(datasets[k][0]["xi"]):].max()) for k,z in at_report.items()}
72     critical_max = {k:float(z[len(datasets[k][0]["xi"]):].max()) for k,z in at_critical.items()}
73     event_C_error = max(abs(critical_max[production]-critical_max[reference]),
74                         abs(critical_max[production]-critical_max[tightened]))
75     empirical_margin = 2*event_C_error + 1e-8
76     idx = a["times_s"] < t_rep
77     tt = np.r_[a["times_s"][idx],t_rep]
78     YY = np.vstack([a["Y"][idx],at_report[production]])

```

```

78 C_phys,radii,fixed = physical_values(a["xi"],YY,tt,inputs)
79 C8,_ = physical_values(b["xi"],np.vstack([b["Y"][idx],at_report[reference]]),tt,inputs)
80 physical_C_error = float(np.nanmax(abs(C_phys-C8)))
81 table_times = np.r_[np.arange(21600.,t_rep,21600.),t_rep]
82 table_ids = [int(np.argmin(abs(tt-t))) for t in table_times]
83 table_C,table_R,table_fixed = physical_values(a["xi"],YY[table_ids],table_times,inputs,spacing=.005)
84 # Dedicated table comparison includes the actual moving surface and ignores paired domain blanks.
85 table8,_ = physical_values(b["xi"],np.vstack([b["Y"][idx],at_report[reference]])[table_ids],table_ti
↪ mes,inputs,spacing=.005)
86 finite_table = np.isfinite(table_C)
87 table_changed =
↪ int(np.count_nonzero(np.round(table_C[finite_table],4)!=np.round(table8[finite_table],4)))
88 bounds_pass = all(md["bounds_Tmin_Tmax_Cmin_Cmax"][2]>=-1e-10 and
↪ md["bounds_Tmin_Tmax_Cmin_Cmax"][3]<2.550000001 for _,md in datasets.values())
89 gates = {
90     "input_and_model_tests":all(md["model_checks"]["pass"] for _,md in datasets.values()),
91     "spatial_fields_2e-5":max(spatial_T,spatial_C,physical_C_error)<2e-5,
92     "temporal_fields_2e-5":max(temporal_T,temporal_C)<2e-5,
93     "spatial_event_1s":abs(ma["event_s"]-mb["event_s"])<1.,
94     "temporal_event_1s":abs(ma["event_s"]-mc["event_s"])<1.,
95     "mass_relative_1e-6":all(md["mass_relative_error"]<1e-6 for _,md in datasets.values()),
96     "heat_equation_residual_1e-6":all(md["heat_equation_residual_max"]<1e-6 for _,md in
↪ datasets.values()),
97     "physical_bounds":bounds_pass,
98     "strict_report_all_checked_grids":all(v<THRESHOLD for v in maxima.values()),
99     "strict_report_empirical_margin":maxima[production]+empirical_margin<THRESHOLD,
100     "event_within_observed_radius":t_rep<=259200.,
101     "table_four_decimals":table_changed==0,
102 }
103 summary = {"version":"q4-baseline-v1","model":"q4-model-v1","production":production,
↪ "result_status":"PENDING",
104     "critical_s":critical,"critical_h":critical/3600.,"report_s":t_rep,"report_h":t_rep/3600.,
105     "report_radius_cm":float(inputs.radius(t_rep)*100),"report_maxima":maxima,"same_time_critic
↪ al_maxima":critical_max,
106     "empirical_C_margin":empirical_margin,"event_spatial_difference_s":abs(ma["event_s"]-mb["ev
↪ ent_s"])),
107     "event_time_difference_s":abs(ma["event_s"]-mc["event_s"])),
108     "max_spatial_T_C":[spatial_T,spatial_C],"max_temporal_T_C":[temporal_T,temporal_C],
109     "max_output_physical_C_difference":physical_C_error,"table_changed_four_decimal_cells":tabl
↪ e_changed,
110     "checks":gates,"pass":all(gates.values()),"runs":{k:md for k,(_,md) in datasets.items()},
111     "heat_validation_scope":"weighted equation residual, not full multiphase energy
↪ conservation",
112     "strict_margin_scope":"empirical grid/time differences, not a rigorous PDE error bound"}
113 (RESULTS_DIR/"q4_validation.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding=
↪ "utf-8")
114 if not summary["pass"]:
115     raise RuntimeError(f"Q4 output blocked: {gates}")
116 np.savez_compressed(RESULTS_DIR/"q4_solution.npz",times_s=tt,xi=a["xi"],Y=YY,
117     radius_m=radii,physical_radii_m=fixed,C_physical=C_phys,
118     table_times_s=table_times,table_C=table_C,critical_state=at_critical[production],r
↪ eport_state=at_report[production])
119 def safe(v):
120     return None if not np.isfinite(v) else float(v)
121 rows = [[float(t)]+[safe(v) for v in row] for t,row in zip(tt[1:],C_phys[1:])]
122 payload = {"sheet":"Sheet1","headers":[" 时间/s\\距离/cm"]+[float(r*100) for r in fixed]+[" 药材表面"],
123     "rows":rows,"report_h":t_rep/3600.}
124 (OUT/"excel_payload.json").write_text(json.dumps(payload,ensure_ascii=False,allow_nan=False),encoding=
↪ "utf-8")
125 with (RESULTS_DIR/"q4_table6.csv").open("w",newline="",encoding="utf-8-sig") as f:
126     writer=csv.writer(f);writer.writerow(["时间/h"]+[f"{r*100:g} cm" for r in
↪ table_fixed]+["药材表面","表面半径/cm"])
127     for t,row,rr in zip(table_times,table_C,table_R):

```

```

128         writer.writerow([f"{t/3600:.4f}"]+["" if np.isnan(v) else f"{v:.4f}" for v in
        ↪ row]+[f"{rr*100:.6f}"]])
129 with (RESULTS_DIR/"q4_radius_output.csv").open("w",newline="",encoding="utf-8-sig") as f:
130     writer=csv.writer(f);writer.writerow([" 时间/s"," 半径/cm"]);writer.writerows(zip(tt,radii*100))
131 plt=setup_mpl();FIGURES_DIR.mkdir(exist_ok=True)
132 fig,ax=plt.subplots(figsize=(6.2,3.8))
133 ax.plot(tt/3600,YY[:,n],label=" 中心 (全域最大值) ")
134 ax.plot(tt/3600,YY[:, -1],label=" 表面")
135 ax.axhline(.15,color="0.3",linestyle="--",label=" 达标阈值")
136 ax.axvline(critical/3600,color="0.6",linestyle=":")
137 ax.set(xlabel=" 时间 / h",ylabel=" 干基含水率 / (kg/kg)",xlim=(0,t_rep/3600))
138 ax.legend(frameon=False);fig.tight_layout()
139 fig.savefig(FIGURES_DIR/"q4_C_history.pdf");fig.savefig(FIGURES_DIR/"q4_C_history.png",dpi=160);plt.cla
    ↪ ose(fig)
140 fig,ax=plt.subplots(figsize=(6.2,3.8))
141 for t,i,rr in zip(table_times,table_ids,table_R):
142     ax.plot(a["xi"]*rr*100,YY[i,n:],label=f"{t/3600:.2f} h")
143 ax.set(xlabel=" 到药材中心的距离 / cm",ylabel=" 干基含水率 / (kg/kg)")
144 ax.legend(frameon=False,ncol=3,fontsize=7);fig.tight_layout()
145 fig.savefig(FIGURES_DIR/"q4_C_profiles.pdf");fig.savefig(FIGURES_DIR/"q4_C_profiles.png",dpi=160);plt.cla
    ↪ ose(fig)
146 # Small visible table companion; this is a result record, not paper prose.
147 headers=[" 时间/h","0 cm","0.5 cm","1.0 cm","1.5 cm","2.0 cm"," 表面"," 表面半径/cm"]
148 lines=["# 第四问表6 (基线结果, 待审) ", "", " | "+" | ".join(headers)+" | ",
    ↪ " | "+" | ".join(["---"*len(headers))+ " | "]
149 for t,row,rr in zip(table_times,table_C,table_R):
150     lines.append(" | "+" | ".join([f"{t/3600:.4f}"]+["—" if np.isnan(v) else f"{v:.4f}" for v in
    ↪ row]+[f"{rr*100:.6f}"])+ " | ")
151 lines += ["", "—表示该固定半径在当前药材区域之外。表面列对应各时刻真实外半径。末行以未舍入值通过严格阈值检查;
    ↪ 四位显示可能仍为0.1500。"]
152 (REPORTS_DIR/"Q4_TABLE6.md").write_text("\n".join(lines)+"\n",encoding="utf-8")
153 print(json.dumps({k:summary[k] for k in ("critical_h","report_h","report_radius_cm","report_maxima","e
    ↪ vent_spatial_difference_s","pass")),ensure_ascii=False),flush=True)
154
155
156 if __name__=="__main__":
157     main()

```

B.14 code/plot_q3_results.py

```

1 """Redraw Q3 profiles and the event window using validated saved states only."""
2 import hashlib
3 import json
4 from pathlib import Path
5
6 import numpy as np
7
8 from problem3 import setup_mpl
9 from q3_closeout import require_q3_delivery
10 from utils import ROOT, RESULTS_DIR, FIGURES_DIR
11
12
13 def main():
14     out = RESULTS_DIR / 'diagnostics/q3_delivery'
15     out.mkdir(parents=True, exist_ok=True)
16     validation = json.loads((RESULTS_DIR / 'q3_validation.json').read_text(encoding='utf-8'))
17     require_q3_delivery(validation, out / 'pre_plot.json')
18     z = np.load(RESULTS_DIR / 'q3_solution.npz')
19     n = int(z['n_nodes'])
20     plt = setup_mpl()
21     fig, ax = plt.subplots(figsize=(6.4, 4.8))
22     for hour in z['table_hours']:
23         if abs(hour * 3600 - float(z['t_rep'])) < 1e-7:
24             state = z['y_end']

```

```

25     else:
26         indices = np.flatnonzero(abs(z['times'] - hour * 3600) < 1e-7)
27         if len(indices) != 1:
28             raise RuntimeError('profile time missing from saved states')
29         state = z['V'][indices[0]]
30         ax.plot(z['r'] * 100, state[n:], lw=1.2, label=fr'$t={hour:g}\,\mathrm{{h}}$')
31     ax.set_xlabel('\u534a\u5f84 $r$ (cm)')
32     ax.set_ylabel('\u5e72\u57fa\u542b\u6c34\u7387 $C$ (kg/kg)')
33     ax.set_xlim(0, 2)
34     ax.legend(frameon=False, ncol=3, loc='lower center', bbox_to_anchor=(.5, 1.01))
35     fig.tight_layout()
36     fig.savefig(FIGURES_DIR / 'q3_C_profiles.pdf')
37     plt.close(fig)
38
39     critical, report = float(z['t_star']), float(z['t_rep'])
40     mask = z['times'] >= critical - 90
41     times = np.r_[z['M_hist'][mask], critical, report]
42     maxima = np.r_[z['M_hist'][mask], np.max(z['y_star'][n:]), np.max(z['y_end'][n:])]
43     fig, ax = plt.subplots(figsize=(6.4, 3.8))
44     ax.axhline(0, color='.4', lw=1, ls='--', label='$C=0.15$')
45     ax.plot(times - critical, (maxima - .15) * 1e6, marker='o', ms=3, lw=1.2,
46             label=r'$\max_i C_i-0.15$')
47     ax.axvline(0, color='.2', lw=.9, ls=':', label=fr'$t_*={critical/3600:.6f}\,\mathrm{{h}}$')
48     ax.axvline(report - critical, color='#c44242', lw=.9, ls='-.',
49             label=fr'$t_{\{\mathrm{{rep}}\}}={report/3600:.6f}\,\mathrm{{h}}$')
50     ax.set_xlabel('\u76f8\u5bf9\u4e34\u754c\u65f6\u523b $t-t_*$ (s)')
51     ax.set_ylabel(r'$\{\max_i C_i-0.15\}\, /\, 10^{-6}$ (kg/kg)')
52     ax.set_xlim(-90, 7)
53     ax.legend(frameon=False)
54     fig.tight_layout()
55     fig.savefig(FIGURES_DIR / 'q3_event_zoom.pdf')
56     plt.close(fig)
57     artifacts = ['code/plot_q3_results.py', 'results/q3_solution.npz',
58                 'results/q3_validation.json', 'figures/q3_C_profiles.pdf', 'figures/q3_event_zoom.pdf']
59     provenance = {'result_version': validation['result_version'],
60                  'event_times_s': times.tolist(), 'event_maxima': maxima.tolist(),
61                  'event_lines': 'linear guides connecting saved states; no new PDE solve',
62                  'sha256': {p: hashlib.sha256((ROOT / p).read_bytes()).hexdigest() for p in artifacts}}
63     (out / 'figure_provenance.json').write_text(json.dumps(provenance, indent=2), encoding='utf-8')
64     print('Q3 profiles and event window redrawn from saved states')
65
66
67 if __name__ == '__main__':
68     main()

```

B.15 code/verify_closeout.py

```

1  """Audit the Q1/Q2 closeout against the preserved pre-closeout deliverables.
2
3  Run with numerical Python for --gate/--modal; bundled Python for --files/--render.
4  """
5  from pathlib import Path
6  import ast
7  import hashlib
8  import json
9  import sys
10 import tempfile
11 import numpy as np
12
13 ROOT = Path(__file__).resolve().parents[1]
14 OUT = ROOT / 'results/verification/closeout'
15 ARCHIVE = ROOT / 'results/archive/pre-closeout-20260911'
16
17
18 def dump(name, report):

```

```

19     OUT.mkdir(parents=True, exist_ok=True)
20     (OUT/name).write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding='utf-8')
21     print(json.dumps(report, ensure_ascii=True, indent=2), flush=True)
22
23
24 def gate_test():
25     from delivery import make_gate, require_delivery
26     results = {}
27     with tempfile.TemporaryDirectory(dir=ROOT/'tmp') as d:
28         folder = Path(d)
29         for q in (1,2):
30             v = json.loads((ROOT/f'results/q{q}_validation.json').read_text(encoding='utf-8'))
31             baseline = v['delivery_gate']['checks']
32             assert baseline and all(baseline.values())
33             for key in baseline:
34                 checks = dict(baseline, **{key: False})
35                 sentinel = folder/'result.xlsx'
36                 sentinel.write_bytes(b'exiting approved data')
37                 try:
38                     require_delivery({'delivery_gate': make_gate(checks)}, folder/'diagnostic.json')
39                     sentinel.write_bytes(b'invalid new result')
40                 except RuntimeError:
41                     pass
42                 assert sentinel.read_bytes() == b'exiting approved data'
43                 assert key in json.loads((folder/'diagnostic.json').read_text(encoding='utf-8'))['delivery_
↳ _gate']['failed_checks']
44             require_delivery({'delivery_gate': make_gate(baseline)}, folder/'diagnostic.json')
45             # Ensure the production main itself puts the guard ahead of all writers.
46             tree = ast.parse((ROOT/f'code/problem{q}.py').read_text(encoding='utf-8'))
47             main = next(n for n in tree.body if isinstance(n, ast.FunctionDef) and n.name == 'main')
48             calls = [n for n in ast.walk(main) if isinstance(n, ast.Call)]
49             guard = next(n.lineno for n in calls if isinstance(n.func, ast.Name) and
↳ n.func.id == 'require_delivery')
50             writers = [n.lineno for n in calls if (isinstance(n.func, ast.Name) and n.func.id in
↳ ('write_result_workbook', 'write_paper_csv', 'plot_figures')) or
↳ (isinstance(n.func, ast.Attribute) and n.func.attr in ('savez', 'write_text'))]
51             assert writers and min(writers) > guard
52             results[f'q{q}'] = {'failure_cases_blocked': len(baseline), 'valid_case_passed': True,
↳ 'all_production_writers_after_guard': True}
53     dump('gate_test.json', results)
54
55
56 def files():
57     from openpyxl import load_workbook
58     report = {}
59     for q, end, table_t in
↳ [(1,1800,np.array([100,300,600,900,1200,1500,1800])),(2,10800,np.arange(1800,10801,1800))]:
60         z = np.load(ROOT/f'results/q{q}_solution.npz')
61         old = np.load(ARCHIVE/f'results/q{q}_solution.npz')
62         validation = json.loads((ROOT/f'results/q{q}_validation.json').read_text(encoding='utf-8'))
63         r = {'version': validation['result_version'], 'delivery_passed':
↳ validation['delivery_gate']['meets_delivery_gate'], 'fields':{}}
64         assert r['delivery_passed']
65         assert np.array_equal(z['times'], np.arange(1,end+1))
66         assert np.all(np.diff(z['r']) > 0)
67         assert np.allclose(z['output_radai_m'], np.arange(21)*.001, rtol=0, atol=1e-15)
68         idx = np.array([np.argmax(abs(z['r']-x)) for x in z['output_radai_m']])
69         assert np.max(abs(z['r'][idx]-z['output_radai_m'])) < 1e-14
70         wb = load_workbook(ROOT/f'results/result{q}.xlsx', read_only=True, data_only=False)
71         before = load_workbook(ARCHIVE/f'results/result{q}.xlsx', read_only=True, data_only=False)
72         assert wb.sheetnames == before.sheetnames == ['温度', '水分浓度']
73         for key, name, kind in [('T', '温度', 'temperature'), ('C', '水分浓度', 'moisture')]:
74             ws = wb[name]
75             assert (ws.max_row, ws.max_column) == (end+1, 22)
76             same = True

```

```

77     rows = ws.iter_rows()
78     header=next(rows)
79     assert np.allclose([c.value for c in header[1:]], np.arange(21)*.1,atol=1e-14,rtol=0)
80     values=[]
81     numeric=formatted=0
82     for i,row in enumerate(rows,1):
83         assert row[0].value==i
84         values.append([c.value for c in row[1:]])
85         numeric+=sum(c.data_type=='n' for c in row[1:])
86         formatted+=sum(c.number_format=='0.0000' for c in row[1:])
87     a=np.array(values)
88     assert np.isfinite(a).all()
89     assert numeric==formatted==end*21
90     assert np.max(abs(a-z[key+'_out']))<1e-12
91     for row1,row0 in zip(ws.iter_rows(),before[name].iter_rows(),strict=True):
92         if any((a.value,a.number_format,a.data_type)!=
93             (b.value,b.number_format,b.data_type) for a,b
94             ↪ in zip(row1,row0,strict=True)):
95             same=False
96         diff=float(np.max(abs(z[key+'_out']-old[key+'_out'])))
97         assert diff<=2e-5
98         assert np.array_equal(np.round(z[key+'_out'][table_t-1,:5],4),
99             ↪ np.round(old[key+'_out'][table_
100             ↪ t-1,:5],4))
101         csv=np.loadtxt(ROOT/f'results/q{q}_table_{kind}.csv',delimiter=',',skiprows=1)
102         assert np.max(abs(csv[:,1:]-z[key+'_out'][table_t-1,:5]))<=5.1e-9 # CSV intentionally stores
103             ↪ 8 decimals.
104         cp=z[key+'_checkpoints']
105         assert cp.shape==(len(z['checkpoint_times']),len(z['r']))
106         assert np.all(cp[0]==(28 if key=='T' else 2.55))
107         assert np.allclose(cp[1:,idx],z[key+'_out'][z['checkpoint_times'][1:].astype(int)-1],rtol=0,at
108             ↪ ol=1e-12)
109         if key+'_full' in z.files:
110             assert np.max(abs(z[key+'_full'][:,idx]-z[key+'_out']))<1e-12
111         r['fields'][key]={ 'output_cells':int(a.size), 'max_change_from_previous':diff, 'all_workbook_val
112             ↪ ues_types_formats_unchanged':same, 'paper_four_decimals_unchanged':True, 'excel_npz_csv_chec
113             ↪ kpoints_consistent':True}
114     wb.close();before.close()
115     r['source_sha256']={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in
116         ↪ [ROOT/f'code/problem{q}.py',ROOT/f'results/q{q}_solution.npz',ROOT/f'results/q{q}_validation.j
117         ↪ son',ROOT/f'results/result{q}.xlsx']}
118     report[f'q{q}']=r
119     dump('files.json',report)
120
121 def modal():
122     from scipy.special import hyp1f1
123     from q1_analytic import BesselDuhamel
124     report={}
125     for q in (1,2):
126         p=__import__(f'problem{q}')
127         _,env,_=getattr(p,f'load_q{q}_inputs')()
128         D=float(p.moisture_diffusivity(p.C0) if q==1 else p.moisture_diffusivity_q2(p.C0,p.T0))
129         values={}
130         coeff=env._pchip.c[:,0]
131         for terms in (120 if q==1 else 80,320,640,1280):
132             ref=BesselDuhamel(p.HM*p.R/D,D,p.R,terms)
133             moments=[hyp1f1(1,k+2,-ref.beta)/(k+1) for k in range(3)]
134             I=(env(0)-p.C0)*np.exp(-ref.beta)+coeff[2]*moments[0]+2*coeff[1]*moments[1]+3*coeff[0]*moments
135             ↪ [2]
136             exact=env(1)-I@ref.radial_modes(p.OUTPUT_RADII_M)
137             values[terms]=exact
138         ode=BesselDuhamel(p.HM*p.R/D,D,p.R,640).evaluate_many(p.OUTPUT_RADII_M,np.array([1.]),env,p.C0)[0]
139         diff=float(np.max(abs(ode-values[640])))
140         assert diff<1e-8
141         assert np.max(abs(values[640]-values[1280]))<1e-8

```

```

132     report[f'q{q}']={f'surface_C_at_1s':{str(k):float(v[-1]) for k,v in
    ↪ values.items()},f'640_vs_1280_max':float(np.max(abs(values[640]-values[1280]))),f'ode_vs_exact_cj
    ↪ onvolution_max':diff,'passed':True}
133 dump('modal.json',report)
134
135
136 def render():
137     import pypdfium2 as pdfium
138     from PIL import Image, ImageDraw
139     folder=OUT/'previews';folder.mkdir(parents=True,exist_ok=True)
140     records=[]
141     paths=sorted((ROOT/'figures').glob('q[12]*.pdf'))
142     if (ROOT/'paper/q12_closeout.pdf').exists(): paths.append(ROOT/'paper/q12_closeout.pdf')
143     for path in paths:
144         doc=pdfium.PdfDocument(str(path))
145         for i in range(len(doc)):
146             page=doc[i];bm=page.render(scale=1.5)
147             dest=folder/f'{path.stem}_{i+1}.png'
148             im=bm.to_pil().convert('RGB');im.save(dest)
149             records.append((dest,im.copy()))
150             bm.close();page.close()
151         doc.close()
152     for offset in range(0,len(records),6):
153         contact=Image.new('RGB',(1500,1800),'white');draw=ImageDraw.Draw(contact)
154         for j,(path,im) in enumerate(records[offset:offset+6]):
155             im.thumbnail((740,560));x=j%2*750;y=j//2*600
156             contact.paste(im,(x,y+25));draw.text((x+5,y+5),path.stem,fill='black')
157         contact.save(folder/f'contact_{offset//6+1}.png')
158     dump('renders.json',{'pages':[str(p.relative_to(ROOT)) for p,_ in records]})
159
160
161 if __name__=='__main__':
162     if '--gate' in sys.argv: gate_test()
163     elif '--modal' in sys.argv: modal()
164     elif '--render' in sys.argv: render()
165     else: files()

```

B.16 code/verify_q2_results.py

```

1  """Q2 audit. Original results and production solver are never overwritten."""
2  from pathlib import Path
3  import hashlib
4  import json
5  import re
6  import sys
7  import time
8  import numpy as np
9
10 ROOT = Path(__file__).resolve().parents[1]
11 OUT = ROOT / 'results/verification/q2'
12 OUT.mkdir(parents=True, exist_ok=True)
13
14 def dump(name, data):
15     (OUT/name).write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding='utf-8')
16     print(name, json.dumps(data, ensure_ascii=True), flush=True)
17
18 def files():
19     from openpyxl import load_workbook
20     import pypdfium2 as pdfium
21     from PIL import Image, ImageDraw
22     paths = [ROOT/'data/A.pdf', ROOT/'data/附件/附件 1.xlsx', ROOT/'data/附件/附件 3/result2.xlsx']
23     paths += list((ROOT/'results').glob('q2*')) + [ROOT/'results/result2.xlsx']
24     paths += list((ROOT/'figures').glob('q2*.pdf'))
25     paths += [ROOT/'code'/n for n in
    ↪ ['problem2.py','radial_coupled.py','appendix3.py','utils.py','radial_fvm.py']]

```

```

26 hashes = {str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in paths}
27 z = np.load(ROOT/'results/q2_solution.npz')
28 v = json.loads((ROOT/'results/q2_validation.json').read_text(encoding='utf-8'))
29 report = dict(source_sha256=hashes, npz_keys=z.files, workbook={}, tables={}, figures=[])
30 report['axes'] = dict(times_ok=bool(np.array_equal(z['times'],np.arange(1,10801))),
31                       grid_increasing=bool(np.all(np.diff(z['r'])>0)), n_nodes=len(z['r']))
32 template = load_workbook(ROOT/'data/附件/附件 3/result2.xlsx',read_only=True)
33 report['template'] = {s.title:dict(dimensions=[s.max_row,s.max_column], first_rows=list(s.values)[:4])
34 ↪ for s in template}
35 template.close()
36 wb = load_workbook(ROOT/'results/result2.xlsx',read_only=True,data_only=False)
37 report['sheet_order_ok'] = wb.sheetnames == [' 温度', ' 水分浓度']
38 src = (ROOT/'reports/RESULTS_REPORT.md').read_text(encoding='utf-8').split('## 问题二结果')[1]
39 lines = [x for x in src.splitlines() if re.match(r'^\s*[\d\.\d\s\,]*$')]
40 for k,(sheet,key,filename) in enumerate([(' 温度','T','temperature'),(' 水分浓度','C','moisture')]):
41     ws=wb[sheet]; it=ws.iter_rows(); header=next(it)
42     values=[]; times=[]; numeric=formatted=0
43     for row in it:
44         times.append(row[0].value)
45         values.append([c.value for c in row[1:]])
46         numeric += sum(c.data_type=='n' for c in row[1:])
47         formatted += sum(c.number_format=='0.0000' for c in row[1:])
48     a=np.array(values,dtype=float); expected=z[key+'_out']
49     report['workbook'][key] = dict(dimensions=[ws.max_row,ws.max_column],
50     times_ok=bool(np.array_equal(times,z['times'])),
51     radii_ok=bool(np.allclose([c.value for c in header[1:]],np.arange(21)*.1,rtol=0,atol=1e-14)),
52     all_finite=bool(np.isfinite(a).all()), numeric_cells=numeric, formatted_cells=formatted,
53     max_difference_vs_npz=float(np.max(abs(a-expected))), minimum=float(a.min()),
54     ↪ maximum=float(a.max()))
55     expected=expected[np.arange(1800,10801,1800)-1,:5]
56     csv=np.loadtxt(ROOT/'results/q2_table_{filename}.csv',delimiter=',',skiprows=1)
57     printed=np.array([[float(s.strip()) for s in line.split('|')[2:-1]] for line in lines[6*k:6*k+6]])
58     j=np.array([v['paper_table_'+key][f'h{h:g}_r{r:g}'] for h in np.arange(.5,3.1,.5) for r in
59     ↪ np.arange(5)*.5]).reshape(6,5)
60     report['tables'][key]=dict(csv_max_difference=float(abs(csv[:,1:]-expected).max()),
61     csv_hours_ok=bool(np.array_equal(csv[:,0],np.arange(.5,3.1,.5))),
62     report_four_decimal_matches=bool(np.array_equal(printed,np.round(expected,4))),
63     json_max_difference=float(abs(j-expected).max()))
64 wb.close()
65 previews=[]
66 for p in sorted((ROOT/'figures').glob('q2*.pdf')):
67     doc=pdfium.PdfDocument(str(p)); texts=[]
68     for i in range(len(doc)):
69         pg=doc[i]; tp=pg.get_textpage(); texts.append(tp.get_text_range()); tp.close()
70         bm=pg.render(scale=2); im=bm.to_pil().convert('RGB'); im.save(OUT/'f'{p.stem}_{i+1}.png')
71         previews.append((p.stem,im.copy())); bm.close(); pg.close()
72     report['figures'].append(dict(name=p.name,pages=len(doc),text='\n'.join(texts)))
73     doc.close()
74 contact=Image.new('RGB',(1600,1600),'white'); draw=ImageDraw.Draw(contact)
75 for i,(name,im) in enumerate(previews):
76     im.thumbnail((790,480)); x=i%2*800; y=i//2*530
77     contact.paste(im,(x,y+25));draw.text((x+10,y+5),name,fill='black')
78 contact.save(OUT/'figure_contact.png')
79 doc=pdfium.PdfDocument(str(ROOT/'data/A.pdf'))
80 for i in [1,3]:
81     pg=doc[i]; bm=pg.render(scale=1.5); bm.to_pil().save(OUT/'f'problem_page_{i+1}.png');
82     ↪ bm.close();pg.close()
83 doc.close()
84 report['source_files_unchanged']=all(hashlib.sha256((ROOT/p).read_bytes()).hexdigest()==h for p,h in
85 ↪ hashes.items())
86 dump('file_audit.json',report)
87
88 def numerical():
89     import problem2 as p
90     start=time.perf_counter()

```

```

86 z=np.load(ROOT/'results/q2_solution.npz')
87 Ta,Ca,knots=p.load_q2_inputs()
88 report={'scope':'G4 reproduction, G4 time refinement, G8 space refinement, all 10800 seconds',
89 ↪ 'runs':{}}
90 for tag,level,rtol,at,step in
91 ↪ [ ('G4_reproduction',4,1e-9,1e-8,1.), ('G4_time',4,2e-10,2e-9,.5), ('G8_space',8,1e-9,1e-8,1.)]:
92     print('Starting',tag,flush=True)
93     f=p.CoupledRadialFVM(p.graded_radial_nodes(level)); n=f.n_nodes
94     y0=f.pack(np.full(n,p.T0),np.full(n,p.C0))
95     ts,Y=p.integrate_coupled(f,y0,p.T_END,Ta,Ca,rtol,at,step)
96     T,C=p.extract_output(f,Y[1:])
97     result={key:**p.compare_on_output(z[key+'_out'],a,ts[1:]),**p.four_decimal_stats(z[key+'_out'],a,
98 ↪ ts[1:])} for key,a in [ ('T',T), ('C',C)]
99     if tag=='G4_reproduction':
100         result['conservation_C']=p.conservation_moisture(f,ts,Y,Ca)
101         result['conservation_T']=p.conservation_heat(f,ts,Y,Ta)
102         result['full_grid_bounds']={ 'T_min':float(Y[:,n].min()), 'T_max':float(Y[:,n].max()), 'C_min'
103 ↪ :float(Y[:,n].min()), 'C_max':float(Y[:,n].max())}
104         cp=np.array([0,1,60,100,1800,3600,5400,7200,9000,10800])
105         np.savez_compressed(OUT/'G4_checkpoints.npz',times=ts[cp],r=f.r,Y=Y[cp])
106     if tag=='G8_space':
107         changed=[]
108         for key,a in [ ('T',T), ('C',C)]:
109             for ti in np.arange(1800,10801,1800)-1:
110                 for ri in range(0,21,5):
111                     b=float(z[key+'_out'][ti,ri]); q=float(a[ti,ri])
112                     if round(b,4)!=round(q,4):
113                         ↪ changed.append(dict(field=key,t_s=int(ti+1),r_cm=ri*.1,G4=b,G8=q))
114         result['paper_changed_details']=changed
115         np.savez_compressed(OUT/f'{tag}_outputs.npz',times=ts[1:],T_out=T,C_out=C)
116         report['runs'][tag]=result
117         dump('numerical_audit.json',report)
118         del Y,T,C,f
119     report['manufactured_solution']=p.run_mms(4)
120     # A short frozen-D check separates modal truncation from early FVM error.
121     f=p.CoupledRadialFVM(p.graded_radial_nodes(4)); n=f.n_nodes
122     D0=float(p.moisture_diffusivity_q2(p.C0,p.T0))
123     ts,Y=p.integrate_coupled(f,f.pack(np.full(n,p.T0),np.full(n,p.C0)),100.,Ta,Ca,2e-10,2e-9,.1,D_const=D0)
124     _,Cf=p.extract_output(f,Y[1:]); refs={}
125     for terms in [80,160,320]:
126         ref=p.BesselDuhamel(p.HM*p.R/D0,D0,p.R,n_terms=terms).evaluate_many(p.OUTPUT_RADII_M,ts[1:],Ca,p.C
127 ↪ 0)
128         refs[terms]=ref
129         report.setdefault('frozen_D_early',{str(terms)}[dict(max_abs=float(abs(Cf-ref).max()),t1_surfac
130 ↪ e_numeric=float(Cf[0,-1]),t1_surface_analytic=float(ref[0,-1]))
131     report['frozen_D_early']['160_vs_320_max_abs']=float(abs(refs[160]-refs[320]).max())
132     report['elapsed_s']=time.perf_counter()-start
133     dump('numerical_audit.json',report)
134
135 def modal():
136     # Closed integration of the first PCHIP derivative polynomial at t=1 s.
137     # This avoids the time integrator used by the production modal reference.
138     import problem2 as p
139     from scipy.special import hyp1f1
140     _,env,_=p.load_q2_inputs()
141     coeff=env._pchip.c[:,0]
142     D0=float(p.moisture_diffusivity_q2(p.C0,p.T0))
143     values={}
144     for n in [80,160,320,640]:
145         ref=p.BesselDuhamel(p.HM*p.R/D0,D0,p.R,n_terms=n)
146         beta=ref.beta
147         moments=[hyp1f1(1,k+2,-beta)/(k+1) for k in range(3)]
148         modes=(float(env(0))-p.C0)*np.exp(-beta)+coeff[2]*moments[0]+2*coeff[1]*moments[1]+3*coeff[0]*mome
149 ↪ nts[2]

```

```

142     values[str(n)]=float(env(1)-modes@ref.radial_modes(np.array([p.R]))[:,0])
143     dump('modal_truncation_audit.json',dict(t_s=1,surface_C=values,
144         difference_320_vs_640=abs(values['320']-values['640']),method='exact first-interval PCHIP
        ↪ polynomial convolution, hypergeometric moments'))
145
146 if __name__=='__main__':
147     if '--numerical' in sys.argv: numerical()
148     elif '--modal' in sys.argv: modal()
149     else: files()

```

B.17 code/verify_q3_closeout.py

```

1  """Regression and exhaustive delivery checks for Q3 closeout."""
2  from __future__ import annotations
3  import copy
4  import hashlib
5  import json
6  import tempfile
7  from pathlib import Path
8  from unittest.mock import patch
9  import numpy as np
10 from openpyxl import load_workbook
11 import problem3
12 import q3_closeout
13 from utils import ROOT,RESULTS_DIR
14 from q3_balance import combine
15
16
17 def main():
18     v=json.loads((RESULTS_DIR/'q3_validation.json').read_text(encoding='utf-8'))
19     assert v['result_version']=='q3-closeout-v1'
20     assert v['delivery_gate']['meets_delivery_gate'] and all(v['delivery_gate']['checks'].values())
21     checked=[]
22     with tempfile.TemporaryDirectory(prefix='q3_gate_',dir=ROOT/'tmp') as td:
23         root=Path(td);out=root/'result3.xlsx';out.write_bytes(b'ORIGINAL')
24         for name in v['delivery_gate']['checks']:
25             bad=copy.deepcopy(v);bad['delivery_gate']['checks'][name]=False
26             # Deliberately leave the top-level flag true to test recomputation.
27             with patch.object(problem3,'RESULTS_DIR',root),patch.object(q3_closeout,'export_workbook') as
                ↪ writer:
28                 try:
29                     problem3.write_result3_xlsx(out,np.zeros((1,21)),np.array([60.]),validation=bad)
30                     raise AssertionError(f'failed gate allowed writing: {name}')
31                 except RuntimeError:
32                     pass
33                 assert not writer.called and out.read_bytes()==b'ORIGINAL'
34             checked.append(name)
35             with patch.object(q3_closeout,'DIAG',root/'diagnostics'):
36                 try:
37                     q3_closeout.publish_results(None, None, bad, None)
38                     raise AssertionError('failed gate reached production publisher')
39                 except RuntimeError:
40                     pass
41             missing=copy.deepcopy(v);del missing['delivery_gate']['checks'][name]
42             with patch.object(q3_closeout,'DIAG',root/'diagnostics'):
43                 try:
44                     q3_closeout.publish_results(None, None, missing, None)
45                     raise AssertionError('missing check reached production publisher')
46                 except RuntimeError:
47                     pass
48             empty=copy.deepcopy(v);empty['delivery_gate']['checks']={}
49             with patch.object(problem3,'RESULTS_DIR',root),patch.object(q3_closeout,'export_workbook') as
                ↪ writer:
50                 try:
51                     problem3.write_result3_xlsx(out,np.zeros((1,21)),np.array([60.]),validation=empty)

```

```

52         raise AssertionError('empty gate passed')
53     except RuntimeError:
54         pass
55     assert not writer.called
56     with patch.object(problem3, 'RESULTS_DIR', root), patch.object(q3_closeout, 'export_workbook') as
57         ↪ writer:
58         problem3.write_result3_xlsx(out, np.zeros((1, 21)), np.array([60.]), validation=v)
59         assert writer.call_count==1
60     strict=problem3.find_strict_report(lambda t:.15-1e-6*(t-100.2), 100.2, 300., margin=1e-5)
61     assert strict['t_rep_s']==int(strict['t_rep_s'])
62     assert strict['M_hour4']<.15-1e-5
63     assert abs(strict['t_hour4_h']*10000-round(strict['t_hour4_h']*10000))<1e-8
64     for evaluator, hi in [(lambda t:float('nan'), 300.), (lambda t:.15-1e-6*(t-100.2), 100.21)]:
65         try:
66             problem3.find_strict_report(evaluator, 100.2, hi)
67             raise AssertionError('invalid/out-of-range strict report passed')
68         except RuntimeError:
69             pass
70     parts=v['continuous_balances']['parts']
71     combine(parts)
72     for field, delta in [(('start_s', .001), ('M_start', 1e-5), ('E_start', 1.))]:
73         bad=copy.deepcopy(parts); bad[1][field]+=delta
74         try:
75             combine(bad)
76             raise AssertionError('discontinuous balance segments passed')
77         except ValueError:
78             pass
79     # Independent weighted RHS identity checks both signs and the B-prime term.
80     from appendix3 import heat_capacity, heat_capacity_deriv
81     from radial_coupled import CoupledRadialFVM
82     from utils import R, L, H, HM
83     fvm=CoupledRadialFVM(np.linspace(0, R, 41)); n=fvm.n_nodes
84     T=35+8*(fvm.r/R)**2; C=1.2-.5*(fvm.r/R)**2
85     y=np.r_[T, C]; dy=fvm.rhs(0, y, 50., .05); factor=2*np.pi*L
86     storage_rate=factor*np.dot(fvm.W, heat_capacity(C)*dy[:n]+heat_capacity_deriv(C)*(T-28)*dy[n:])
87     correction=factor*np.dot(fvm.W, heat_capacity_deriv(C)*(T-28)*dy[n:])
88     def energy(z):
89         return factor*np.dot(fvm.W, heat_capacity(z[n:])* (z[:n]-28))
90     finite_difference=(energy(y+1e-3*dy)-energy(y-1e-3*dy))/(2e-3)
91     assert np.isclose(finite_difference, storage_rate, rtol=1e-7)
92     assert np.isclose(storage_rate-correction, factor*R*H*(50-T[-1]), rtol=1e-12)
93     assert np.isclose(fvm.W@dy[n:], -R*HM*(C[-1]-.05), rtol=1e-12)
94     source=np.load(RESULTS_DIR/'q3_solution.npz')
95     wb=load_workbook(RESULTS_DIR/'result3.xlsx', read_only=True, data_only=True)
96     assert wb.sheetnames==[' 水分浓度']
97     rows=list(wb.active.iter_rows())
98     assert len(rows)==len(source['excel_times'])+1
99     assert all(len(row)==22 for row in rows)
100    assert np.allclose([c.value for c in rows[0][1:]], np.arange(21)*.1, atol=1e-14, rtol=0)
101    arr=np.array([[c.value for c in row] for row in rows[1:]], dtype=float)
102    delta=float(np.max(abs(arr[:, 1:]-source['C_out'])))
103    assert delta<1e-14
104    assert np.allclose(arr[:, 0], source['excel_times'], atol=1e-8, rtol=0)
105    assert all(c.number_format=='0.0000' for row in rows[1:] for c in row[1:])
106    wb.close()
107    assert np.array_equal(source['excel_times'][:-1], np.arange(60., source['t_rep'], 60.))
108    assert float(source['t_rep'])==v['t_rep_s']
109    n=int(source['n_nodes'])
110    assert source['Y'].shape==(len(source['times']), 2*n)
111    assert np.max(source['y_end'][n:]+v['moisture_margin'])<.15
112    indices=np.array([np.argmax(abs(source['r']-r)) for r in problem3.OUTPUT_RADII_M])
113    times, iy, ix=np.intersect1d(source['times'], source['excel_times'], return_indices=True)
114    assert len(times)==len(source['excel_times'])-1
115    assert np.array_equal(source['Y'][iy, n:][:, indices], source['C_out'][ix])
116    assert np.array_equal(source['Y'][iy, n:][:, indices], source['T_out'][ix])

```

```

116 assert np.array_equal(source['y_end'][n:][indices],source['C_out'][-1])
117 assert np.array_equal(source['Y'][0,:n],np.full(n,28.))
118 assert np.array_equal(source['Y'][0,n:],np.full(n,2.55))
119 assert abs(source['table_hours'][-1]-v['t_rep_h'])<1e-12
120 table=np.loadtxt(RESULTS_DIR/'q3_table_moisture.csv',delimiter=',',skiprows=1)
121 assert np.max(abs(table[:,1:]-source['C_table']))<6e-11
122 assert np.max(abs(table[:,0]-source['table_hours']))<6e-9
123 arc=RESULTS_DIR/'archive/q3-pre-closeout-20260911'
124 manifest=json.loads((arc/'manifest.json').read_text(encoding='utf-8-sig'))
125 archive_checks={p['path']:hashlib.sha256((arc/p['path']).read_bytes()).hexdigest().lower()==p['sha256']
↳ ].lower() for p in manifest}
126 assert all(archive_checks.values()),archive_checks
127 old=np.load(arc/'results/q3_solution.npz')
128 # Compare common regular 6-hour rows; the two report times differ deliberately.
129 table_changed=int(np.count_nonzero(np.round(old['C_table'][:-1],4)!=np.round(source['C_table'][:-1],4))
↳ )
130 matched,io,inew=np.intersect1d(old['excel_times'],source['excel_times'],return_indices=True)
131 common_delta=float(np.max(abs(old['C_out'][io]-source['C_out'][inew])))
132 protected=json.loads((arc/'protected.json').read_text(encoding='utf-8-sig'))
133 protected_checks={p['path']:hashlib.sha256((ROOT/p['path']).read_bytes()).hexdigest().lower()==p['sha2']
↳ 56'].lower() for p in protected}
134 assert all(protected_checks.values()),protected_checks
135 source_checks={p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest()==sha for p,sha in
↳ v['source_sha256'].items()}
136 assert all(source_checks.values()),source_checks
137 js=json.loads((q3_closeout.DIAG/'js_gate_test.json').read_text(encoding='utf-8'))
138 assert js['pass'] and set(js['required_checks'])==q3_closeout.REQUIRED_CHECKS
139 baseline={}
140 for q in (1,2,3,4):
141     data=json.loads((RESULTS_DIR/f'q{q}_validation.json').read_text(encoding='utf-8'))
142     gate=data.get('delivery_gate',{})
143     baseline[f'q{q}']=bool(gate.get('meets_delivery_gate',data.get('pass',False)))
144 assert all(baseline.values()),baseline
145 report_rows=[]
146 for line in (ROOT/'reports/Q3_VERIFY_REPORT.md').read_text(encoding='utf-8').splitlines():
147     cells=[c.strip() for c in line.strip('|').split('|')]
148     if not line.startswith('|') or len(cells)!=6:
149         continue
150     try:
151         report_rows.append([float(c) for c in cells])
152     except ValueError:
153         continue
154 report_table=np.asarray(report_rows)
155 assert report_table.shape==(len(source['table_hours']),6)
156 assert np.allclose(report_table[:,0],source['table_hours'],rtol=0,atol=1e-10)
157 assert np.array_equal(report_table[:,1:],np.round(source['C_table'],4))
158 provenance=json.loads((RESULTS_DIR/'diagnostics/q3_delivery/figure_provenance.json').read_text(encodin
↳ g='utf-8'))
159 assert all(hashlib.sha256((ROOT/p).read_bytes()).hexdigest()==sha for p,sha in
↳ provenance['sha256'].items())
160 visual=json.loads((RESULTS_DIR/'diagnostics/q3_delivery/visual_review.json').read_text(encoding='utf-8')
↳ )
161 assert visual['pass']
162 figure_hashes={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest()
↳ for p in sorted((ROOT/'figures').glob('q3_*.pdf'))}
163 assert len(figure_hashes)==5
164 prior=json.loads((RESULTS_DIR/'verification/closeout/files.json').read_text(encoding='utf-8'))
165 prior_hashes={path:hashlib.sha256((ROOT/path).read_bytes()).hexdigest()==sha
↳ for q in ('q1','q2') for path,sha in prior[q]['source_sha256'].items()}
166 assert all(prior_hashes.values()),prior_hashes
167 audit={'pass':True,'gate_failures_blocked':checked,'missing_required_checks_blocked':checked,
170 'publisher_failure_blocked':True,'js_gate_test':js,'balance_continuity_regression':True,
171 'weighted_rhs_balance_identity':True,'source_hashes_match':source_checks,

```

```

172         'internal_fields_and_initial_state_match':True,'empty_gate_blocked':True,'valid_gate_reaches_wr|
↪ iter':True,
173         'strict_report_rounding_regression':True,'workbook_rows_including_header':len(rows),'columns':2|
↪ 2,
174         'export_max_abs_difference':delta,'regular_table_four_decimal_changes_from_old':table_changed,
175         'common_excel_max_C_change_from_old':common_delta,'old_critical_h':float(old['t_star'])/3600,
176         'new_critical_h':v['t_star_h'],'old_report_h':float(old['t_rep'])/3600,'new_report_h':v['t_rep_|
↪ h'],
177         'protected_files_unchanged':protected_checks,'archive_hashes_match':archive_checks,
178         'q12_prior_evidence_hashes_match':prior_hashes,'four_question_numerical_gates':baseline,
179         'report_table_matches_source':True,'figure_provenance_matches':True,
180         'figure_sha256':figure_hashes,'visual_review':visual,
181         'result3_sha256':hashlib.sha256((RESULTS_DIR/'result3.xlsx').read_bytes()).hexdigest()}
182     (RESULTS_DIR/'q3_closeout_delivery_validation.json').write_text(json.dumps(audit,ensure_ascii=False,in|
↪ dent=2),encoding='utf-8')
183     print(json.dumps(audit,ensure_ascii=False),flush=True)
184
185
186 if __name__=='__main__':main()

```

B.18 code/verify_q4_delivery.py

```

1  """Read-only exhaustive audit of the exported Q4 workbook."""
2  import hashlib
3  import json
4  from pathlib import Path
5  import numpy as np
6  from openpyxl import load_workbook
7  from problem4 import OUT
8  from utils import RESULTS_DIR
9
10 payload=json.loads((OUT/'excel_payload.json').read_text(encoding='utf-8'))
11 path=RESULTS_DIR/'result4.xlsx'
12 wb=load_workbook(path,read_only=True,data_only=True)
13 assert wb.sheetnames==['Sheet1'],wb.sheetnames
14 ws=wb.active
15 rows=list(ws.iter_rows())
16 assert len(rows)==len(payload['rows'])+1
17 actual_columns=max(map(len,rows))
18 assert actual_columns==len(payload['headers'])
19 assert all(len(row)==actual_columns for row in rows)
20 assert [c.value for c in rows[0]]==payload['headers']
21 max_error=0.;blanks=0;numeric=0
22 for row,expected in zip(rows[1:],payload['rows']):
23     for j,(cell,value) in enumerate(zip(row,expected)):
24         if value is None:
25             assert cell.value is None,(cell.coordinate,cell.value)
26             blanks+=1
27         else:
28             assert isinstance(cell.value,(int,float)),(cell.coordinate,cell.value)
29             max_error=max(max_error,abs(cell.value-value))
30             numeric+=1
31             if j>0:
32                 assert cell.number_format=='0.0000',(cell.coordinate,cell.number_format)
33 assert max_error<1e-10,max_error
34 wb.close()
35 source=np.load(RESULTS_DIR/'q4_solution.npz')
36 times=source['times_s'][1:]
37 assert np.array_equal(times[:-1],np.arange(60.,times[-1],60.))
38 assert source['C_physical'].shape[0]==len(times)+1
39 n=len(source['xi'])
40 imax=np.argmax(source['Y'][:,n:],axis=1)
41 center_gap=float(np.max(np.max(source['Y'][:,n:],axis=1)-source['Y'][:,n]))
42 assert center_gap<1e-10,center_gap

```

```

43 audit={'pass':True,'sheet':'Sheet1','rows_including_header':len(rows),'columns':actual_columns,
44       'numeric_cells':numeric,'domain_blank_cells':blanks,'max_export_abs_error':max_error,
45       'initial_state_only_in_source':True,'final_time_s':float(times[-1]),
46       'max_position_is_center_all_saved_times':bool(np.all(imax==0)),
47       'max_center_gap_all_saved_times':center_gap,
48       'event_max_position_xi':float(source['xi'][np.argmax(source['critical_state'])[n:]]),
49       'report_max_position_xi':float(source['xi'][np.argmax(source['report_state'])[n:]]),
50       'sha256':hashlib.sha256(path.read_bytes()).hexdigest()}
51 (RESULTS_DIR/'q4_delivery_validation.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2),encod
    ↪ ing='utf-8')
52 print(json.dumps(audit,ensure_ascii=False))

```

B.19 tmp/q3_export/build.mjs

```

1 import fs from 'node:fs/promises';
2 import path from 'node:path';
3 import {fileURLToPath} from 'node:url';
4 import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
5 import {requireQ3Delivery} from './gate.mjs';
6 const here=path.dirname(fileURLToPath(import.meta.url));
7 const root=path.resolve(here,'../..');
8 const original=path.join(root,'results/result3.xlsx');
9 const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(original));
10 if(process.argv.includes('--preview-only')){
11   console.log((await
    ↪   wb.inspect({kind:'workbook,sheet,table',maxChars:1200,tableMaxRows:3,tableMaxCols:4})).ndjson);
12   const view=await wb.render({sheetName:' 水分浓度',range:'A1:H10',scale:1.3,format:'png'});
13   await fs.writeFile(path.join(here,'before.png'),new Uint8Array(await view.arrayBuffer()));
14 } else {
15   const v=JSON.parse(await
    ↪   fs.readFile(path.join(root,'results/diagnostics/q3_closeout/pre_export.json'),'utf8'));
16   requireQ3Delivery(v);
17   const data=JSON.parse(await
    ↪   fs.readFile(path.join(root,'results/diagnostics/q3_closeout/excel_payload.json'),'utf8'));
18   const sheet=wb.worksheets.getItem(data.sheet);
19   const matrix=[data.headers,...data.rows],nr=matrix.length,nc=matrix[0].length;
20   sheet.getUsedRange().clear({applyTo:'contents'});
21   sheet.getRangeByIndexes(0,0,nr,nc).values=matrix;
22   sheet.getRangeByIndexes(1,1,nr-1,nc-1).setNumberFormat('0.0000');
23   sheet.getRangeByIndexes(1,0,nr-1,1).setNumberFormat('0');
24   sheet.getRangeByIndexes(nr-1,0,1,1).setNumberFormat('0.00');
25   sheet.getRangeByIndexes(0,0,nr,1).format.columnWidth=30;
26   sheet.getRangeByIndexes(0,1,nr,nc-1).format.columnWidth=11;
27   wb.calculate();
28   console.log((await wb.inspect({kind:'table',range:'水分浓度!A1:F4',include:'values,formulas',tableMaxRow
    ↪   s:4,tableMaxCols:6,maxChars:1200})).ndjson);
29   console.log((await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#NUM!',options:{u
    ↪   seRegex:true,maxResults:10},maxChars:1000})).ndjson);
30   const preview=await wb.render({sheetName:data.sheet,sheet,range:'A1:H10',scale:1.3,format:'png'});
31   await fs.writeFile(path.join(here,'after.png'),new Uint8Array(await preview.arrayBuffer()));
32   const tail=await wb.render({sheetName:data.sheet,range:`A${nr-6}:V${nr}`,scale:1,format:'png'});
33   await fs.writeFile(path.join(here,'tail.png'),new Uint8Array(await tail.arrayBuffer()));
34   const output=await SpreadsheetFile.exportXlsx(wb);await output.save(data.path);
35   console.log(JSON.stringify({path:data.path,rows:nr,cols:nc}));
36 }

```

B.20 tmp/q3_export/gate.mjs

```

1 export const requiredChecks = [
2   'event_found', 'finite', 'strict_below_all_verified_grids',
3   'strict_below_with_empirical_margin', 'moisture_bounds', 'temperature_bounds',
4   'moisture_balance_1e-6', 'heat_balance_1e-6', 'quadrature_convergence_1e-7',
5   'space_event_60s', 'time_event_60s', 'space_full_fields_2e-5',

```

```

6   'time_full_fields_2e-5', 'table_four_decimals', 'q2_overlap',
7 ];
8
9 export function requireQ3Delivery(v) {
10  const gate = v?.delivery_gate;
11  if (v?.result_version !== 'q3-closeout-v1' || gate?.meets_delivery_gate !== true ||
12      !requiredChecks.every(name => gate?.checks?.[name] === true) ||
13      !Object.values(gate.checks).every(value => value === true)) {
14    throw new Error('Numerical validation failed. Q3 export blocked.');
```

B.21 tmp/q3_export/test_gate.mjs

```

1  import assert from 'node:assert/strict';
2  import fs from 'node:fs/promises';
3  import {requireQ3Delivery, requiredChecks} from './gate.mjs';
4  const valid = {result_version:'q3-closeout-v1', delivery_gate:{
5    checks:Object.fromEntries(requiredChecks.map(name=>[name,true])), meets_delivery_gate:true,
6  }};
7  requireQ3Delivery(valid);
8  for (const name of requiredChecks) {
9    const bad = structuredClone(valid);
10   bad.delivery_gate.checks[name] = false;
11   assert.throws(()=>requireQ3Delivery(bad));
12   delete bad.delivery_gate.checks[name];
13   assert.throws(()=>requireQ3Delivery(bad));
14 }
15 for (const bad of [{}, {...valid,delivery_gate:{checks:{},meets_delivery_gate:true}},
16   {...valid,delivery_gate:{...valid.delivery_gate,meets_delivery_gate:false}}]) {
17   assert.throws(()=>requireQ3Delivery(bad));
18 }
19 const result = {pass:true, required_checks:requiredChecks, failed_and_missing_checks_blocked:30,
20   malformed_gates_blocked:3, valid_gate_passes:true};
21 await fs.writeFile(new URL('../results/diagnostics/q3_closeout/js_gate_test.json',import.meta.url),
22   ↪ JSON.stringify(result,null,2));
23 console.log(JSON.stringify(result));
```

B.22 tmp/q4_export/build.mjs

```

1  import fs from 'node:fs/promises';
2  import path from 'node:path';
3  import {fileURLToPath} from 'node:url';
4  import {Workbook,SpreadsheetFile} from '@oai/artifact-tool';
5
6  const here=path.dirname(fileURLToPath(import.meta.url));
7  const root=path.resolve(here,'../..');
8  const audit=JSON.parse(await fs.readFile(path.join(root,'results/q4_validation.json'),'utf8'));
9  if(!audit.pass) throw new Error('Q4 numerical validation failed; export stopped.');
```

```

24 sheet.getRangeByIndexes(nr-1,0,1,1).setNumberFormat('0.00');
25 const header=sheet.getRangeByIndexes(0,0,1,nc);
26 header.format.fill='#E7EDF4';
27 header.format.font={name:'Microsoft YaHei',size:10,bold:true};
28 header.format.rowHeight=26;
29 header.format.horizontalAlignment='center';
30 sheet.getRangeByIndexes(1,0,nr-1,nc).format.horizontalAlignment='right';
31 sheet.freezePanes.freezeRows(1);
32 sheet.freezePanes.freezeColumns(1);
33 sheet.showGridLines=false;
34 wb.recalculate();
35 console.log((await wb.inspect({kind:'table',range:'Sheet1!A1:F5',include:'values,formulas',tableMaxRows:5,
↵ tableMaxCols:6,maxChars:1800})).ndjson);
36 console.log((await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#NUM!',options:{use
↵ Regex:true,maxResults:10},maxChars:1000})).ndjson);
37 const preview=await wb.render({sheetName:'Sheet1',range:'A1:H12',scale:1.4,format:'png'});
38 await fs.writeFile(path.join(here,'preview.png'),new Uint8Array(await preview.arrayBuffer()));
39 const tailPreview=await wb.render({sheetName:'Sheet1',range:`A${nr-7}:W${nr}`,scale:1,format:'png'});
40 await fs.writeFile(path.join(here,'preview_tail.png'),new Uint8Array(await tailPreview.arrayBuffer()));
41 const xlsx=await SpreadsheetFile.exportXlsx(wb);
42 await xlsx.save(path.join(root,'results/result4.xlsx'));
43 console.log(JSON.stringify({output:'results/result4.xlsx',rows:nr,columns:nc,report_h:payload.report_h}));

```